

Computer Vision & Image Analysis

Thomas de Jaeger

thomas.de-jaeger@epita.fr

Class details

- 15h: 3h*5
- Grade: 40% Labs
60% Final Project

Class organisation

- Lectures
- Labs during class
- Project

Final Project Overview: Tumor Detection in Medical Images

- **Project Goal:** Build a complete AI pipeline that detects the presence of tumors in medical images (e.g., brain MRIs) using image processing and deep learning techniques.

- **What You Will Do:**

1. Data Cleaning & Preprocessing

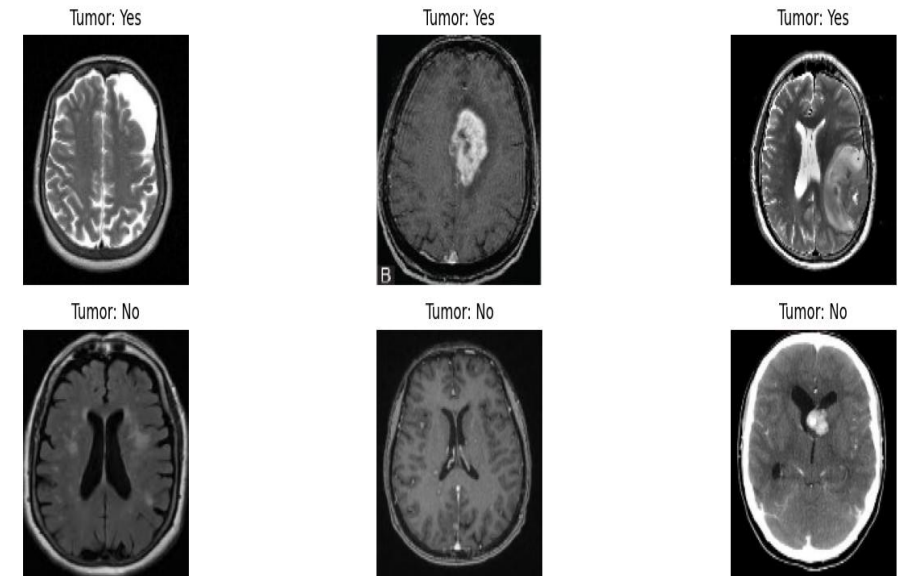
- Remove corrupt images
- Standardize orientation
- Resize and normalize images

2. Exploratory Analysis

- Visualize examples
- Understand class distribution

3. Image Processing with OpenCV

- Grayscale conversion, filtering (blur, threshold)
- Optional: region isolation (tumor contour detection)



Final Project Overview : Tumor Detection in Medical Images

- **What You Will Do:**

- 4. Model Training with PyTorch

- Train a CNN classifier (tumor vs no tumor) **YOU NEED TO TEST DIFFERENT MODELS**
- Use transformations: normalization, data augmentation
- Evaluate: accuracy, confusion matrix, predictions

- 5. Visualization & Reporting

- Show model predictions on sample test images
- Present results, explain pipeline, discuss limitations

Deliverables:

- Jupyter Notebook, google colab
- Oral presentation to explain the code and the choice
- Report pdf

Deadline:

- Wednesday 17th June 2026

you are free to apply the same pipeline to any other image dataset or domain of your choice, as long as it follows the same structure: data preprocessing, model training, evaluation, and result visualization.

BEFORE TO START SHOW YOUR DATASET!!!

Syllabus

- **Session 1:** Introduction & Basics of Computer Vision
- **Session 2:** Filtering, Edge Detection & Shape Detection
- **Session 3:** Intro to PyTorch + CNNs for Classification
- **Session 4:** Feature Extraction & Segmentation
- **Session 5:** Project presentation

Google Colab

What is Google Colab? Google Colab is a free cloud service that supports Jupyter notebooks and provides free access to computing resources including GPUs.

To use it:

- Log in with a Google account
- Open a .ipynb notebook from Drive or GitHub, or create a new one
- To enable GPU: Runtime > Change runtime type > GPU

Why we use it:

- Free and powerful
- No local install needed
- Familiar interface for Jupyter users

Upload files as a drive from your local machine

```
from google.colab import files  
input_file = files.upload()
```

Introduction & Basics of Computer Vision

Session 1

Introduction & Basics of Computer Vision

Session 1

Objectives:

- Understand how images are represented digitally (pixels, color channels)
- Learn about common color spaces: RGB, BGR, HSV, Grayscale
- Understand image histograms and histogram equalization
- Practice basic image manipulations with OpenCV

Vision

Vision is the most power sense!!

It allows us to interact with the physical world without physical contact.

Roughly 60% of our brain activity is for vision



What is Computer Vision?

Definition:

Computer vision is a field of AI that enables computers to interpret and process visual data such as images and videos.

Why using IA ?

- Human vision is intuitive, but limited by fatigue, perception, and speed.
- Computer Vision can:
 - Analyze thousands of images/videos rapidly
 - Detect patterns invisible to humans
 - Eliminate subjectivity in analysis

What is Computer Vision?

Human Vision vs Computer Vision

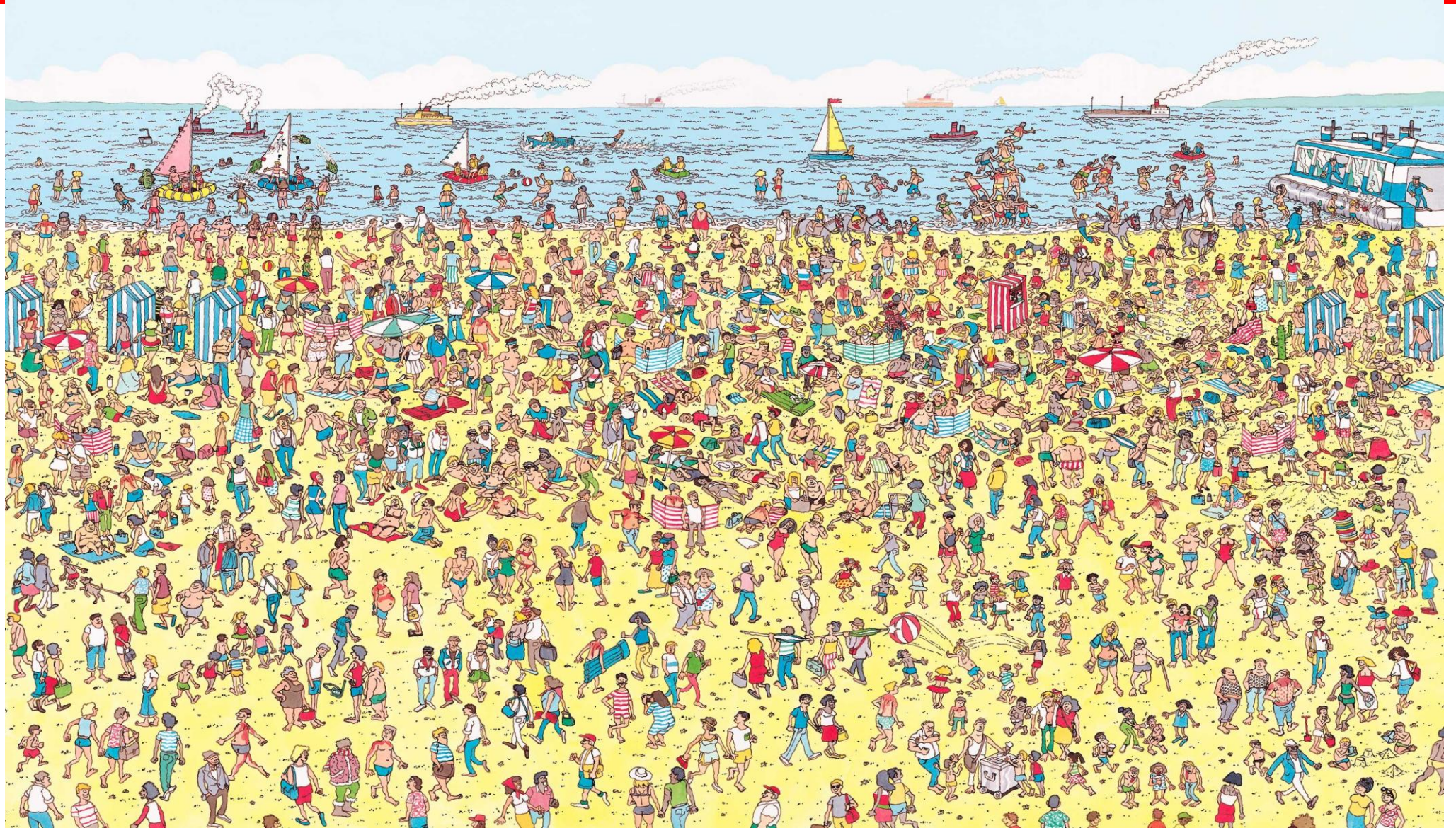


Where is Computer Vision used?

Applications across fields:

- Medical (tumor detection, retinal analysis)
- Automotive (autonomous driving)
- Security (face recognition)
- Industry (defect detection)
- Agriculture, Finance, Telecom, etc

Where is Waldo?



Where is Waldo?



AutoML Vision **BETA** waldos + ADD IMAGES LABEL STATS EXPORT DATA

IMAGES TRAIN EVALUATE PREDICT

All images 26
Labeled 26
Unlabeled 0

Type to filter...
waldo 19
waldobody 7
Add label

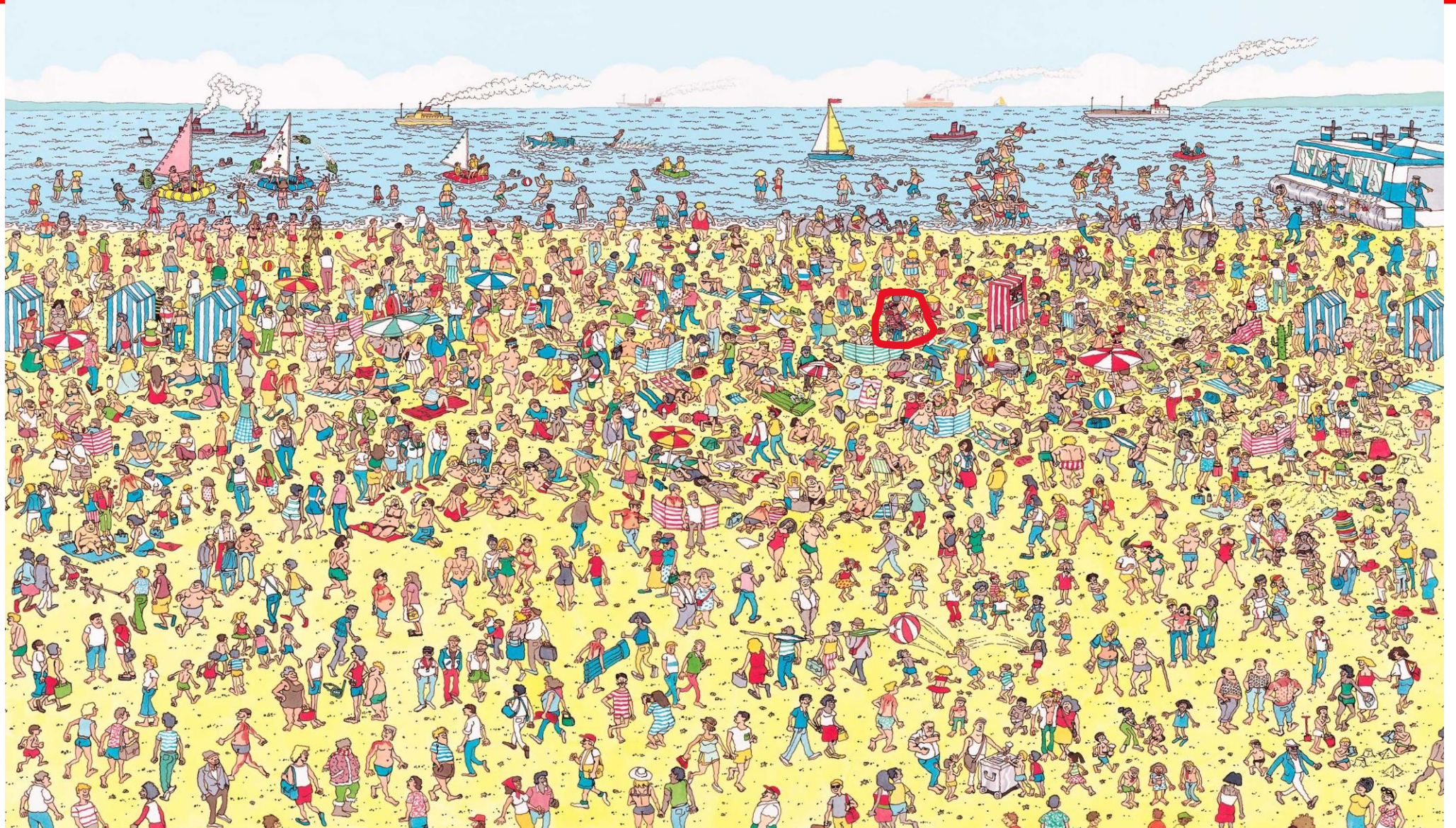
2 selected Label Delete

waldo waldo waldo waldo waldo

waldo waldo waldo waldo waldo

waldo waldo waldo waldo waldo

Where is Waldo?



How Computer vision works?

Three Key Steps:

Computer Vision System



Input



Sensing Device

Acquire Data:

Capture visual input (camera, video, sensors)



Intrepreing Device

Train Models:

Use labeled data to learn visual patterns

**Bananas,
Grapes &
Strawberry**

Output

Make Predictions:

Apply trained model to detect/analyze objects.

What can you do with CV?



What is a image?

**What eye see for a gray scale
image:**



What is a image?

If we zoom:

→ you will see some small square boxes on this image

→ **Pixels**

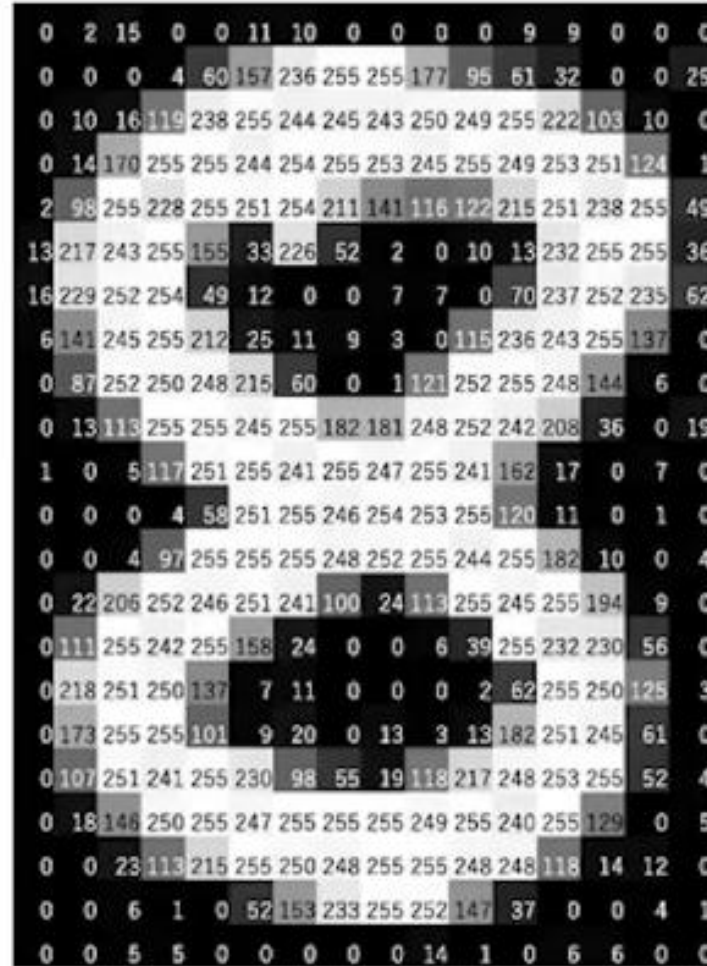
It is the unit of measurement for the size and quality of a digital image



What is a image?

What a computer sees:

- Images are represented as a 2D or 3D matrix of pixels
- In grayscale: Each pixel has intensity [0–255] (0 = black, 255 = white)
- The intensity range depends on the **number of bits used to store each pixel**.
 - 8 bits = $2^8 = 256$ values
 - 16 bits = $2^{16} = 65,535$ values



What happens to color images?



Colour Image

What happens to color images?

Almost **all colors** can be generated from the three primary colors – **Red, Green, and Blue**



Colour Image

=



Red

+



Green

+



Blue

What happens to color images?

Almost **all colors** can be generated from the three primary colors – **Red, Green, and Blue**

141	142	143	144	145
151	152	153	154	155
161	162	163	164	165
171	172	173	174	175
181	182	183	184	185
191	192	193	194	195

R

					141	142	143	144	145
					151	152	153	154	155
					161	162	163	164	165
35	36	37	38	39	173	174	175		
45	46	47	48	49	183	184	185		
55	56	57	58	59	193	194	195		
65	66	67	68	69	G				
75	76	77	78	79					
85	86	87	88	89					

R

G

31	32	33	34	35	6	77	78	79
41	42	43	44	45	6	87	88	89
51	52	53	54	55				
61	62	63	64	65				
71	72	73	74	75				
81	82	83	84	85				

B

141	142	143	144	145
151	152	153	154	155
161	162	163	164	165
171	172	173	174	175
181	182	183	184	185

G

R

G

B

For the computer one image has the dimension of N (width) * M (height) * 3 (RGB)

Algorithms extract features (edges, color regions) from pixel patterns

OpenCV

OpenCV (Open Source Computer Vision Library) is an open source computer vision and machine learning software library. OpenCV was built to provide a common infrastructure for computer vision applications and to accelerate the use of machine perception in the commercial products.

What is an image for OpenCV?

```
import cv2

image = cv2.imread('some_image.png')
print(type(image))
```

OPENCV use BGR color

<class 'numpy.ndarray'>

OpenCV Image Manipulation Essentials

Read/Display Using OpenCV

■ `cv2.imread('filename', flag)` : A function that reads an image and turns it into a Numpy object

▶ flag options:

`cv2.IMREAD_COLOR` (1) : load a color image. Any transparency of image will be neglected. It is the default flag.

`cv2.IMREAD_GRAYSCALE` (0) : load image in grayscale mode

`cv2.IMREAD_UNCHANGED` (-1) : load image including alpha channel

`cv2.IMREAD_ANYDEPTH` : Use precision as 16/32 bits or 8 bits, depending on the image

`cv2.IMREAD_ANYCOLOR` : 3 channels available, used as color images

`cv2.IMREAD_REDUCED_GRAYSCALE_2` : 1 channel, 1/2 size, grayscale application

`cv2.IMREAD_REDUCED_GRAYSCALE_4` : 1 channel, 1/4 size, grayscale application

`cv2.IMREAD_REDUCED_GRAYSCALE_8` : 1 channel, 1/8 size, grayscale application

`cv2.IMREAD_REDUCED_COLOR_2` : 3 channels, 1/2 size, using BGR images

`cv2.IMREAD_REDUCED_COLOR_4` : 3 channels, 1/4 size, using BGR images

`cv2.IMREAD_REDUCED_COLOR_8` : 3 channels, 1/8 size, using BGR images

■ `cv2.imshow('windowname', image)` : A function that shows a specific image to the window name

■ `cv2.imwrite('filename.png/jpg/...', image)` : Save image as a file

■ `cv2.cvtColor(image, flag)` : A function to change the color shape of an image

▶ flag options:

`cv2.COLOR_BGR2RGB` or 4 : change BGR to RGB

`cv2.COLOR_BGR2GRAY` or 6 : change BGR to Gray

`cv2.COLOR_RGB2GRAY` or 7 : change RGB to Gray

`cv2.COLOR_GRAY2RGB` or 8 : change Gray to RGB

e.g. `gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)`

OpenCV Image Manipulation Essentials

- **Resize:**

`resized = cv2.resize(img, (200, 300))` # Resize to specific dimensions

`resized = cv2.resize(img, None, fx=0.5, fy=0.5)` # Resize by scale

Explanation:

- (200, 300) means you explicitly set width = 200 and height = 300.
- None means OpenCV ignores fixed size and uses scaling factors:
- `fx=0.5, fy=0.5` resizes the image to 50% of its original width and height

OpenCV Image Manipulation Essentials

- **Resize:**

```
resized = cv2.resize(img, (200, 300))      # Resize to specific dimensions  
resized = cv2.resize(img, None, fx=0.5, fy=0.5)  # Resize by scale
```

- **Crop:**

```
cropped = img[50:200, 100:300] # y1:y2, x1:x2.
```

- **Rotate:**

```
(h, w) = img.shape[:2]          # Get image height and width  
center = (w // 2, h // 2)      # Compute the center  
M = cv2.getRotationMatrix2D(center, 45, 1.0) # 45° rotation, scale=1  
rotated = cv2.warpAffine(img, M, (w, h))  # Apply the rotation
```

Explanation:

- Scale factor (1.0: original size)
- Angle (Positive angle → Counter-clockwise)
- `fx=0.5, fy=0.5` resizes the image to 50% of its original width and height

OpenCV Image Manipulation Essentials

- **Resize:**

`resized = cv2.resize(img, (200, 300))` # Resize to specific dimensions

`resized = cv2.resize(img, None, fx=0.5, fy=0.5)` # Resize by scale

- **Crop:**

`cropped = img[50:200, 100:300]` # y1:y2, x1:x2.

- **Rotate:**

`(h, w) = img.shape[:2]` # Get image height and width

`center = (w // 2, h // 2)` # Compute the center

`M = cv2.getRotationMatrix2D(center, 45, 1.0)` # 45° rotation, scale=1

`rotated = cv2.warpAffine(img, M, (w, h))` # Apply the rotation

- **Flip:**

`flip_h = cv2.flip(img, 1)` # Horizontal

`flip_v = cv2.flip(img, 0)` # Vertical

`flip_both = cv2.flip(img, -1)` # Vertical & horizontal

- **SAVE**

`cv2.imwrite("output.jpg", img)`

OpenCV Drawing Functions

- **Rectangle**

`cv2.rectangle(img, (x1, y1), (x2, y2), (255, 0, 0), 2)`

- **Circle**

`cv2.circle(img, (x, y), radius, (0, 255, 0), thickness)`

(255, 0, 0): color as a tuple
> 0 Line thickness in pixels
-1 Fill the shape entirely

- **Arrowed Line**

`cv2.arrowedLine(img, (x1, y1), (x2, y2), (0, 0, 255), 2)`

- **Line**

`cv2.line(img, (x1, y1), (x2, y2), (255, 255, 0), 3)`

- **Ellipse**

`cv2.ellipse(img, (x, y), (axis1, axis2), angle, 0, 360, (255, 0, 255), 2)`

- **Text**

`cv2.putText(img, "Hello CV", (10, 500), cv2.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 255), 2): Font size`

Color Masking & Bitwise Operations

Understand how to isolate specific regions of interest using binary masks and bitwise operations in OpenCV.

Key Commands:

- **cv2.inRange(src, lowerb, upperb):** Creates a binary mask from a range of values (e.g. in HSV)
- **cv2.bitwise_and(src1, src2, mask=None):** Combines two arrays or applies a mask (e.g. to keep only certain regions)
- **cv2.bitwise_or(src1, src2, mask=None):** Combines two arrays (union of their non-zero pixels)
- **cv2.bitwise_not(src):** Inverts an array (turns 0s into 255s and vice versa)
- **cv2.bitwise_xor(src1, src2, mask=None):** Keeps only the non-overlapping (exclusive) regions of two masks or arrays: Keeps only non-overlapping areas between two masks/images

HSV Color Space

The HSV color model is an alternative to the RGB color model, designed to be more aligned with how humans perceive and categorize colors.

Hue (H): Color type and is the angle on the color wheel. 0° represents red

Saturation (S): Intensity or purity of the color. 0% = desaturated (gray), 100% = fully saturated (pure color with no gray mixed in). As saturation increases, the color becomes more vibrant.

Value (V): This represents the brightness of the color. It ranges from 0% (black) to 100% (full brightness)

Note that in OpenCV, the HSV color model different

- Hue (H): **Range: 0 to 179**
- Saturation (S): **Range: 0 to 255**
- Value (V): Range: 0 (black) to 255 (full brightness)

`hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)`

Why?

- Separates color (hue) from intensity (value)
- Easier to isolate colors for detection or masking

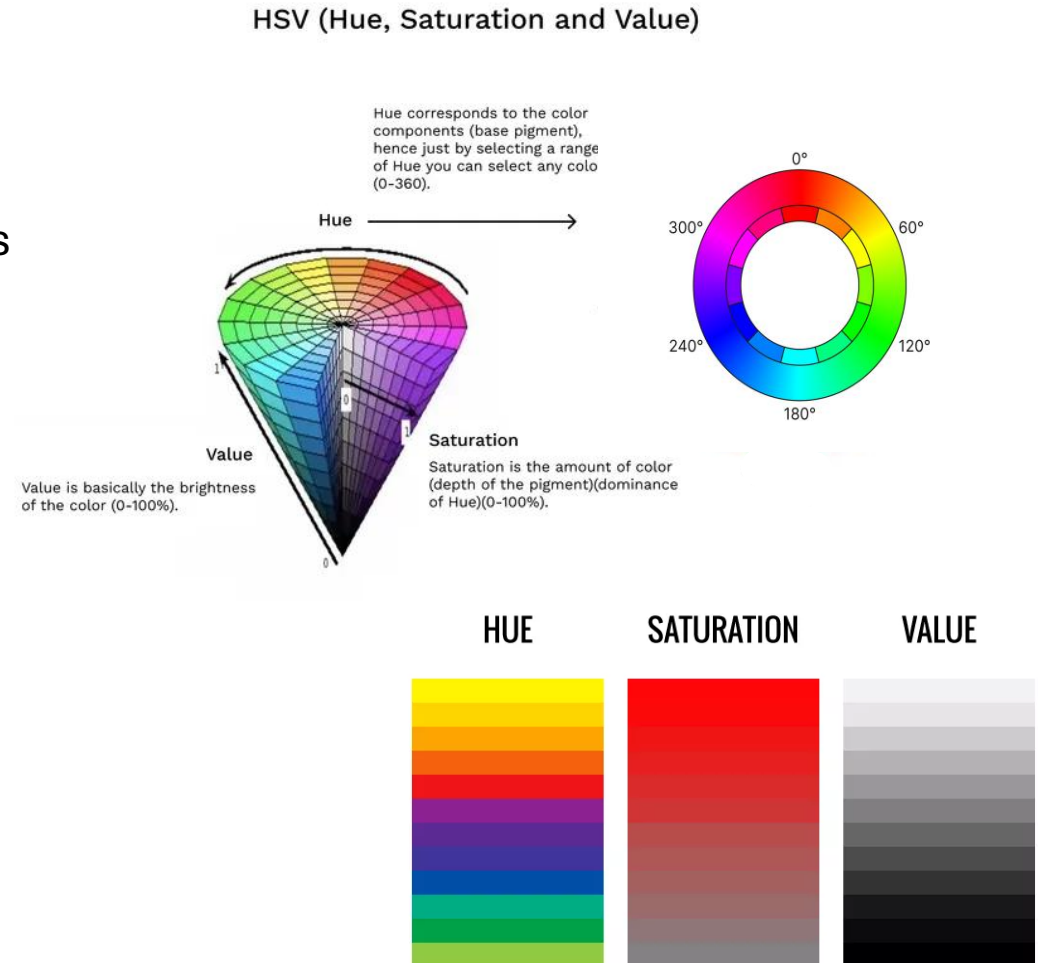


Image histogram

An image histogram is a graphical representation of the distribution of pixel intensities in an image.

The horizontal axis shows intensity values:

- Left = dark areas (shadows)
- Center = mid-tones
- Right = bright areas (highlights)

The vertical axis shows the number of pixels for each intensity level.

Use cases:

Detect if an image is underexposed (too dark) or overexposed (too bright)

Evaluate image contrast

Improve visibility using histogram equalization

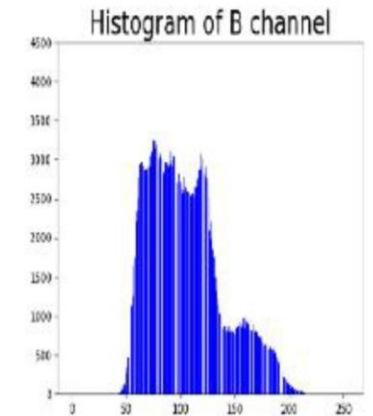
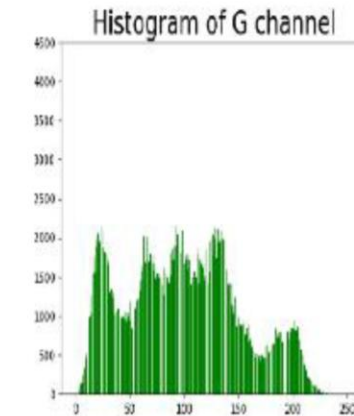
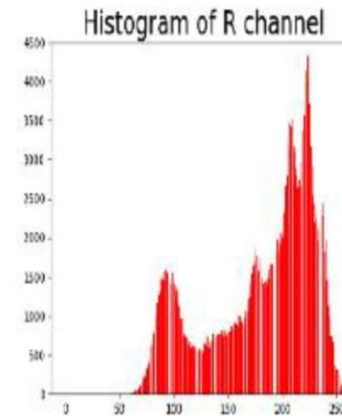
Helps with: exposure detection, thresholding, contrast enhancement

Grayscale: 0 (black) to 255 (white)

Color: histograms per channel (R, G, B)

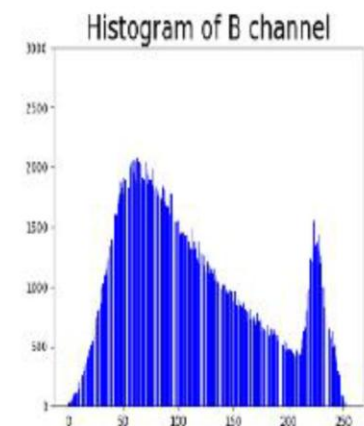
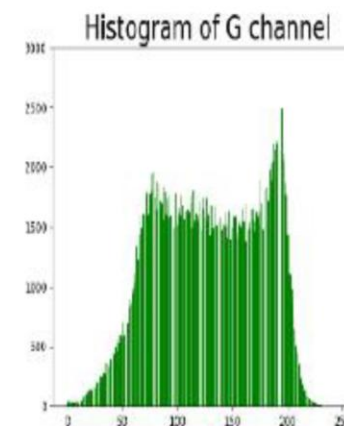
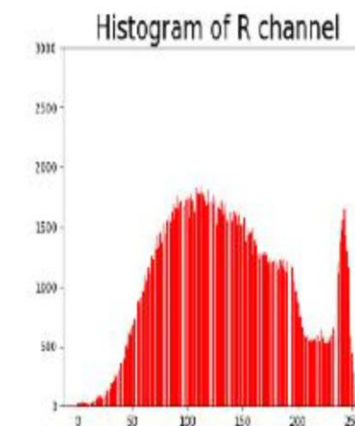
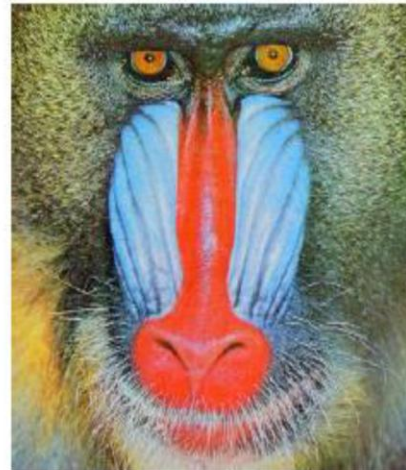
Image histogram

- Lena image shows a distribution dominated by red tones with relatively concentrated values, indicating a warm composition



(a)

- the mandrill image presents a much wider spread across all three channels, reflecting strong color diversity and higher visual complexity.



(b)

Histogram equalization

Histogram equalization is a contrast enhancement technique that spreads out the most frequent intensity values in an image, making it easier to see features in both bright and dark areas.

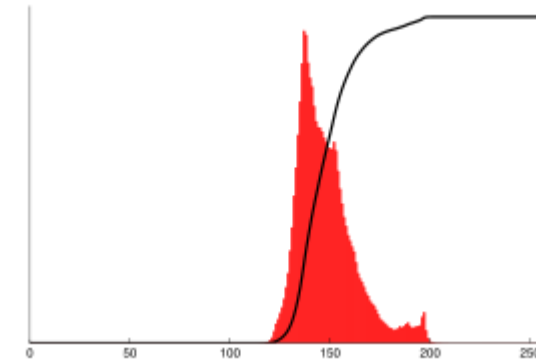
Why use it?

- Improve the contrast in low-contrast images
- Make dark areas lighter and bright areas more defined
- Useful for preprocessing before edge detection or object recognition

How it works:

- Computes the histogram of the image
- Calculates the cumulative distribution function (CDF)
- Redistributes pixel values to use the full range of intensities

Histogram equalization



Before Histogram Equalization

Histogram (red) and cumulative histogram (black)

After Histogram Equalization

Histogram (red) and cumulative histogram (black)



Histogram equalization

Standard Histogram Equalization

- Work best on uniformly grayscale images.
- Affects the entire image globally.
- Can over-amplify noise in certain cases.

equalized = cv2.equalizeHist(img)

CLAHE (Contrast Limited Adaptive Histogram Equalization):

- Applies histogram equalization locally on small regions (tiles)
- Prevents over-amplification of noise by clipping the histogram
- Better results on images with varying lighting or fine details (e.g., medical images)
- CLAHE usually provides smoother and more natural contrast.

clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))

cl1 = clahe.apply(gray)

clipLimit : Limits the contrast amplification.

High value = more contrast, more noise

Low value = smoother, safer enhancement

tileGridSize : Defines the size of the grid for dividing the image into tiles ((4, 4) : finer granularity)

For color images?

You should not apply histogram equalization (standard or CLAHE) directly on RGB channels, because:

- It may distort colors and create unnatural results.
- RGB channels are interdependent, so modifying them independently breaks their color balance.

Instead, first separate the brightness of the image from the color and then run HE:

- Convert the image to LAB color space
- L: Lightness
- A, B: Color components
- Apply CLAHE to the L channel only

Common Image Formats in Computer Vision

Format	Description	Key Properties
JPG	Compressed, very common	Lossy compression, small size, no alpha channel
PNG	Web-friendly, transparency support	Lossless compression, supports alpha channel
TIFF	Scientific/medical use	Lossless, high bit depth, supports metadata
BMP	Raw, uncompressed format	Very large files, rarely used today
DICOM	Medical imaging format	Includes metadata (patient, modality, etc.)

Definitions:

- **Lossy Compression:** Reduces file size by discarding some image data (e.g., JPEG). Results in minor quality loss.
- **Lossless Compression:** Compresses without losing any data. Can fully reconstruct the original image (e.g., PNG, TIFF).
- **Alpha Channel:** Extra channel in an image that represents transparency (e.g., PNG with 4 channels: R, G, B, Alpha).

Filtering, Edge Detection & Shape Detection

Session 2

Filtering, Edge Detection & Shape Detection

Session 2

Objectives:

- Explain how filtering affects image data and pixel values
- Implement various smoothing and sharpening filters using OpenCV
- Use gradient-based techniques to extract edges
- Understand and apply contour-based shape detection
- Detect lines and circles using Hough Transforms

What is Filtering?

Filtering is the process of modifying or enhancing an image by applying mathematical operations to pixel neighborhoods.

Main uses:

- Smoothing: Reduce noise and details
- Sharpening: Enhance contrast and highlight edges
- Edge Detection: Identify transitions in intensity



Noisy image



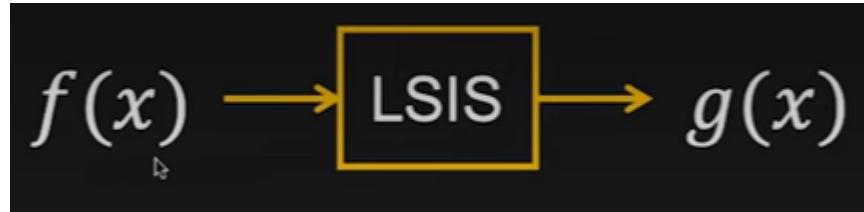
Filtered by Mean Filter



Filtered by Median Filter

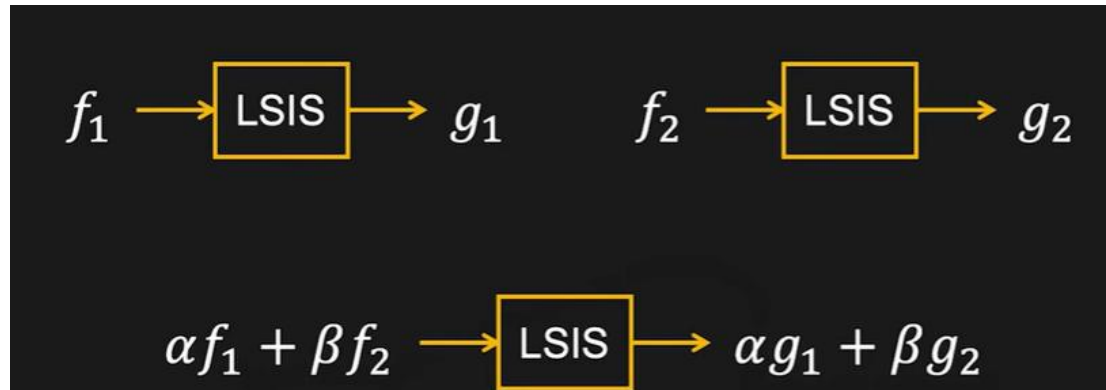
LSIS

LSIS (Linear Shift-Invariant System) is a general way to **transform an image**



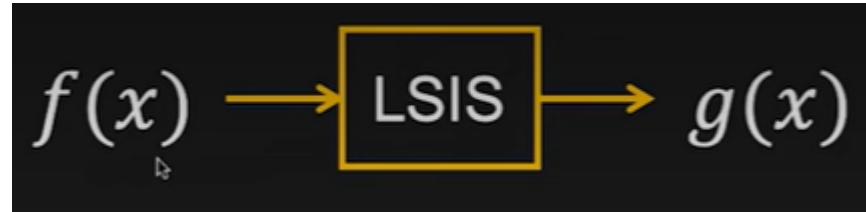
Apply consistent rules over all the image:

- **Linearity:** output is a weighted sum of inputs



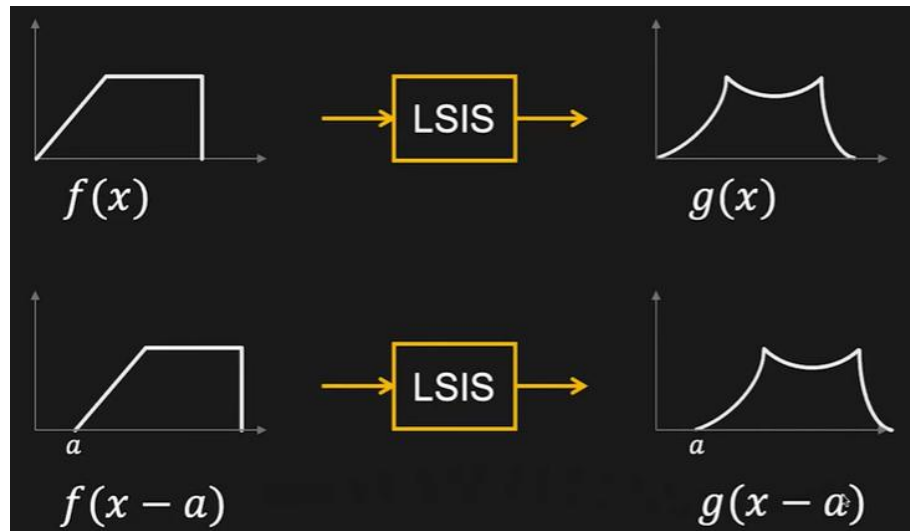
LSIS

LSIS (Linear Shift-Invariant System) is a general way to **transform an image**



Apply consistent rules over all the image:

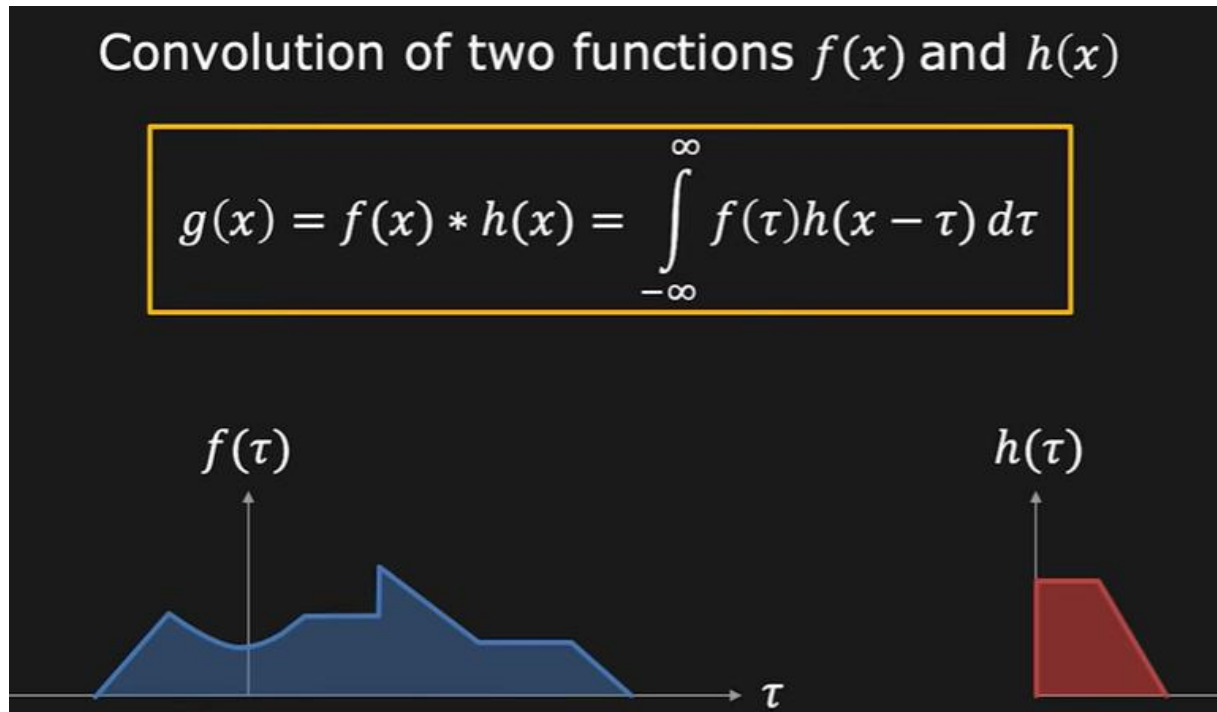
- **Linearity:** output is a weighted sum of inputs
- **Shift-invariant:** same operation everywhere in the image



Convolution?

LSIS is a way to transform an image using a consistent rule.
But how do we actually apply this rule to every pixel?

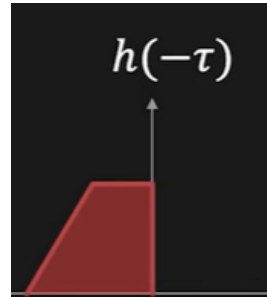
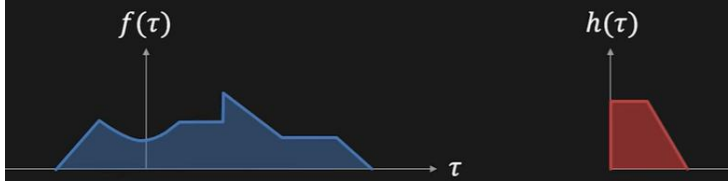
CONVOLUTION



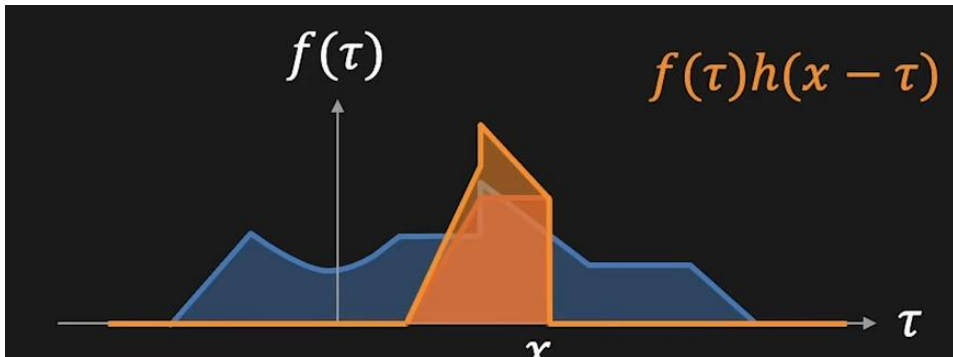
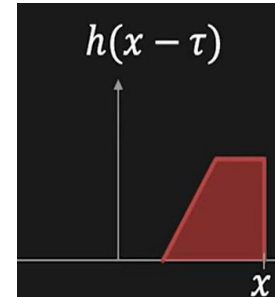
Convolution?

Convolution of two functions $f(x)$ and $h(x)$

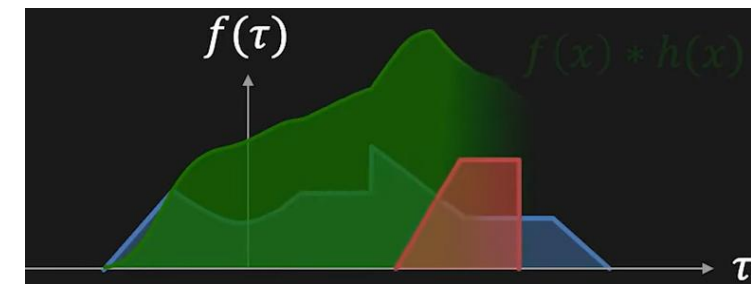
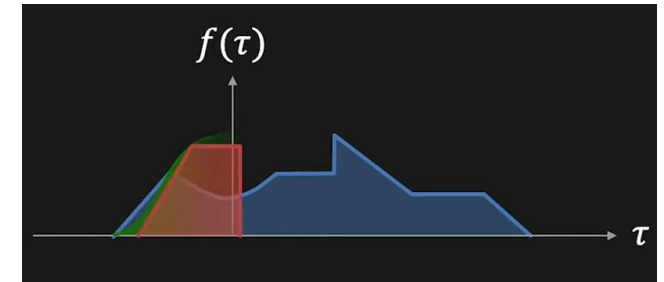
$$g(x) = f(x) * h(x) = \int_{-\infty}^{\infty} f(\tau)h(x - \tau) d\tau$$



Shift to x



Repeat this operation
throughout all the function f

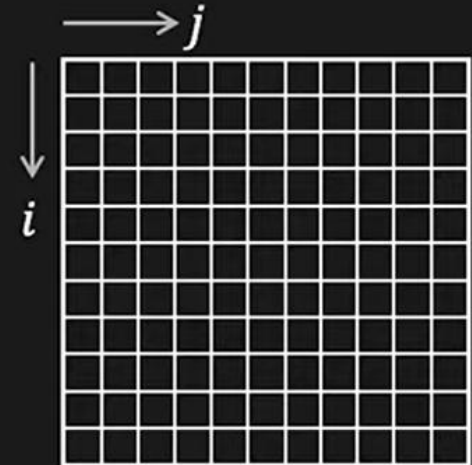
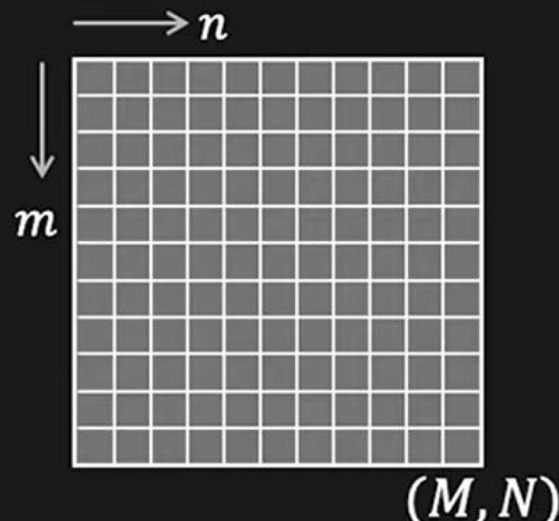


Filters implemented using convolution

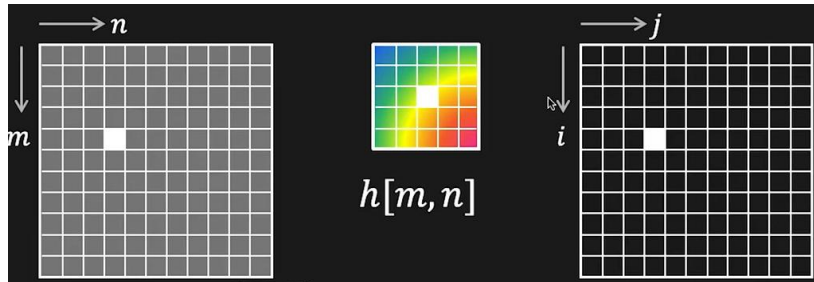
For discrete images, $f[i,j]$, using a Linear Shift-Invariant System $h[i,j]$ and obtain as output $g[i,j]$



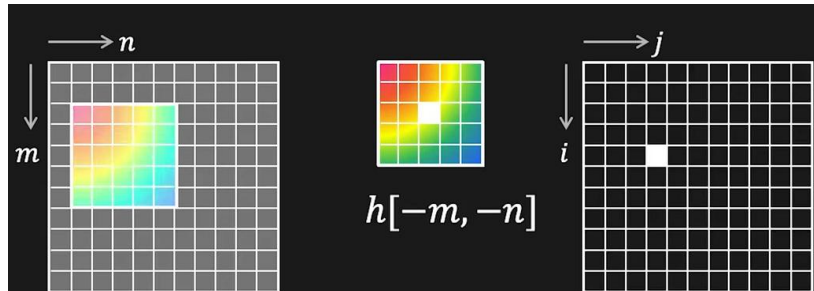
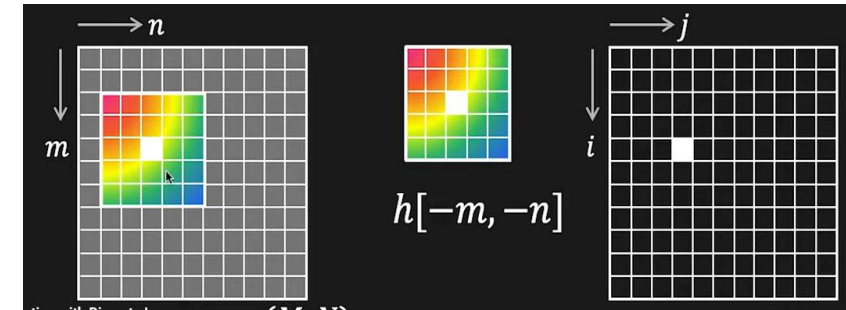
$$g[i,j] = \sum_{m=1}^M \sum_{n=1}^N f[m,n] \underbrace{h[i-m, j-n]}_{\text{"Mask," "Kernel," "Filter"}}$$



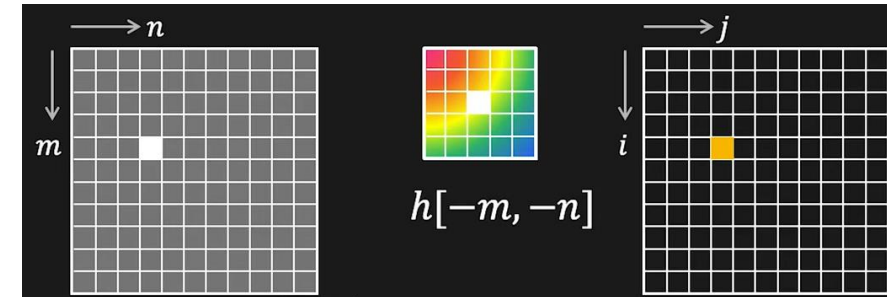
Filters implemented using convolution



Take your mask, flip it with respect to i and j and then place its center value on the pixel of interest



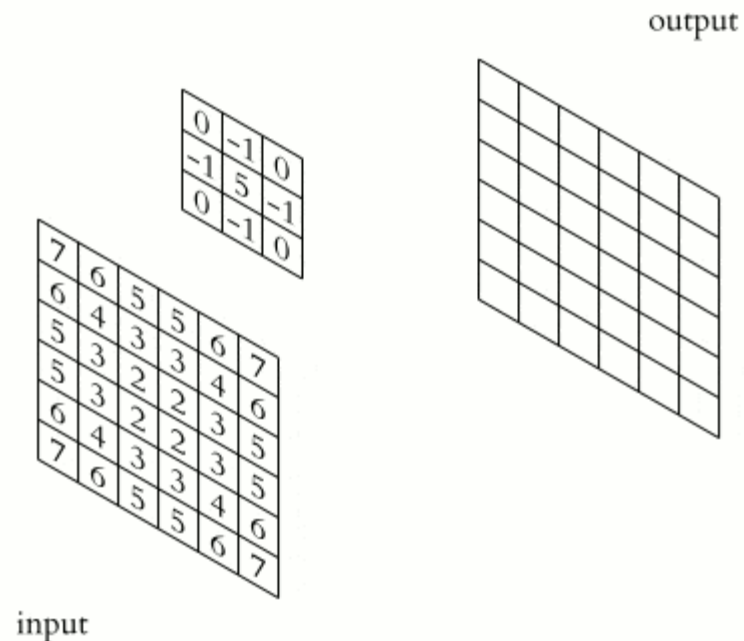
Do the product and sum it, this will be the value of $g(i, j)$



Filters implemented using convolution

Filtering works by sliding a small matrix (kernel) across the image and computing a weighted sum.

A **kernel** (also called a filter or mask) is a small matrix used to transform images. It is applied over the image using a sliding window method.



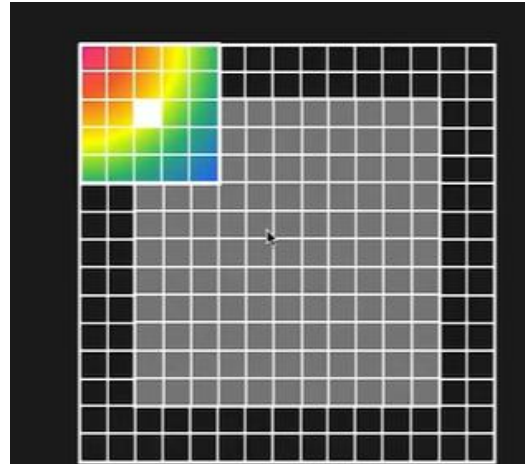
Border issues

Gray zone= your image

If you apply the kernel in a corner, part of the kernel is outside the image!!!

What to do?

- Ignore boarder, make sure that the entire kernel lies in the image
- Pad a constant value: create pixels (row, column) with a constant value (average brightness)
- Pad with reflexion, replication,...



Replication Padding

5	5	0	8	7	8	1	1
5	5	0	8	7	8	1	1
1	1	9	5	0	7	7	7
6	6	0	2	4	6	6	6
9	9	7	6	6	8	4	4
8	8	3	8	5	1	3	3
7	7	2	7	0	1	0	0
7	7	2	7	0	1	0	0

Reflection Padding

9	1	9	5	0	7	7	7
0	5	0	8	7	8	1	8
9	1	9	5	0	7	7	7
0	6	0	2	4	6	6	6
7	9	7	6	6	8	4	8
3	8	3	8	5	1	3	1
2	7	2	7	0	1	0	1
3	8	3	8	5	1	3	1

Use as axe for reflection

Example of filter

What kernel should I select if I want to an averaging filter?

- First select the number of neighbors, for example 8
- Your kernel 3x3 :
- You need to sum of all the number and divide by 9

```
[[1, 1, 1],  
 [1, 1, 1],  
 [1, 1, 1]] * 1/9
```

How to select the good kernel?

Choosing the right kernel depends on the application:

- **Noise removal:**

- Use Gaussian Blur or Median Filter (for salt-and-pepper noise)

- **Edge enhancement:**

- Use sharpening kernels (high-pass filters)

- **Edge detection:**

- Use Sobel, Prewitt, or Laplacian

- **Cartoon effect or smoothing with edge preservation:**

- Use Bilateral filter

0, -1, 0	• The center pixel gets positive weight
-1, 5, -1	
0, -1, 0	• Neighbors get negative weight

Size (3x3, 5x5...): Larger → more smoothing, less detail

You can create your own kernel!!

To design your own kernel, follow these guidelines:

Decide the purpose:	Kernel Purpose	Normalize?	Why
- Smoothing → use positive values, normalized to 1 - Sharpening → center positive, surrounding negative (high-pass) - Edge detection → sum = 0, emphasizes intensity changes	Box blur	Yes	Prevent intensity amplification
	Gaussian blur	Yes	Built-in normalization
- Kernel size: - Odd dimensions (e.g., 3x3, 5x5) so a center pixel exists	Median filter	N/A	Non-linear
	Bilateral filter	Yes	Internal normalization
- Normalize (if needed): - Ensures output pixel values stay within [0, 255] - Prevents brightening or darkening the whole image unintentionally	Sharpen	No	Keeps contrast
	Sobel, Laplacian	No	Used for gradients

Simple linear filter: Box blur filter

Purpose: Simple uniform smoothing to reduce detail and noise.

- **3x3 box kernel:**
$$K = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

- **Pixel transformation:**

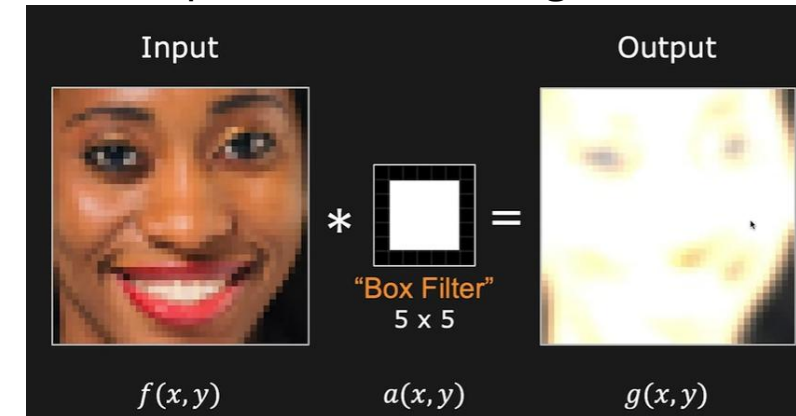
$$I'(x, y) = \frac{1}{9} \sum_{i=-1}^1 \sum_{j=-1}^1 I(x + i, y + j)$$

- **OpenCV:** `cv2.blur(img, (3, 3))` # (width, height) of kernel

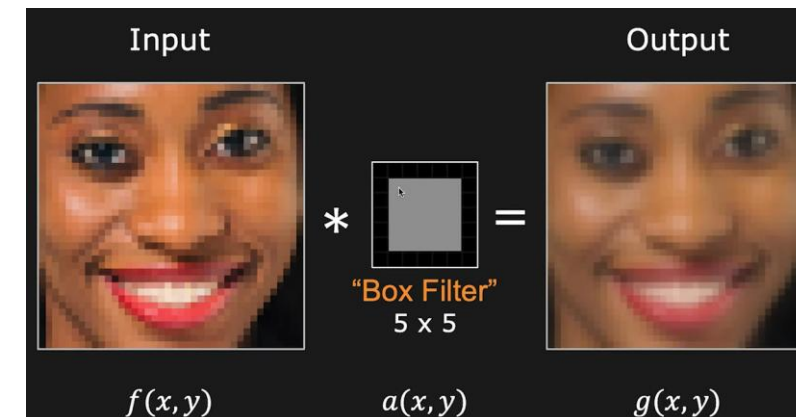
Fast and easy to implement

Blurs all edges, not ideal for preserving structure

Box filter, only 1 everywhere.
Output will be 5*5 brighter



If you divide by 25, smooth image with same brightness



Gaussian blur filter

Purpose: Low-pass filter used for reducing noise (high-frequency components) and for blurring regions of an image.

- **3x3 box kernel:**

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

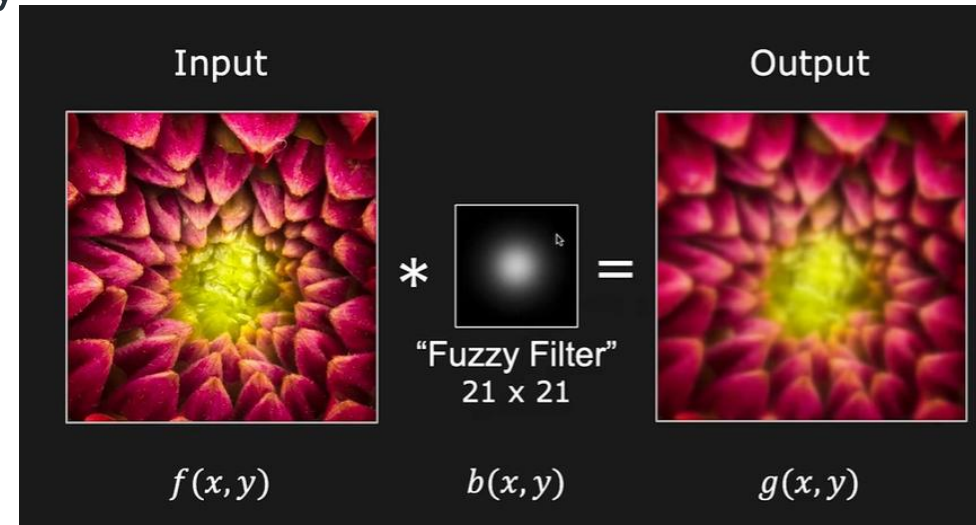
$$\frac{1}{16}$$

1	2	1
2	4	2
1	2	1

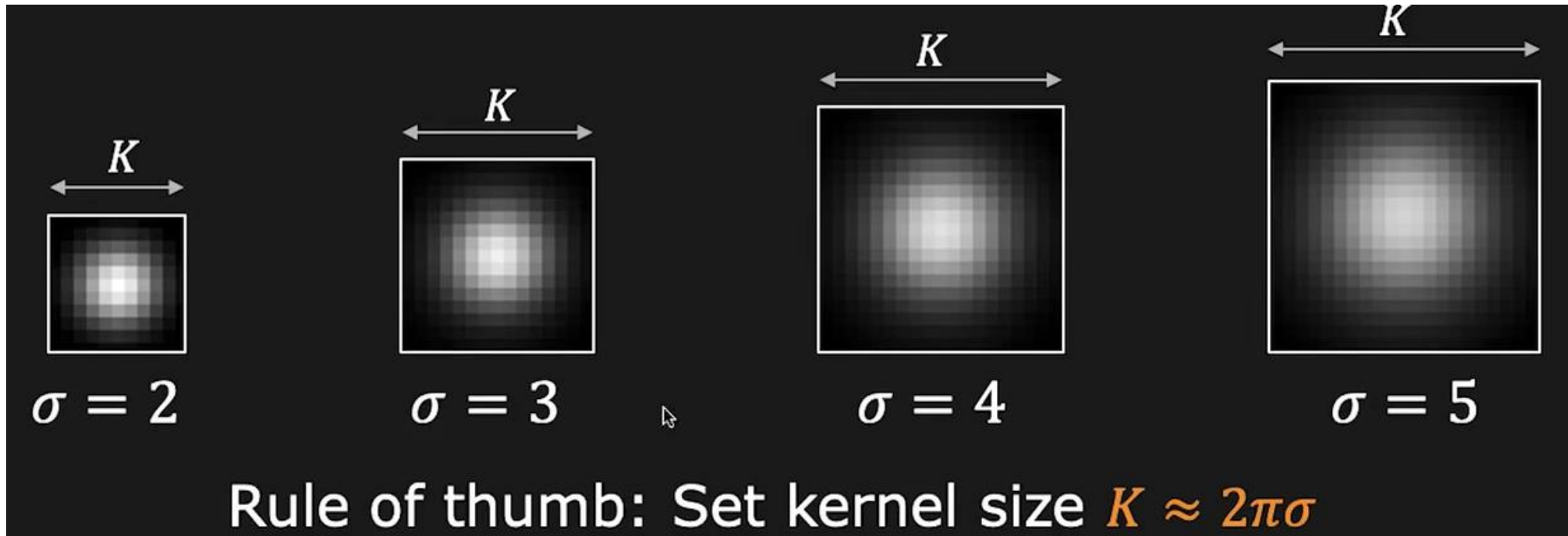
- **Pixel transformation:**

$$I'(x, y) = \sum_{i,j} G(i, j) \cdot I(x + i, y + j)$$

- **OpenCV:** `cv2.GaussianBlur(img, (5, 5), sigmaX=1.0)`
 - (5, 5): kernel size (odd numbers)
 - sigmaX: standard deviation in X direction (Y = X if not specified)



Gaussian blur filter: What sigma

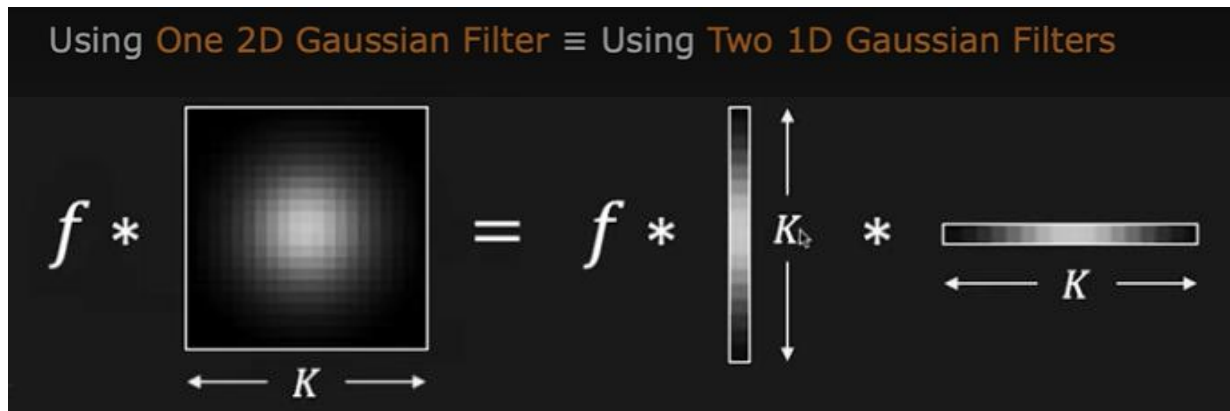
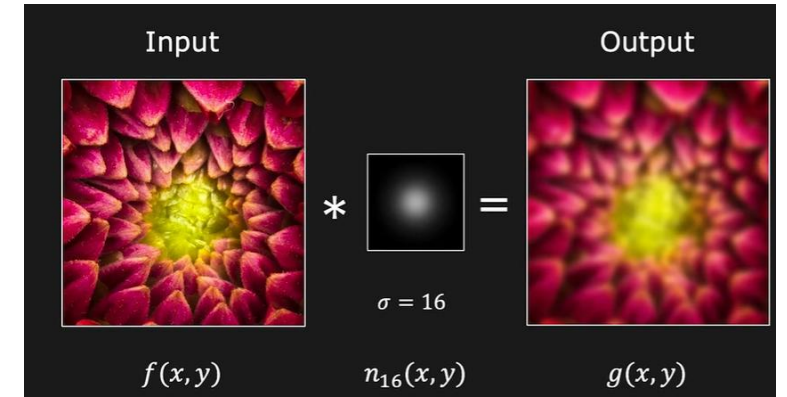
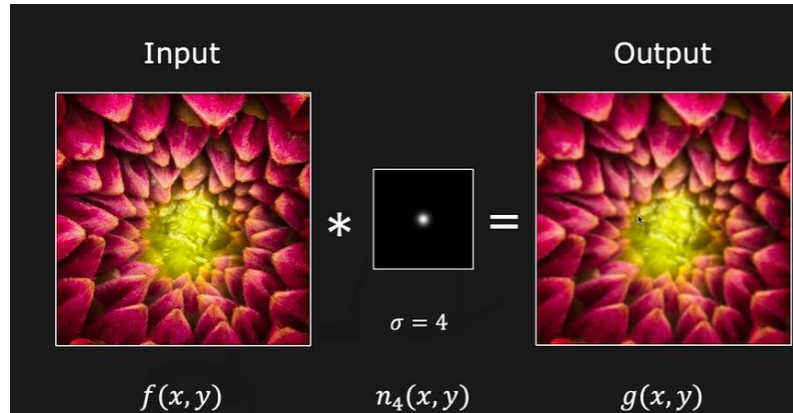


- Low σ (e.g. 0.5): Stronger detail preservation, minimal blur
- High σ (e.g. 3.0): Stronger blur, more smoothing and loss of edges
- Large σ spreads the Gaussian curve : kernel becomes flatter (box blur)

Smother and better edge preservation than box blur

More computational cost than simple averaging

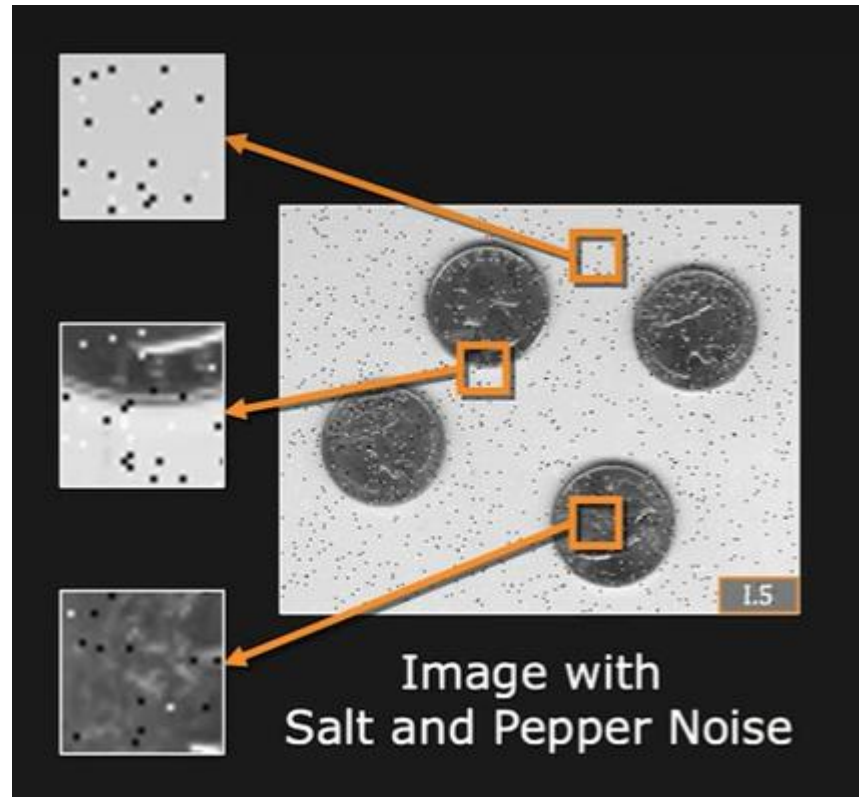
Gaussian blur filter: What sigma



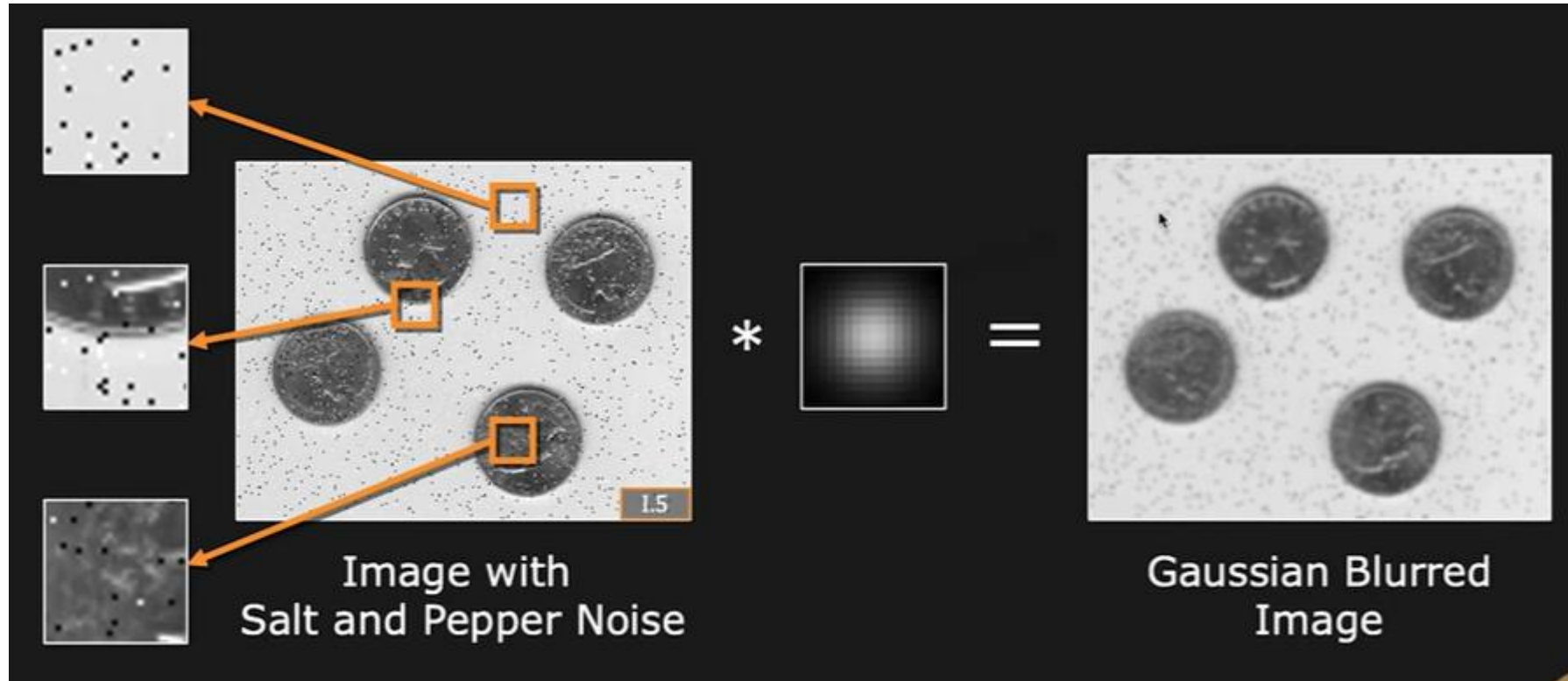
This is important because, two 1D Gaussian filter is faster than one 2D Gaussian filter

Salt-and-pepper noise

Purpose: Remove salt-and-pepper noise while preserving edges and the image (signal).



Salt-and-pepper noise

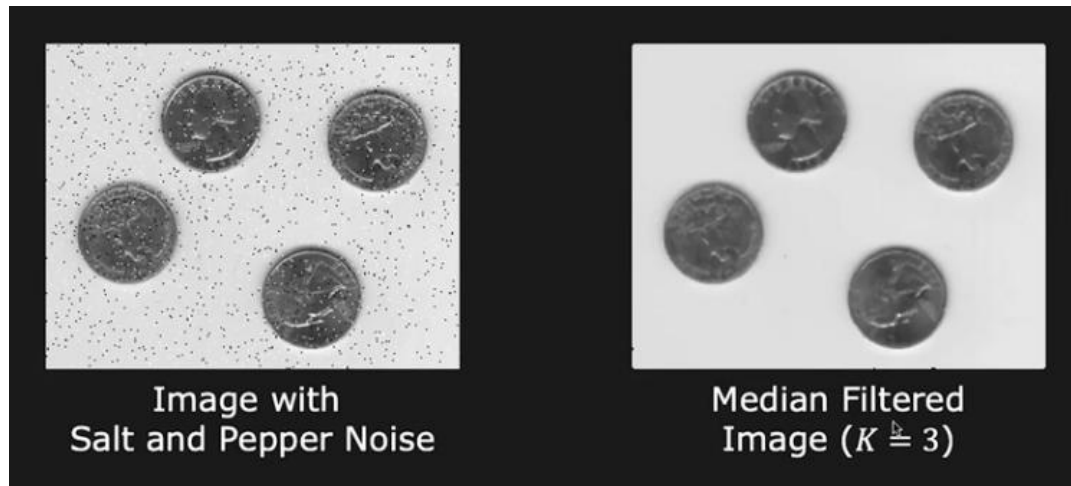


If you apply a Gaussian blur filter, you see a small diminution of the noise but what you are doing is smearing the noise out!! You also lose details on the coins

Non linear filter: Median blur filter

Purpose: Remove salt-and-pepper noise while preserving edges.

- Sort the K^2 values in windows centered at the pixel you want to filter
- Assign the median value to the pixel



Even with a small K , you get pretty good results. Pretty all the noise is gone, you lose only small details.

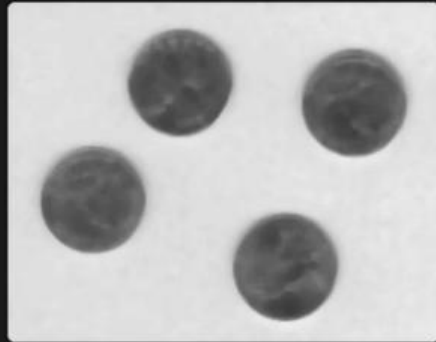
The **median filter** is **non-linear** because it does **not compute a weighted sum** of pixels.

- **OpenCV:** `cv2.medianBlur(img, 5)`
 - 5: size of kernel (must be odd and > 1), i.e., your window is 5x5

Non linear filter: Median blur filter



Image with Noise



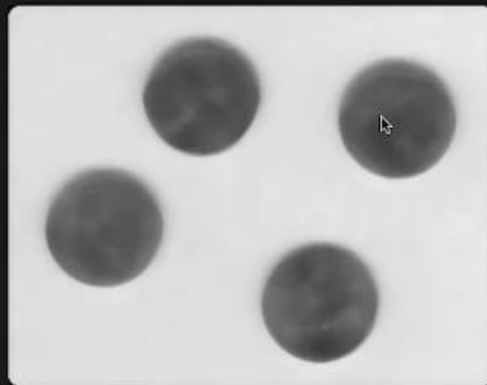
Median Filtered
Image ($K = 7$)

NOT EFFECTIVE WHEN THE NOISE IS NOT ONLY SALT-PEPPER NOISE

When K increase, you remove background (here not fully, you still have structure) and the price is heavy, you remove a lot of details



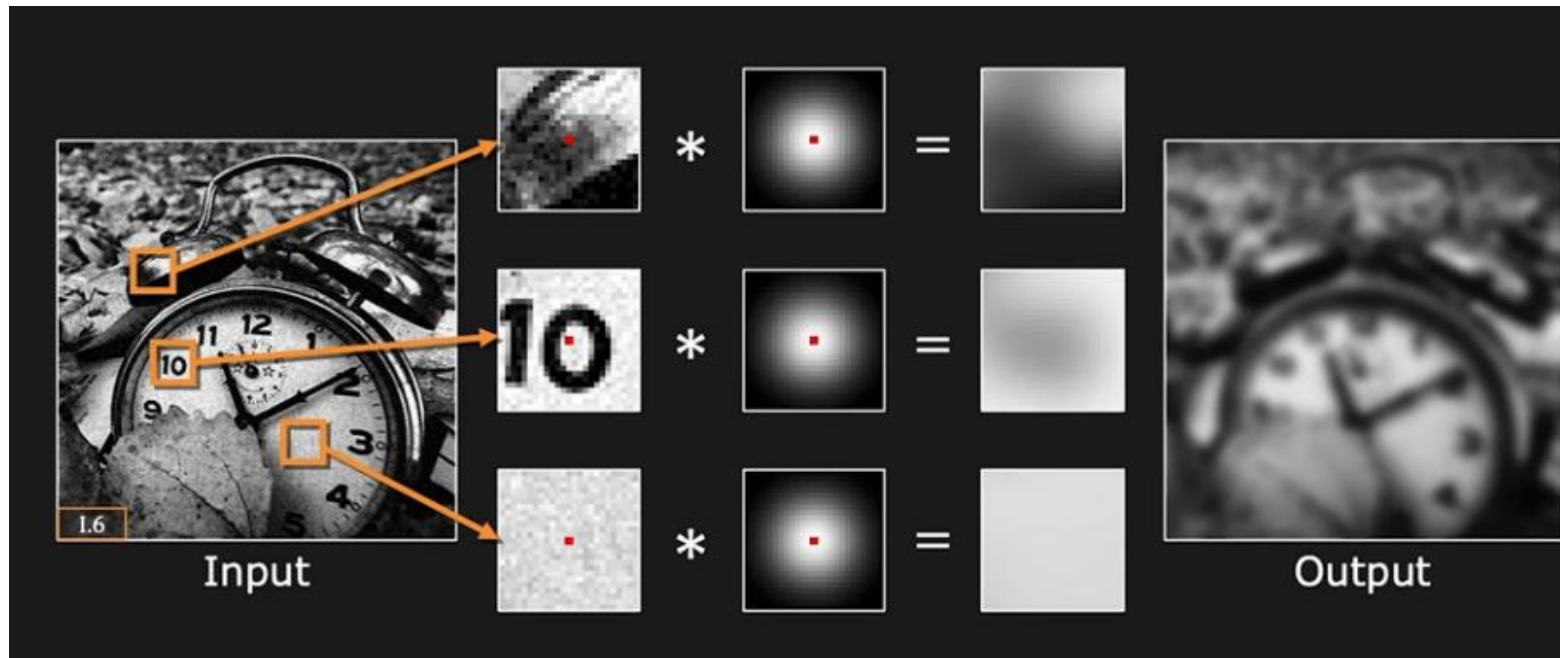
Image with Noise



Median Filtered
Image ($K = 11$)

Larger K remove a lot of details

Bilateral blur filter



You can clearly noise in the image.

You apply Gaussian convolution

Result:

For the flat region is work well BUT for other part no (e.g. 10...)

Issue with the Gaussian filter, same filter is used for every pixel

You need a kernel that gets modified depending on the structure of the image locally!!

-> A new filter for each pixel

Bilateral blur filter

Bilateral, same spatial sigma as the Gaussian but with a sigma brightness of 10. Bilateral much better than Gaussian



Change the brightness sigma, is you go to infinity=Gaussian smooth



Sharpening filter

Purpose: Enhance edges and fine structures by amplifying pixel differences

It takes a smoothed image, subtracts it from the original image to obtain the "details" of the image, and adds the "details" to the original image.

- **Kernel:** $K = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$ Center is strong
Neighbors are subtracted

- **Pixel transformation:**

$$I'(x, y) = 5 \cdot I(x, y) - [I(x - 1, y) + I(x + 1, y) + I(x, y - 1) + I(x, y + 1)]$$

- **OpenCV:** `sharpened = cv2.filter2D(img, -1, kernel)`

Enhances visual sharpness and contrast

May amplify noise if image is not smoothed first

Filter summary

- **Box Blur : Uniform averaging of neighboring pixels**
 - Kernel values are all equal: $1/n^2$
 - Smooths uniformly but blurs edges
- **Gaussian Blur : Weighted average using a Gaussian distribution**
 - Weights decrease with distance from center
 - Better edge preservation than box blur
- **Median Blur: Replaces center pixel with the median of neighborhood**
 - Excellent for salt-and-pepper noise
 - Non-linear, preserves edges better
- **Bilateral Filter: Combines spatial and intensity similarity**
 - Blurs flat regions while preserving sharp edges
 - Slow but high quality
- **Sharpening Filter:** Enhance edges and fine structures by amplifying pixel differences
 - May amplify noise if image is not smoothed first
 - Enhances visual sharpness and contrast

Fast Non-Local Means Denoising

- **Purpose:** Advanced denoising that preserves textures and sharp edges by searching for **similar patches** across the image, not just in the local neighborhood

Function

`cv2.fastNlMeansDenoising()`

`cv2.fastNlMeansDenoisingColored()`

`cv2.fastNlMeansDenoisingMulti()`

`cv2.fastNlMeansDenoisingColoredMulti()`

Description

Grayscale images

RGB images

Multi-frame grayscale (e.g., video)

Multi-frame RGB

- `h`: parameter deciding filter strength. Higher `h` value removes noise better, but removes details of image also. (10 is ok)
- `templateWindowSize`: Size of patch (default: 7)
- `searchWindowSize`: Area to search for similar patches (default: 21)

Preserves texture and detail better than median or Gaussian filters.

Slower than simpler filters, parameters must be tuned.

Edge detection

What is an edge?

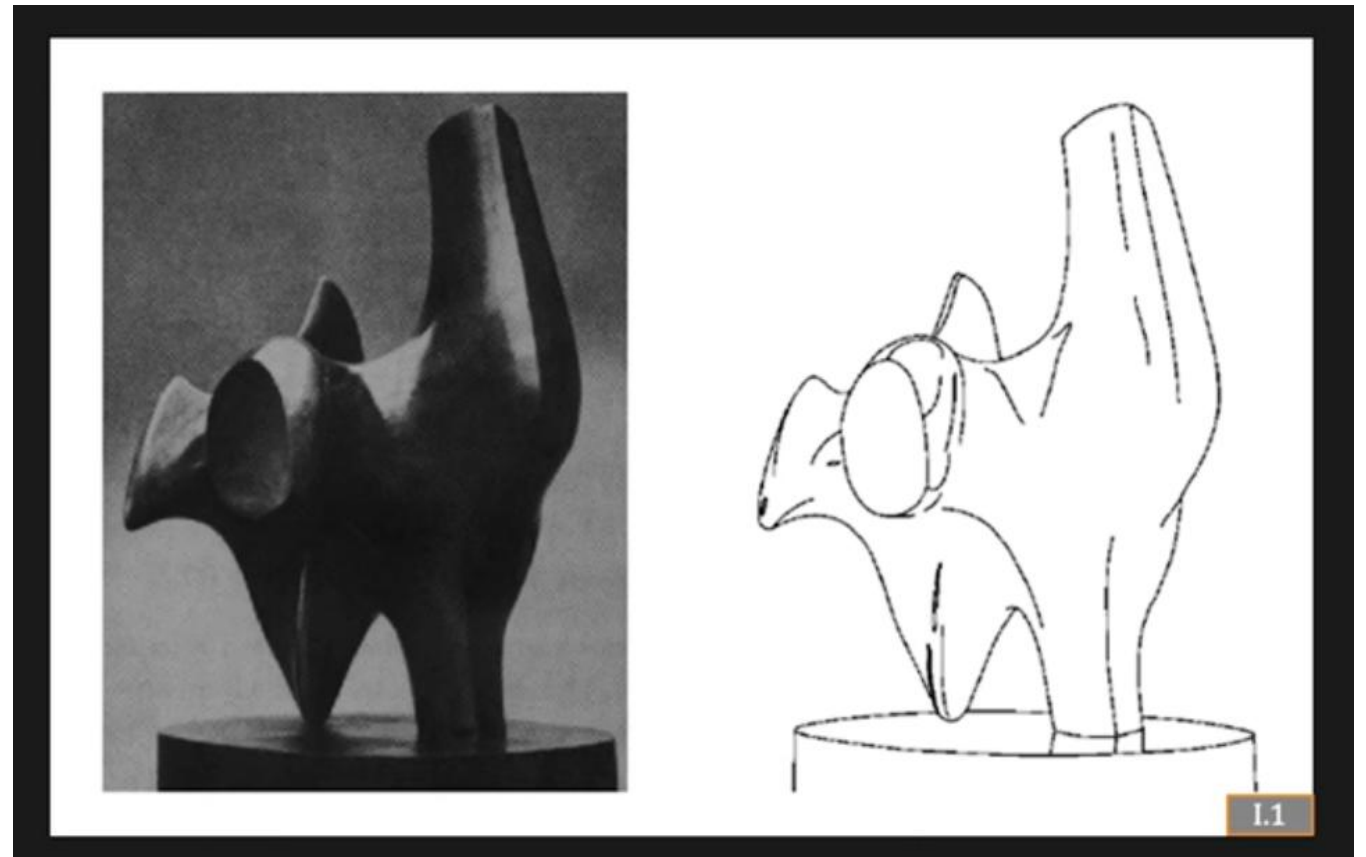
Rapid change in image intensity within a small windows region

Why is it important?

- With only a few strokes, the artist is able to convey a lot of information: e.g. the 3D structure, lighting effect

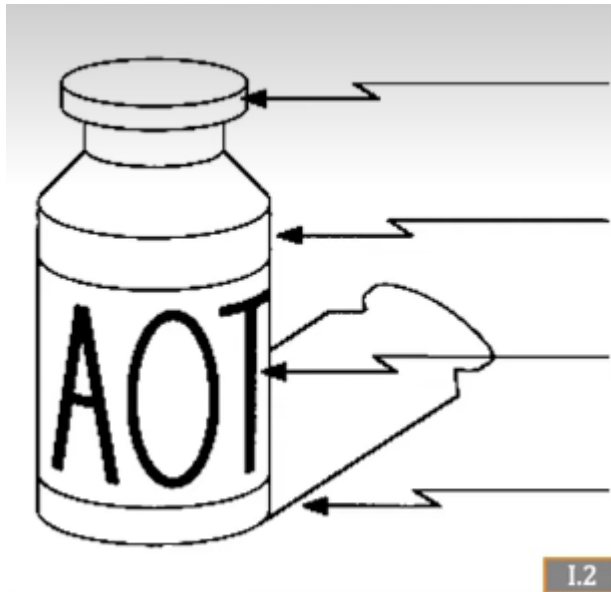
Image of a sculpture of
Henri Moore

Sketch of the image



Edge detection: causes

Rapid change in image intensity are caused by different physical phenomena



Surface normal discontinuity: two surface of the same material but different surface orientation

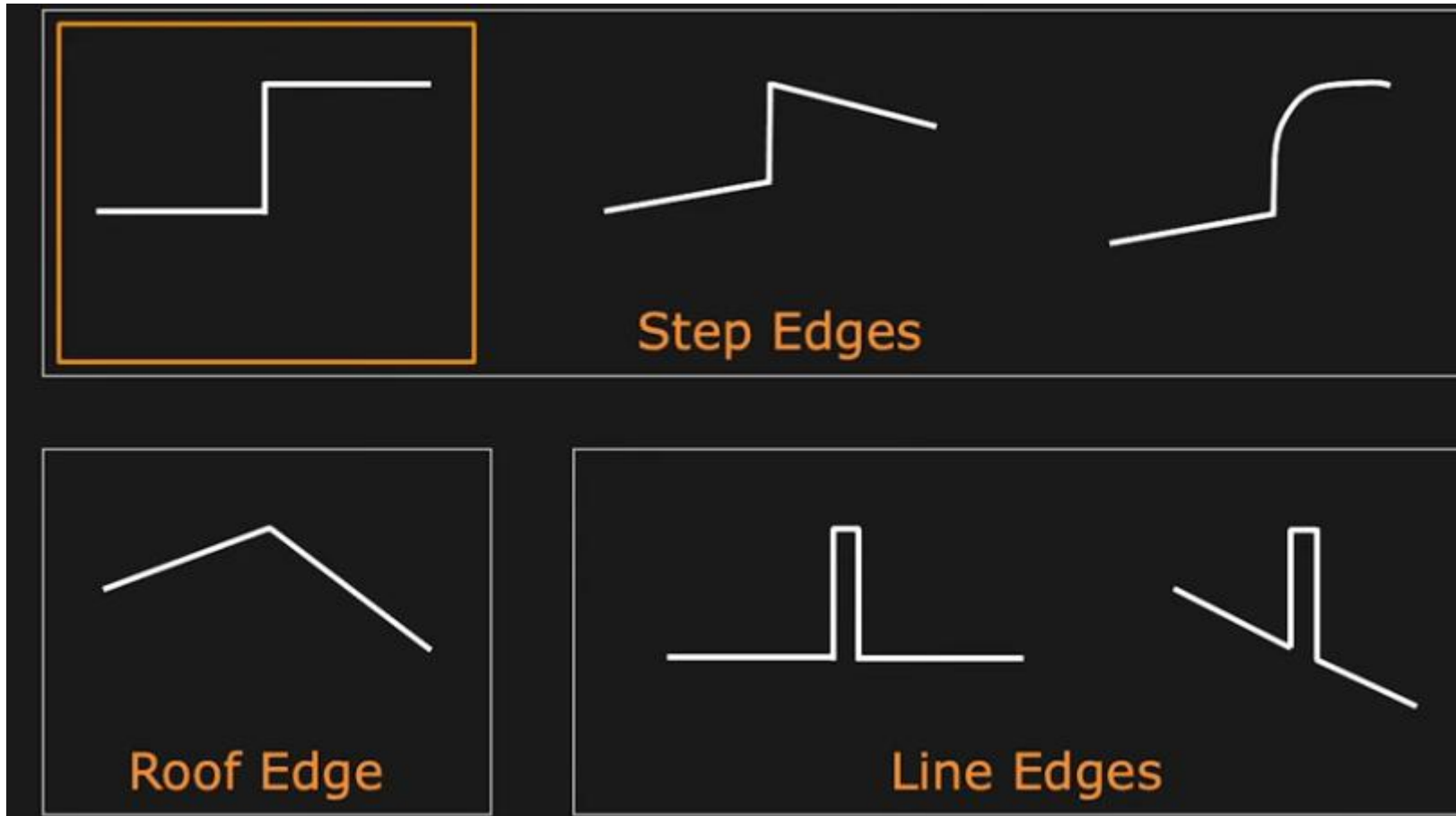
Depth discontinuity: You have one object behind another (like that they are made by 2 different materials (here bottle and background))

Surface reflectance discontinuity: the letters have a different material than the bottle

Illumination discontinuity: shadow of the bottle.

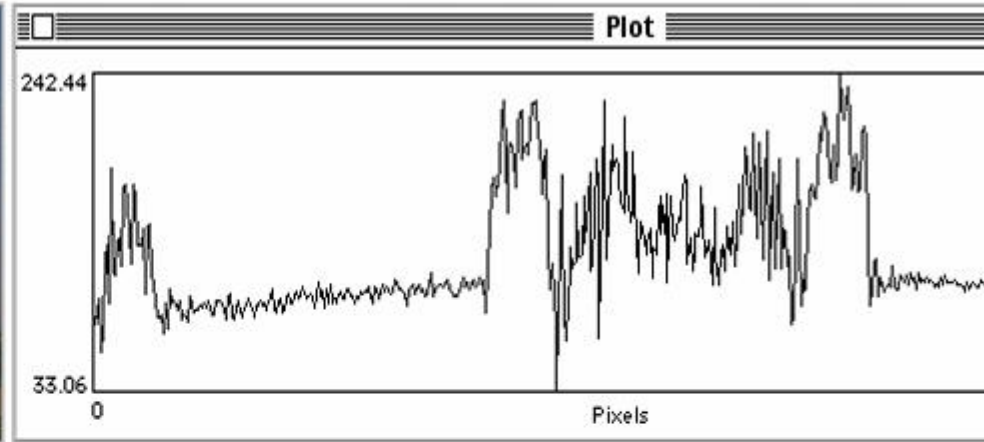
Edge detection: types

Examples of edge types

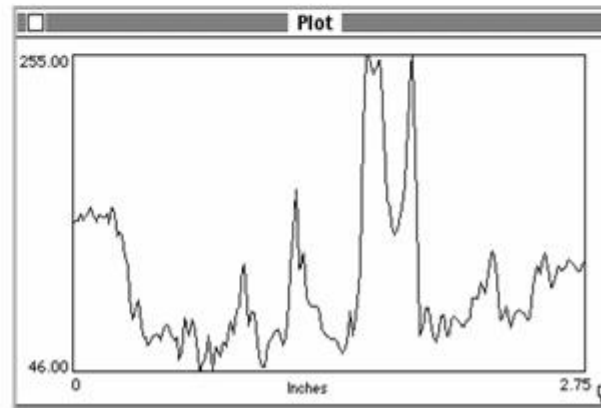
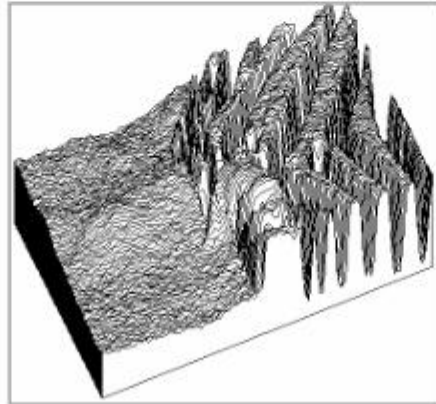


Edge detection: types

Real world examples



As you can see with real-world examples, edges are pretty far from a step function!!!



First edge detector: intensity

12	90	89	86	87	82
10	12	88	85	83	84
9	15	12	84	84	88
12	14	10	82	88	89
11	17	16	12	88	90
10	16	15	17	89	88

Edge detection: intensity

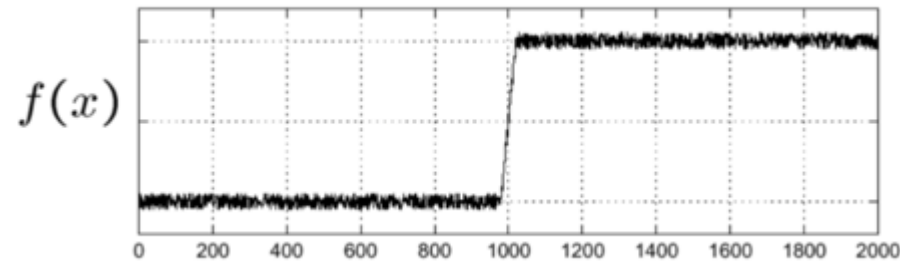


Some issues:

- Size of the neighborhood
- Metrics to measure the change

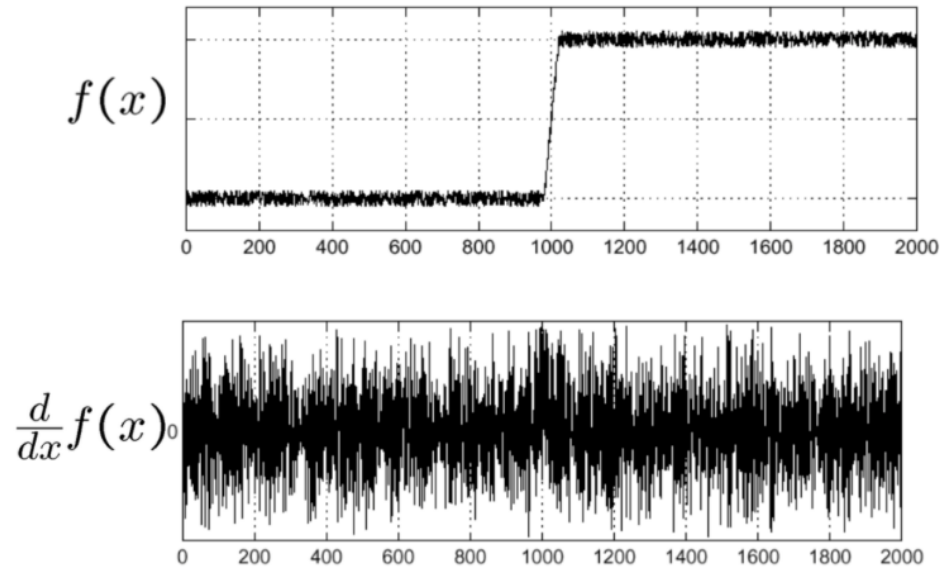
Edge detection: gradient

Real-world images are generally susceptible to a lot of noise. Consequently, plotting the pixel intensities often results in noise, making it impossible to identify where an edge is by only taking the first derivative of the function.



Derivative of a continuous function represents the amount of change in the function

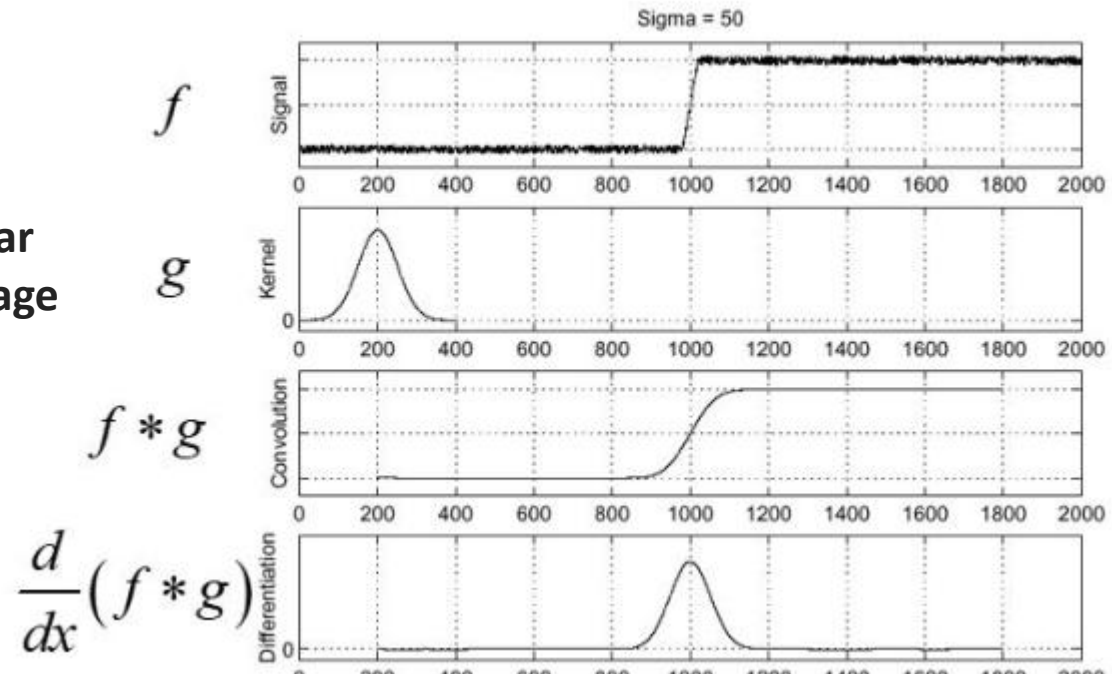
Edge detection: gradient



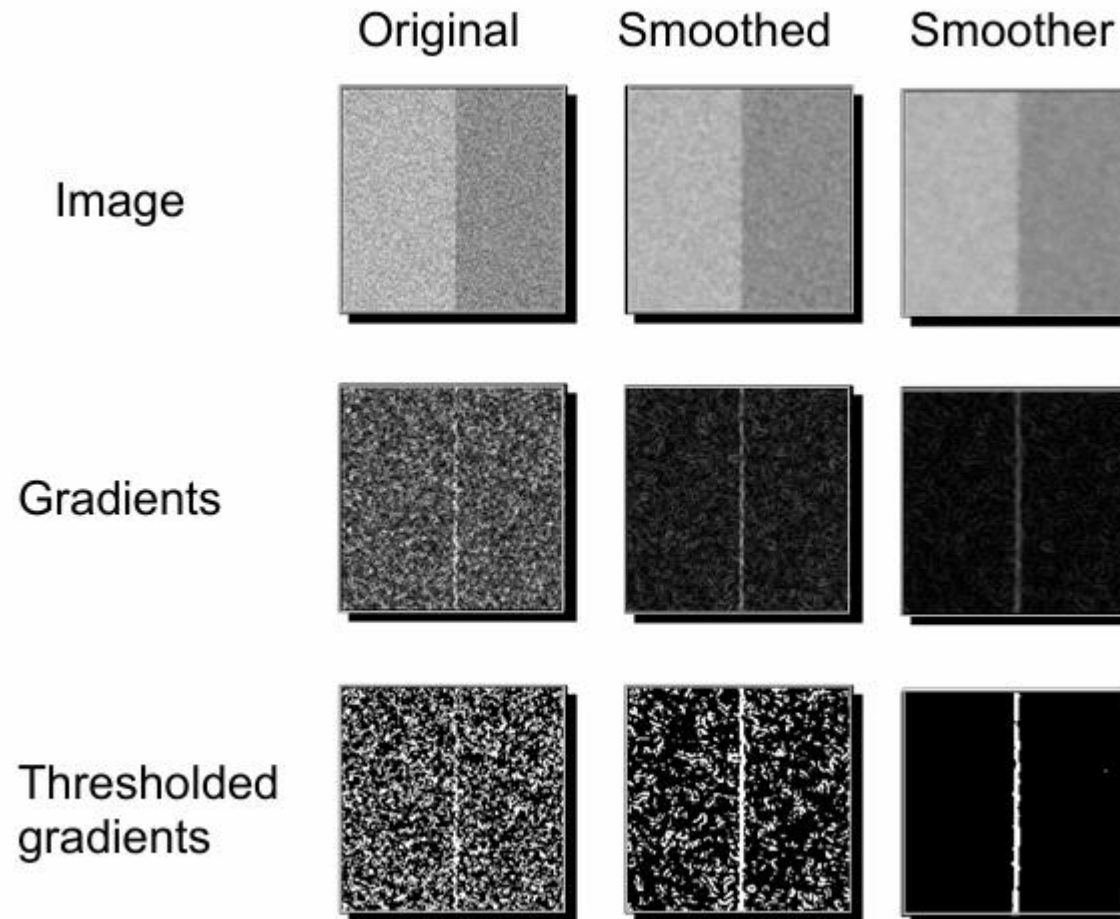
Unable to determine the edges by the location of local maxima or minima. Need to smooth before taking its derivative.

Edge detection: gradient

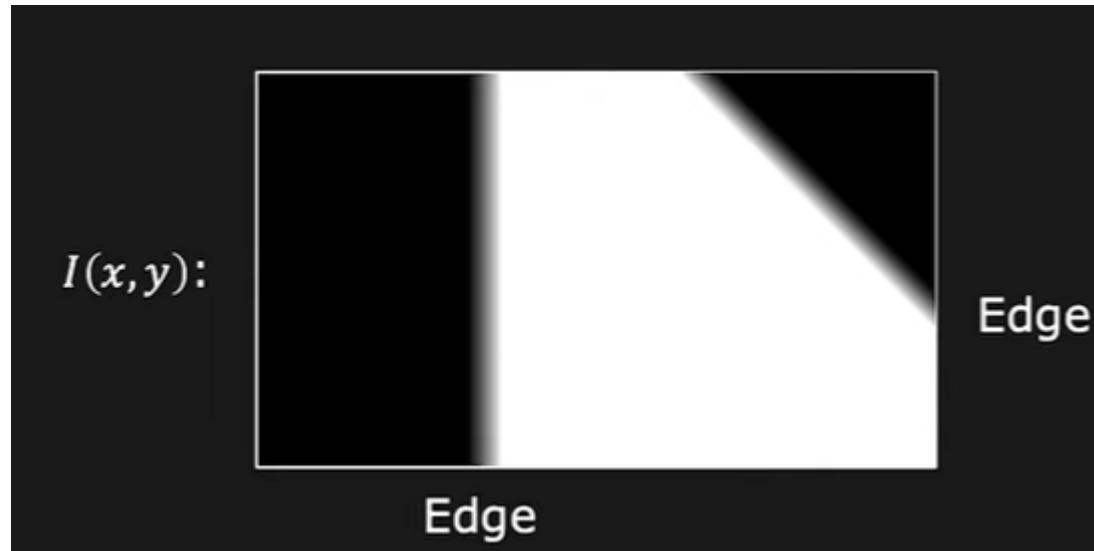
Gaussian filtering is quite popular for smoothing or blurring an image



Edge detection: gradient



Edge detection 2D: gradient



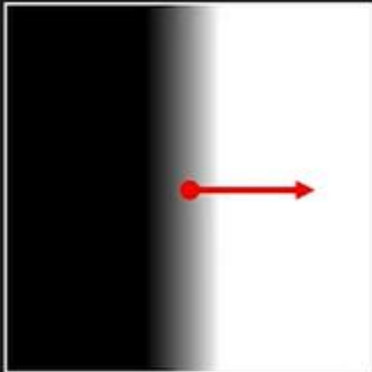
Partial derivatives of a 2D continuous function represent the amount of change along each dimension

Edge detection 2D: gradient

$$\nabla I = \left[\frac{\partial I}{\partial x}, \frac{\partial I}{\partial y} \right]$$

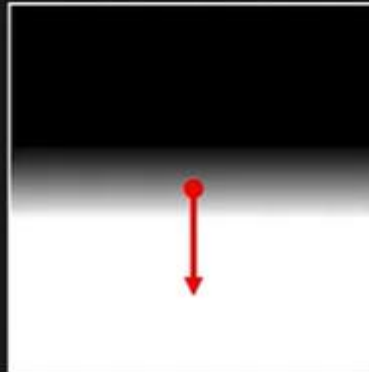
Few examples

Vertical: no change in y
direction



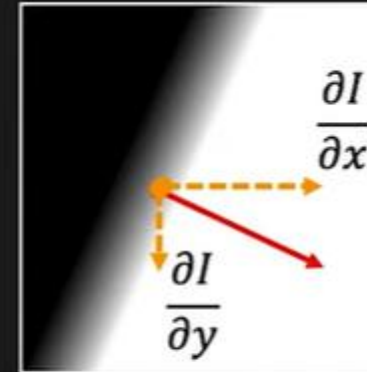
$$\nabla I = \left[\frac{\partial I}{\partial x}, 0 \right]$$

Horizontal: no change
in x direction



$$\nabla I = \left[0, \frac{\partial I}{\partial y} \right]$$

Change in both
direction

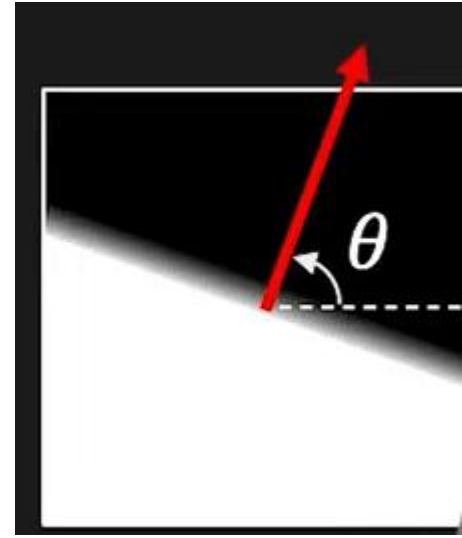


$$\nabla I = \left[\frac{\partial I}{\partial x}, \frac{\partial I}{\partial y} \right]$$

Edge detection 2D: gradient

Gradient Magnitude $S = \|\nabla I\| = \sqrt{\left(\frac{\partial I}{\partial x}\right)^2 + \left(\frac{\partial I}{\partial y}\right)^2}$

Gradient Orientation $\theta = \tan^{-1} \left(\frac{\partial I}{\partial y} / \frac{\partial I}{\partial x} \right)$



Sobel Filter : Gradient-Based Edge Detection.

Purpose: Detect edges by computing gradient magnitude in horizontal and vertical directions. To obtain the gradients in the x (horizontal) and y (vertical) directions, apply the Sobel. **Can apply KERNEL and CONVOLUTION to obtain derivative on a images.**

Kernel:

$$G_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix} \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix}$$

Transformation: Use the gradients to compute the magnitude and direction of the edge at each pixel.

$$I'(x, y) = \sqrt{G_x^2 + G_y^2}$$

$$\theta(x, y) = \arctan \left(\frac{G_y}{G_x} \right)$$

OpenCV:

```
gx = cv2.Sobel(img_gray, cv2.CV_64F, 1, 0, ksize=3)
gy = cv2.Sobel(img_gray, cv2.CV_64F, 0, 1, ksize=3)
magnitude = cv2.magnitude(gx, gy)
```

- input image (usually grayscale)
- depth: output data type (e.g., cv2.CV_64F for precision)
- dx: derivative order in x direction (1 to apply)
- dy: derivative order in y direction (1 to apply)
- ksize: size of extended Sobel kernel (must be odd and > 1)

Preserves texture and detail better than median or Gaussian filters.

Slower than simpler filters, parameters must be tuned

Sobel Filter : Gradient-Based Edge Detection.

Standard: (Single Threshold T)

$\|\nabla I(x, y)\| < T$ Definitely Not an Edge

$\|\nabla I(x, y)\| \geq T$ Definitely an Edge

Hysteresis Based: (Two Thresholds $T_0 < T_1$)

$\|\nabla I(x, y)\| < T_0$ Definitely Not an Edge

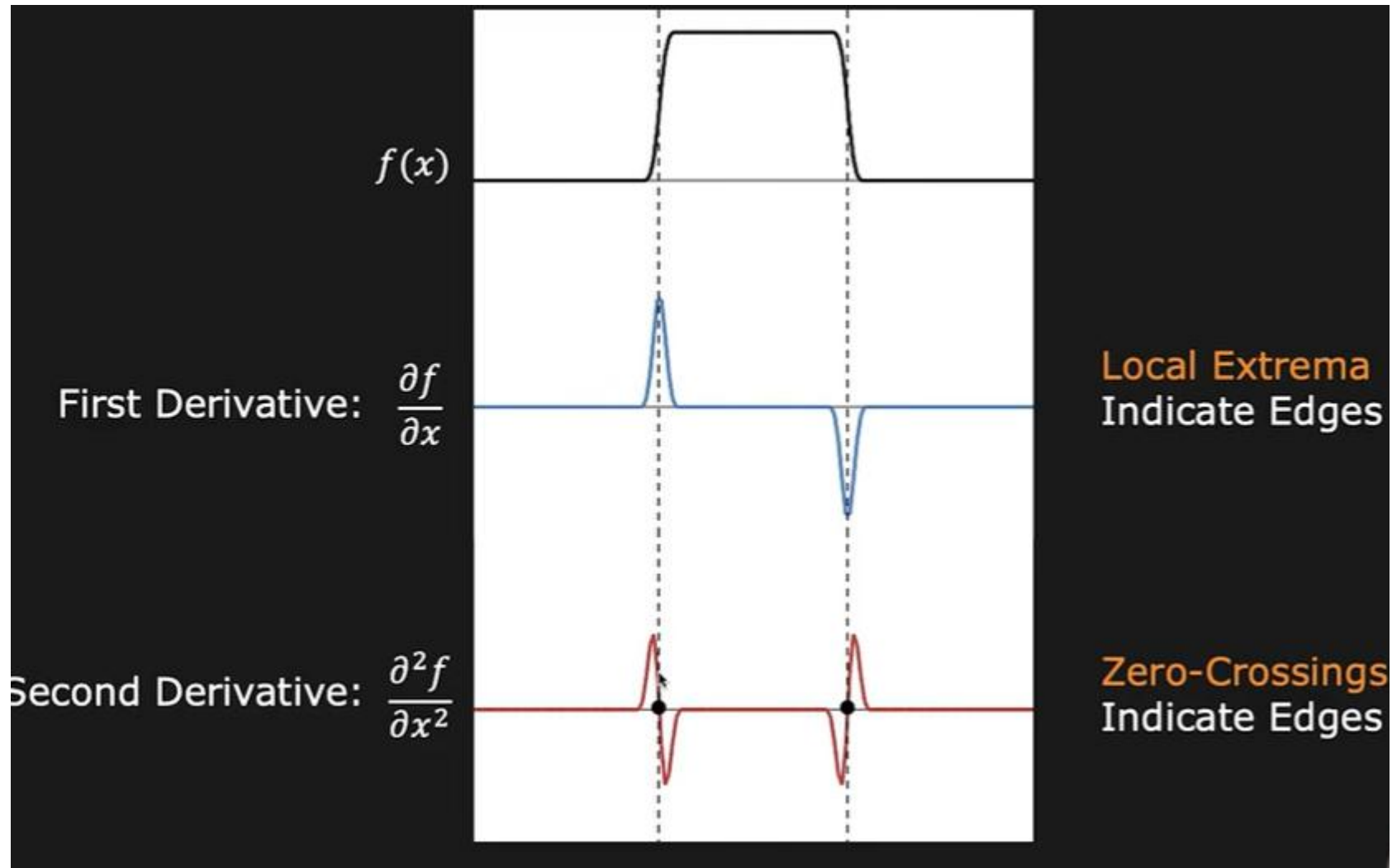
$\|\nabla I(x, y)\| \geq T_1$ Definitely an Edge

$T_0 \leq \|\nabla I(x, y)\| < T_1$ Is an Edge if a Neighboring Pixel is Definitely an Edge

Sobel Filter : Gradient-Based Edge Detection.



Laplacian Filter : Second Derivative of Intensity



At the edges you do not have peak but 0!!

Laplacian Filter : Second Derivative of Intensity

Purpose: Detects areas of rapid intensity change by computing the second derivative of the image. it indicates how much the function's value deviates from the average value of its surrounding points

Laplacien:

$$\nabla^2 I = \frac{\partial^2 I}{\partial x^2} + \frac{\partial^2 I}{\partial y^2}$$

- **Edges: zero crossing in Laplacian images**
- **Laplacian does not provide orientation of the edges**

Laplacian Filter : Second Derivative of Intensity

Purpose: Detects areas of rapid intensity change by computing the second derivative of the image. it indicates how much the function's value deviates from the average value of its surrounding points

Kernel

You need 3 pixels
(ϵ =distance pixel)

$$\nabla^2 \approx \frac{1}{\epsilon^2} \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix} \quad \text{OR} \quad \nabla^2 \approx \frac{1}{6\epsilon^2} \begin{bmatrix} 1 & 4 & 1 \\ 4 & -20 & 4 \\ 1 & 4 & 1 \end{bmatrix}$$

e K. Navar

Simplest kernel, does not
take into account diagonal
edge (only x or y)

Laplacian Filter : Second Derivative of Intensity



OpenCV: `lap = cv2.Laplacian(img_gray, cv2.CV_64F, ksize=3) # CV_64F to preserve negative gradients`
`lap_abs = cv2.convertScaleAbs(lap) #converting float to 8-bit`

- values can be **negative**
- values can exceed 255
- values are floating point

So 128=0, everything brighter is positive, everything darker is negative. For everything crossing you have 128

Sobel or Laplacian

Sobel when:

- You care about **robust edges**
- You want **gradient direction**
- You are doing:
 - object detection
 - feature extraction
 - motion analysis

CAN WE TAKE THE BEST OF THE TWO METHODS??

..... Of course yes

Laplacian when:

- You want **quick edge map**
- You want **edge sharpening**
- You are doing:
 - image enhancement
 - blob/texture detection

Canny edge detector

The canny edge detector is the most widely used edge detector!! It is a good mix of the Sobel and Laplacian edge detector

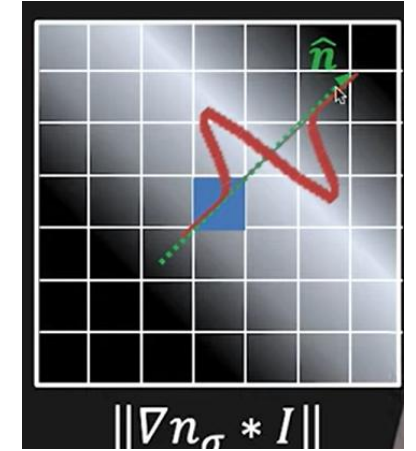
- **Gaussian Blur:** Smooth the image to reduce noise using a Gaussian filter
- **Gradient Calculation:** Compute the intensity gradients of the image using methods like Sobel
- **Gradient Magnitude and Orientation:** Find gradient magnitude and orientation at each pixel

$$\|\nabla n_{\sigma} * I\|$$

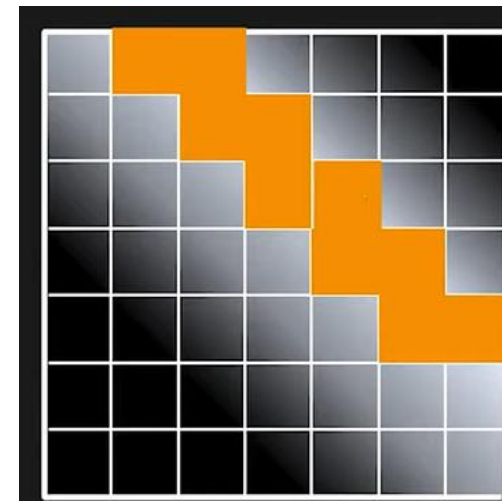
$$\hat{n} = \frac{\nabla n_{\sigma} * I}{\|\nabla n_{\sigma} * I\|}$$

- **Apply 1 direction Laplacian:** Instead to apply 2D Laplacian (x,y) apply only 1D Laplacian (wrt $\mathbf{n}$, the direction of the gradient). It is more robust because only in 1D.
- **Find zero crossing:** Find zero crossing for detecting edges

$$n_{\sigma} * I$$

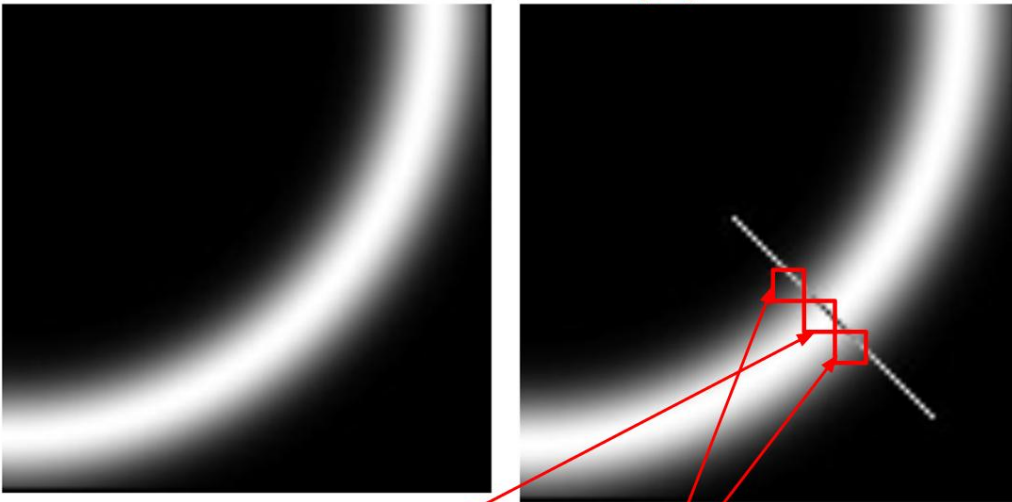


$$\frac{\partial^2 (n_{\sigma} * I)}{\partial \hat{n}^2}$$



Canny edge detector

- **non-maxima suppression** : Thin out the edges by suppressing all gradients that are not local maxima.



If the intensity of the gradient of a pixel is less than at least one of the two neighbours along the gradient, then it must be suppressed (ie. gradient magnitude set to zero)

For each pixel check the neighbours depending on the direction of the gradient

Canny edge detector

- **Hysteresis thresholding:** Identify strong and weak edges based on threshold values. Finalize the edge detection by connecting weak edges to strong edges if they are part of the same edge segment.

L: Lower threshold value. Any value below L is considered not to be an edge.

U: Upper threshold value. Any value above U is considered to be an edge (strong edge)

Any value between L and U are weak edges

OpenCV: `canny = cv2.Canny(image, L=50, U=120)`

- L: Lower threshold value.
- U: Upper threshold value

Too high L/U → Miss fine edges (only strong ones remain)

Too low L/U → Include noise and unwanted details

High accuracy and robustness to noise, Good localization of edges, Produces thin edges.
Sensitive to the choice of thresholds for double thresholding

Canny edge detector



Image



$\sigma = 1$



$\sigma = 2$



$\sigma = 4$

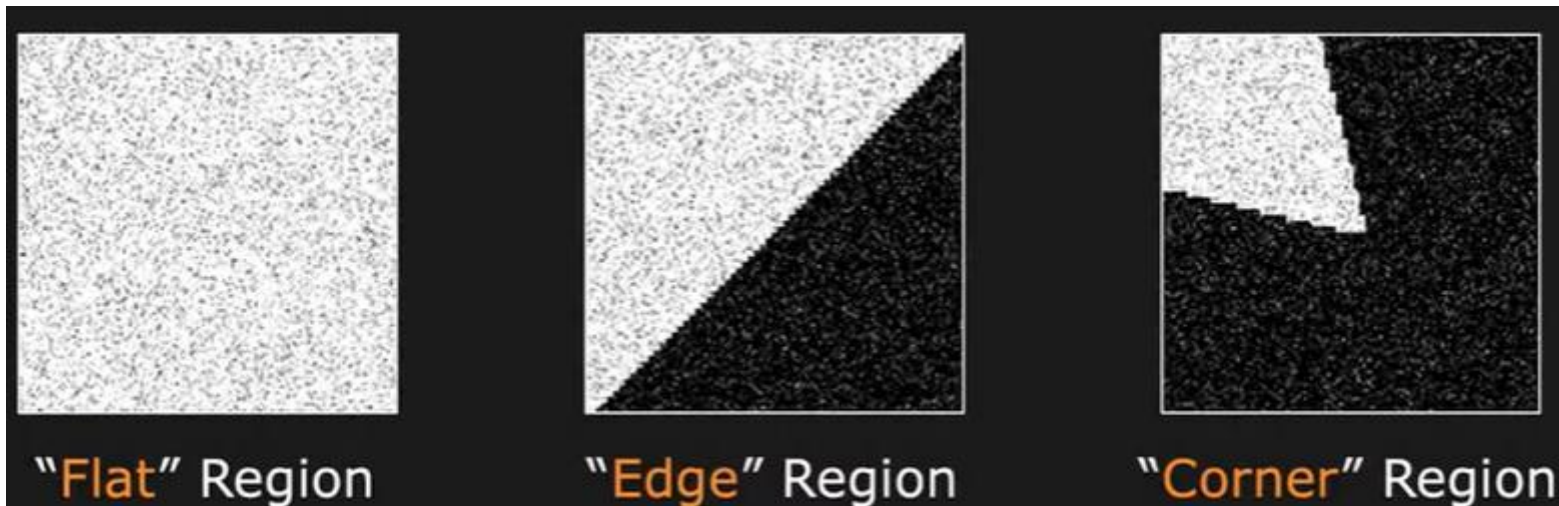
Corner detection

Why corners?

Corners are highly distinctive and stable under transformations, making them excellent for matching and tracking.

What is a corner?

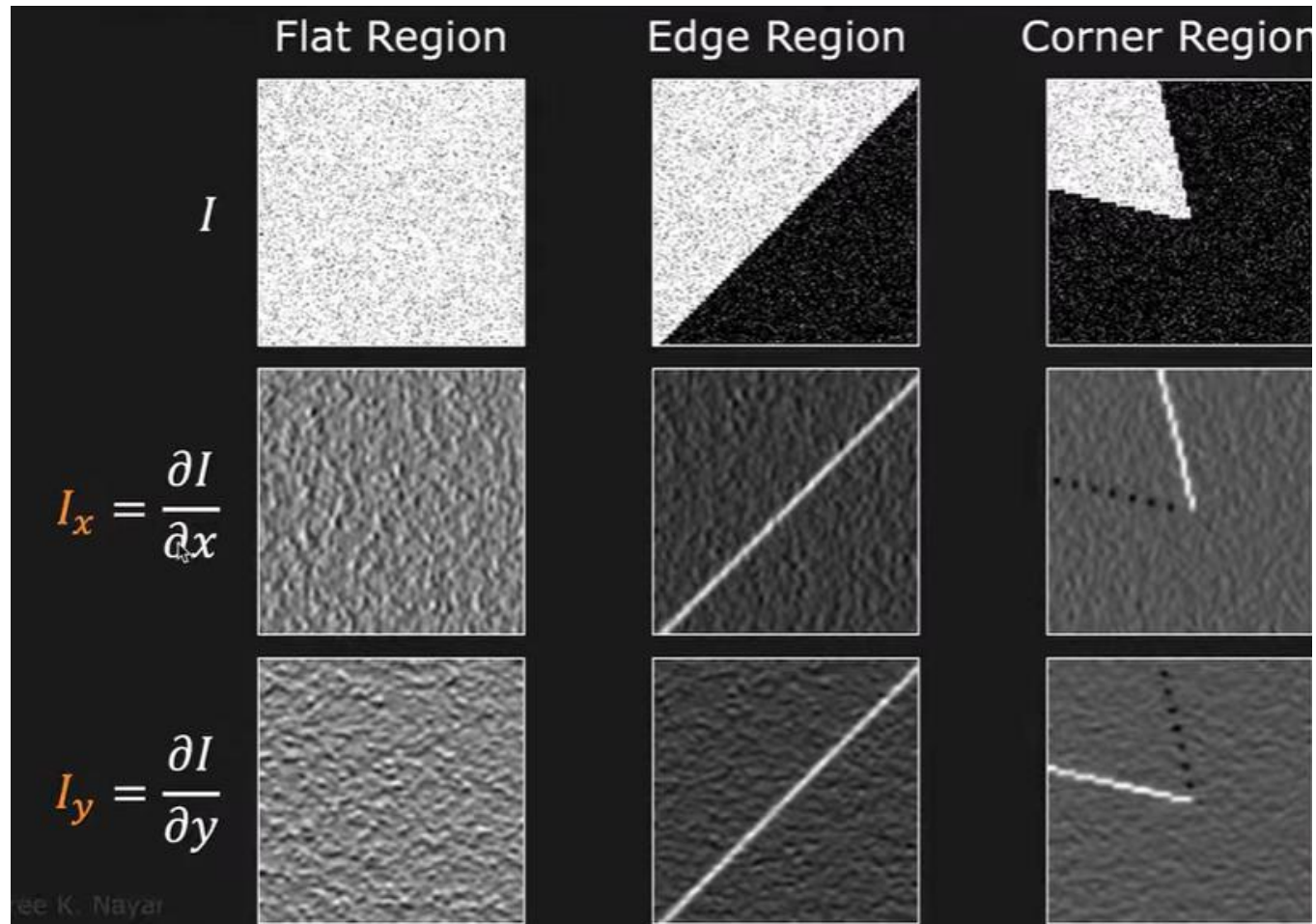
Rapid change of image intensity in two directions



Corner detection

Algorithm Overview:

- Compute gradients in x and y.



Harris Corner detection

Algorithm Overview:

- Compute gradients in x and y.
- For each pixel, build a matrix from the gradients over a window.
- Calculate “cornerness” (R value, **High R**: Likely a corner.).
- Threshold R to detect corners.

OpenCV

`dst = cv2.cornerHarris(gray, blockSize=2, ksize=3, k=0.04)`

- `blockSize=2` → neighborhood size
- `ksize=3` → Sobel kernel size (gradient)
- `k=0.04` → sensitivity (usually 0.04–0.06)

`dst = cv2.dilate(dst, None)` (makes corner points bigger so you can see them clearly.)

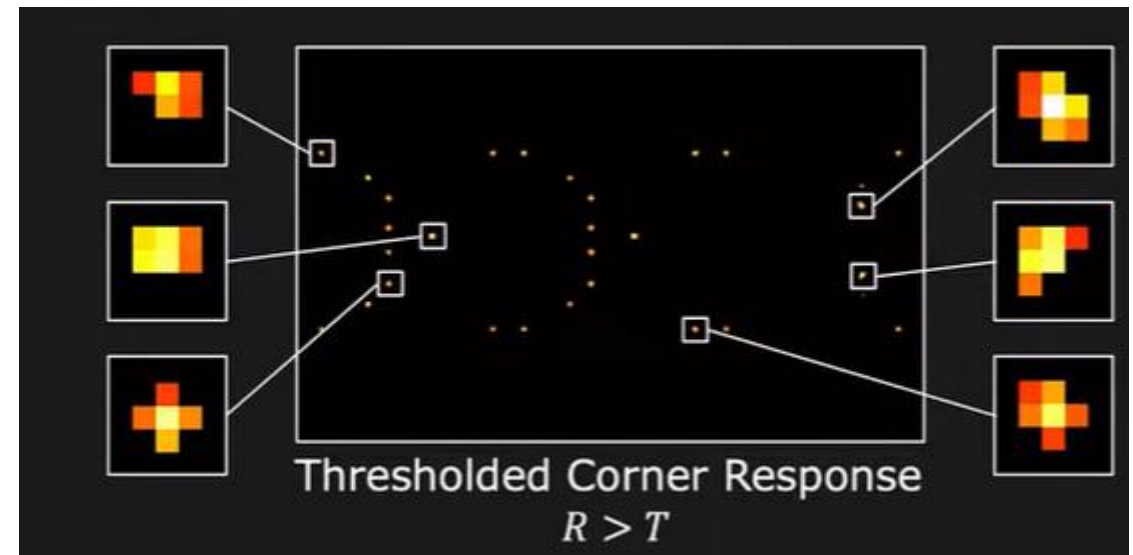
`img[dst > 0.01 * dst.max()] = [0, 0, 255]` (keep only strong response and mark them in red)

Corner detection: example



You have cluster of points with large values close to the corner.

So which one is the peak?



Corner detection: example

Non maxima suppression.

- Slide a windows of size k over the image
- At each position if the central pixel is the highest value keep it otherwise delete it (in the example brightest is the white)



Contours detection

Contours: Curves joining all continuous points along a boundary having the same color or intensity

Why are they useful?

- Object detection
- Shape matching
- Measuring size of objects

OpenCV:

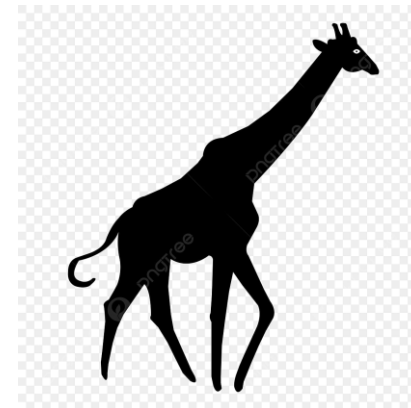
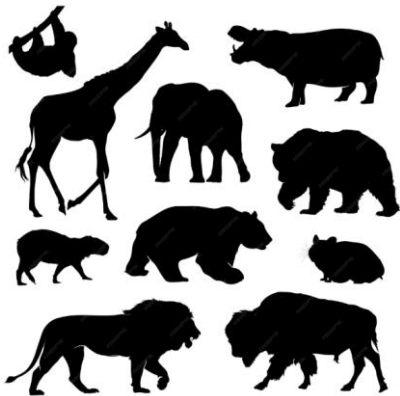
- `contours, hierarchy = cv2.findContours(thresh_img, mode, method)`
 - Mode
 - `cv2.RETR_EXTERNAL` Returns only outermost contours.
 - `cv2.RETR_LIST` Returns all contours. (no idea one is inside or outside)
 - `cv2.RETR_TREE` Returns all contours and reconstructs a full hierarchy of nested contours
 - Method
 - `cv2.CHAIN_APPROX_NONE` Stores all contour points (most detailed).
 - `cv2.CHAIN_APPROX_SIMPLE` Compresses straight lines into endpoints only (saves memory)
- `cv2.drawContours(img, contours, -1, (0,255,0), 2) #draw the contours`

Shape matching

Purpose: Compare shapes quantitatively and determine how similar they are

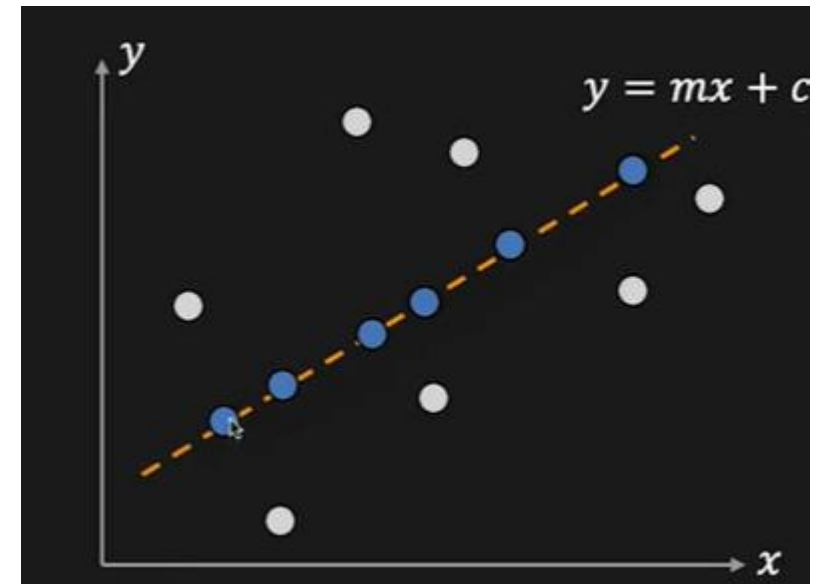
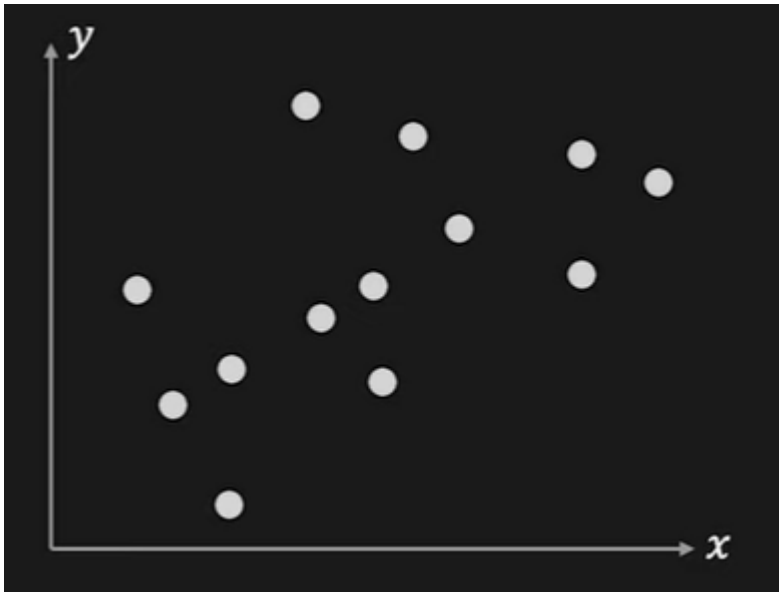
OpenCV:

- `similarity = cv2.matchShapes(contour1, contour2, method, 0.0)`
 - `cv2.CONTOURS_MATCH_I1`: Sum of absolute differences
 - `cv2.CONTOURS_MATCH_I2`: Sum of squared differences
 - `cv2.CONTOURS_MATCH_I3`: Uses Hu Moments differences



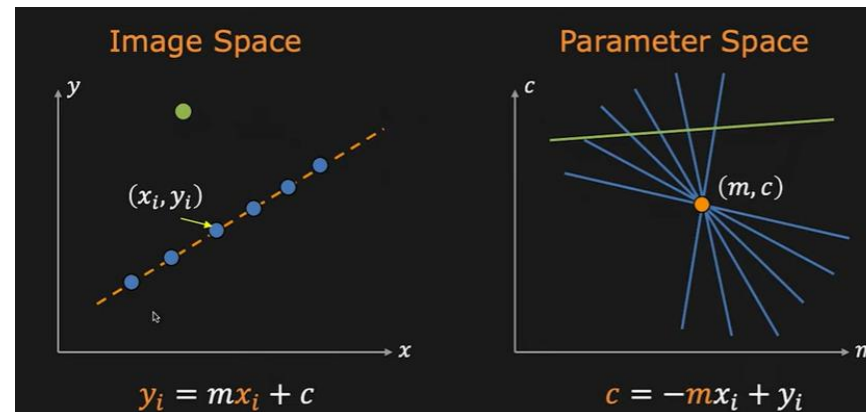
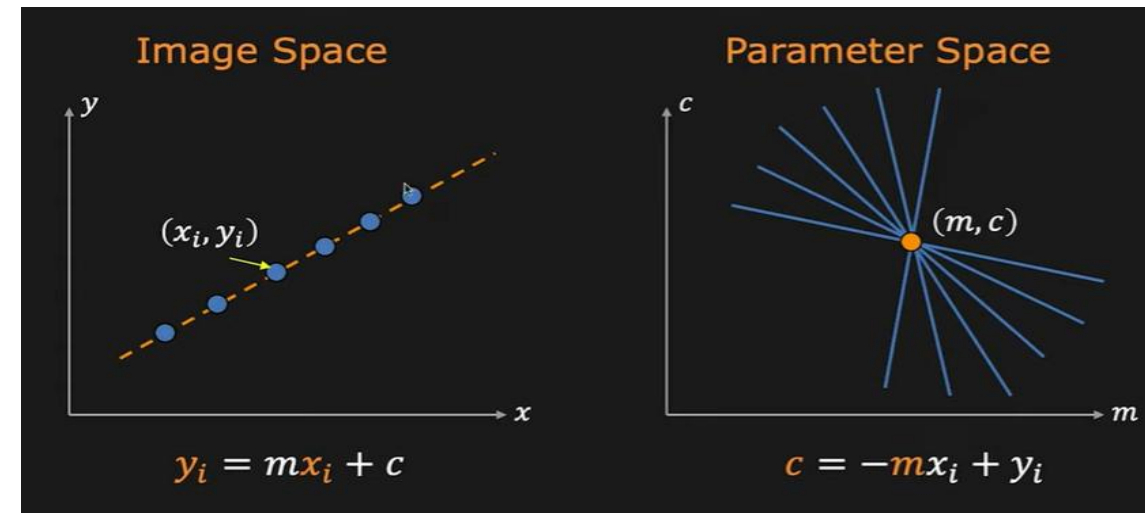
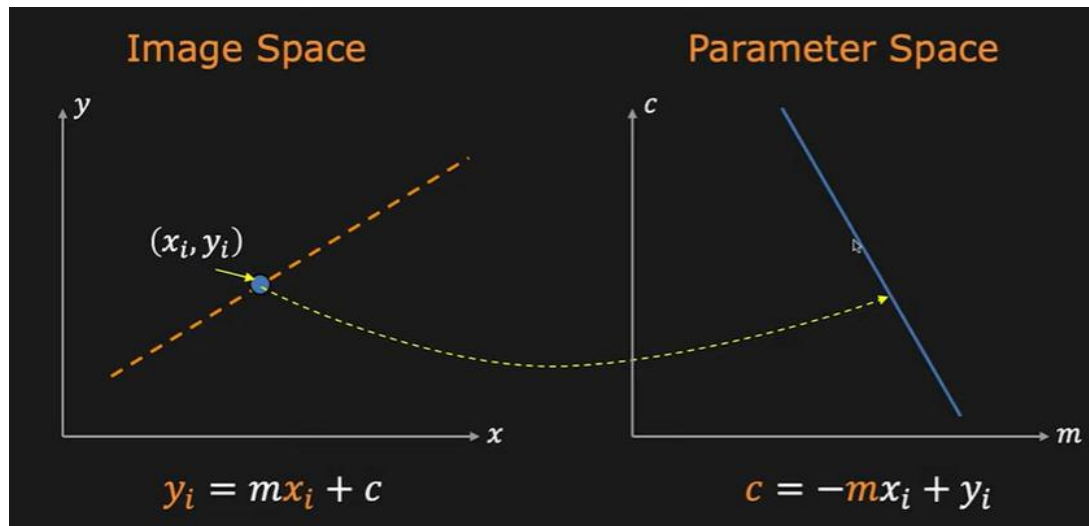
Line Detection : Hough Transform

So you have different edges (x,y) and the goal is to find all the different lines?



Hough Transform

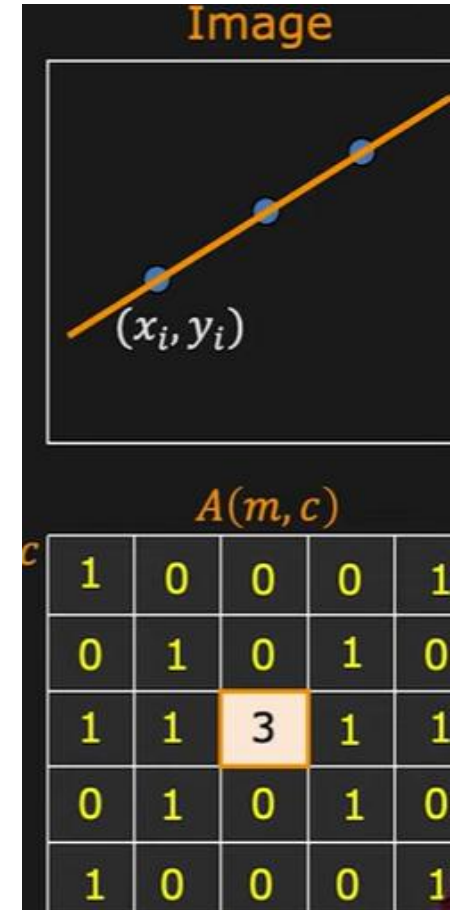
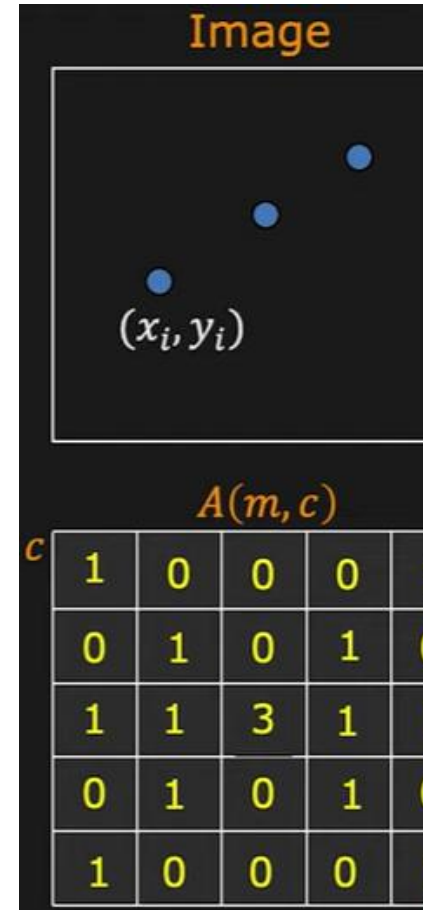
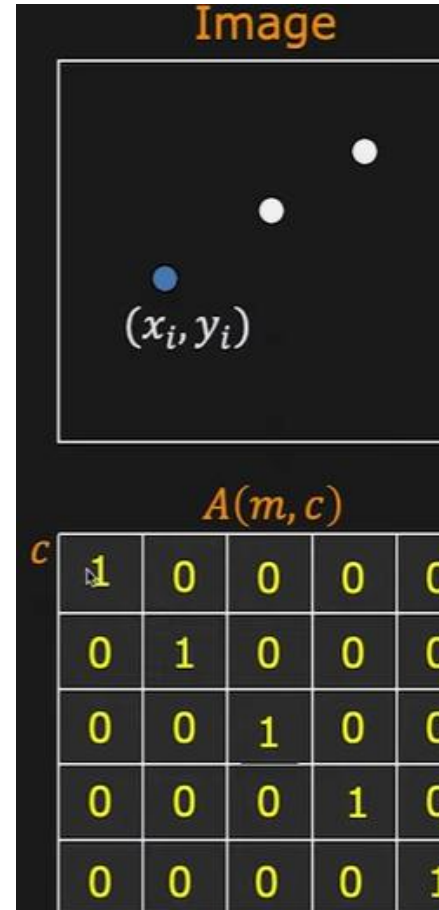
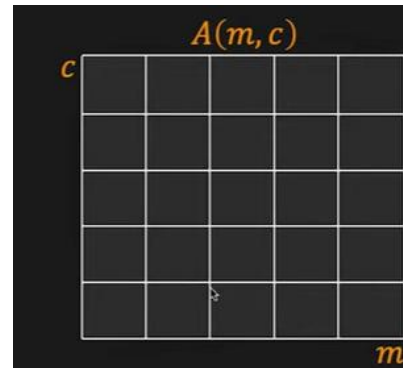
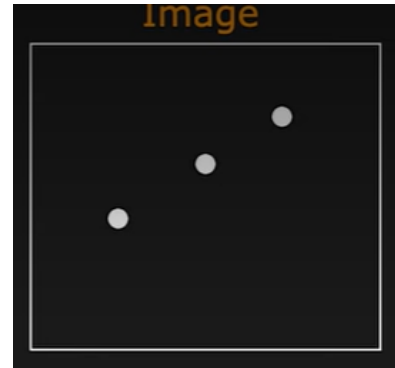
So, if you take only one point (x, y) in the parameter space it give you a line (each point of the blue line is a line that pass through your point in the image space)



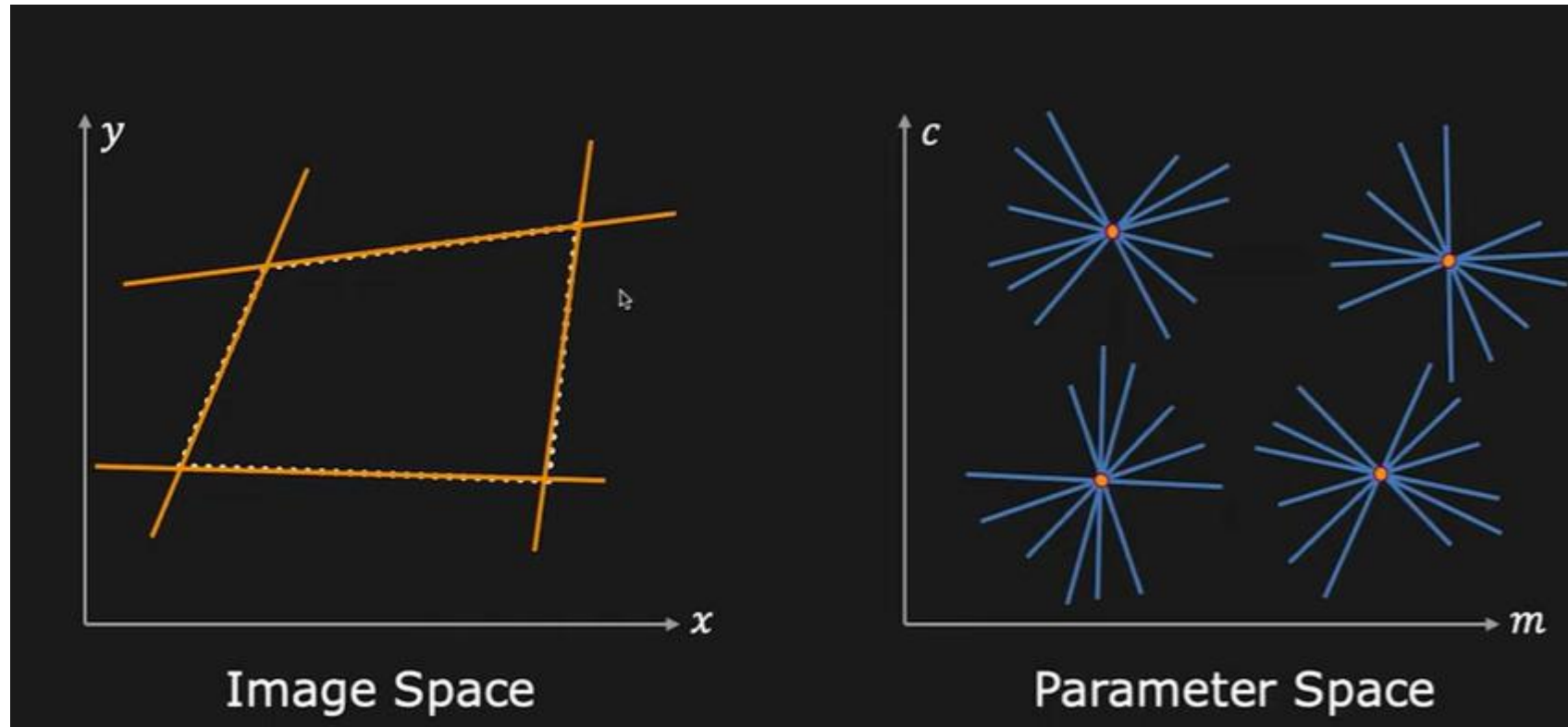
Hough Transform

Simple example with only 3 points.

- Create an accumulator array $A(m,c)$ in the parameter space
- Then, set everything to 0
- For each edge point, add 1 to the $A(m,c)$ if (m,c) is on the line $c = -mx + y$
- Find the maximum of $A(m,c)$, look at the m,c value and this corresponds to a line



Hough Transform



Line Detection : Hough Transform

Purpose: Detect straight lines in edge-detected images by answering which points align to form a straight line

Prerequisite: Use Canny edge detection first.

Hough Transform Principle:

- A line in Cartesian space (x, y) is represented in polar coordinates (ρ, θ) .
- Each point in the image votes for all possible lines it can lie on.
- Peaks in the Hough accumulator space correspond to the most likely lines.

Standard Hough Transform is computationally expensive since it evaluates all edge points.

Probabilistic Hough Line Transform improves efficiency by only using a subset of edge points, reducing computation and allowing faster detection

OpenCV: `cv2.HoughLines()` for standard lines

`cv2.HoughLinesP()` for probabilistic version:

- `threshold` = minimum number of intersections in Hough space.
- `minLineLength` = minimum line length.
- `maxLineGap` = max gap between line segments to treat them as one line

Circle Detection – Hough Transform

Purpose: Detect circles and estimate their center and radius using the Hough Circle Transform

Prerequisite: Use Canny edge detection first.

OpenCV: `cv2.HoughCircles()`
`cv2.HoughLinesP()` for probabilistic version:

- `cv2.HOUGH_GRADIENT`: Detection method
- `dp`: Inverse ratio of accumulator resolution to image resolution (e.g., 1.2)
- `minDist`: Minimum distance between circle centers
- `param1`: Upper threshold for Canny edge detection
- `param2`: Threshold for circle detection (lower = more circles)
- `minRadius`, `maxRadius`: Optional range for radius

Intro to PyTorch + CNNs for Classification

Session 3

Intro to PyTorch + CNNs for Classification

Session 3

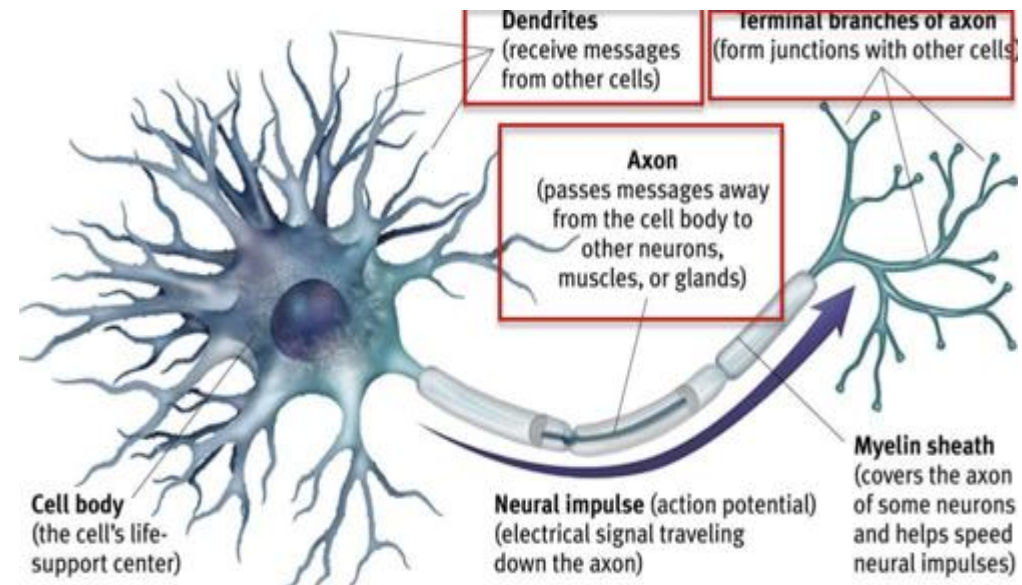
Objectives:

- Understand how artificial neural networks are inspired by the brain
- Learn how a vanilla neural network works through a complete mathematical example
- Break down the components of a CNN
- Learn to build and train CNNs using PyTorch

From Biological to Artificial Neurons

A biological neuron:

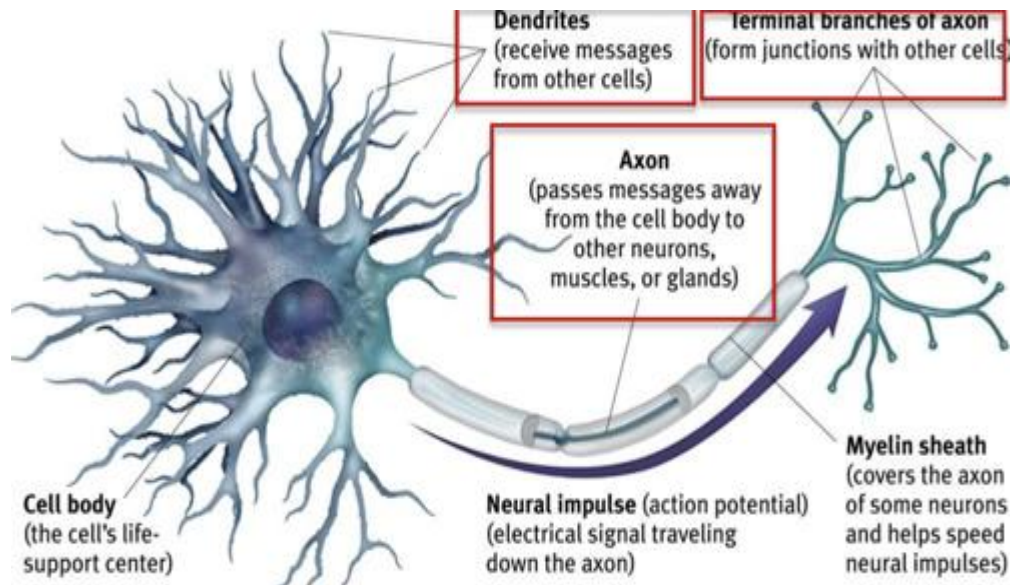
- receives signals via dendrites where each signal has a different synaptic strength (representing its importance)
- These signals are processed in the cell body, and if the **total input exceeds a certain threshold**,
- the neuron sends out an electrical signal via the axon.



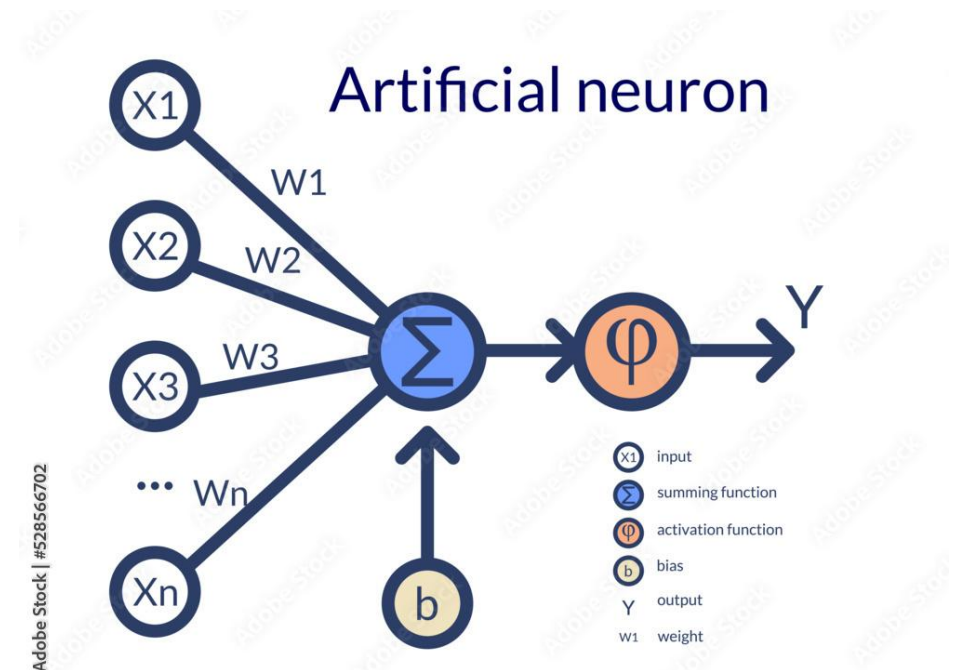
From Biological to Artificial Neurons

A biological neuron:

- **receives signals via dendrites** where each signal has a different **synaptic strength** (representing its importance)
- These signals are processed in the cell body, and if the **total input exceeds a certain threshold**,
- the neuron **sends out an electrical signal via the axon**.

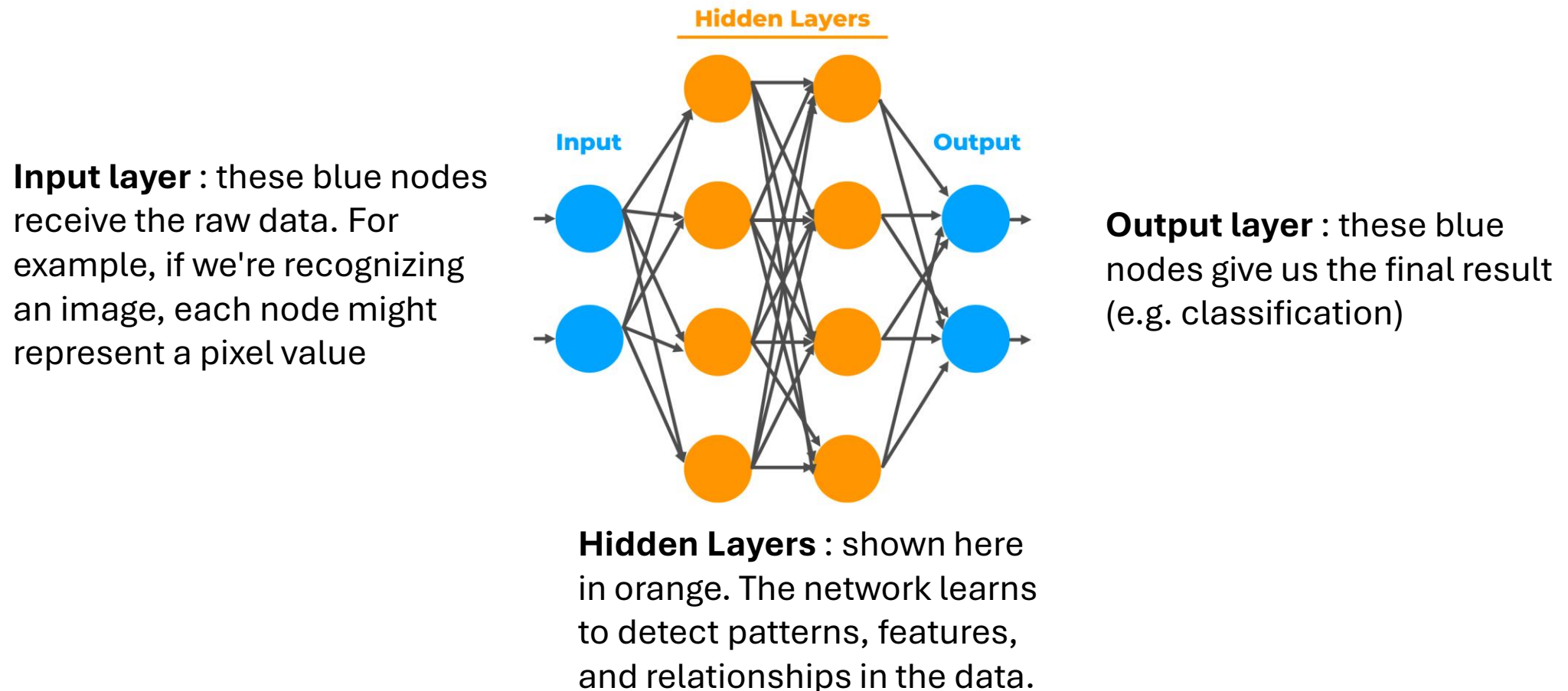


An artificial neuron: mimic this behavior using a mathematical formula: **a weighted sum of inputs**, plus a bias, passed **through an activation function**, and **give an output**



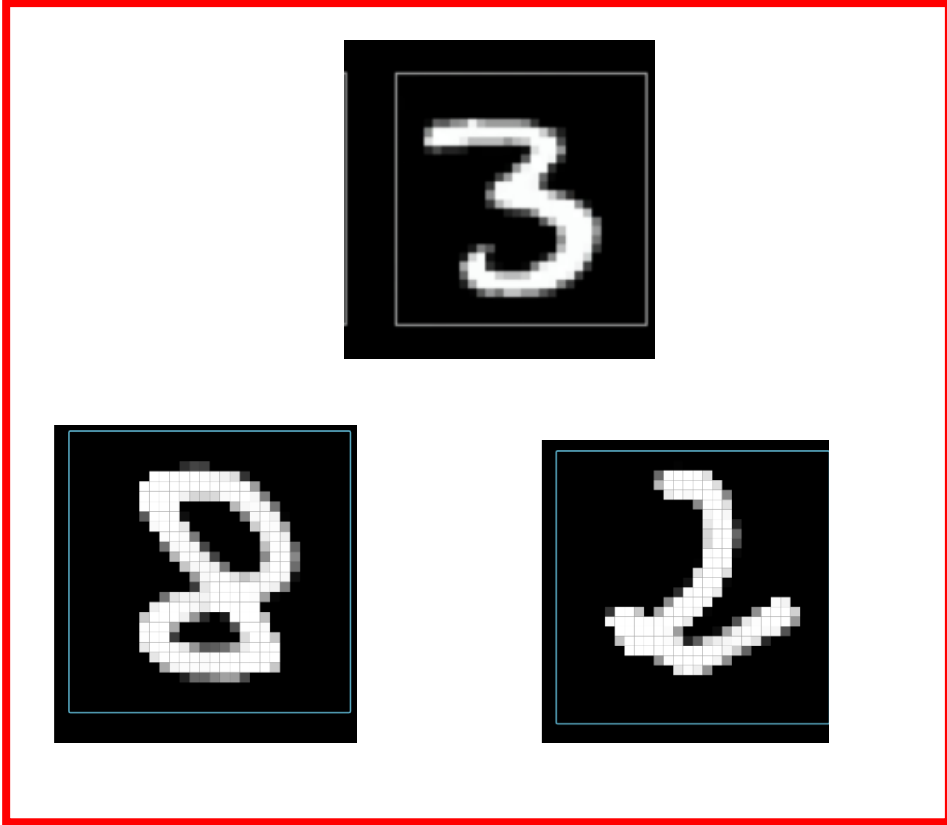
From Biological to Artificial Neurons

This diagram shows the basic structure of an **artificial neural network**

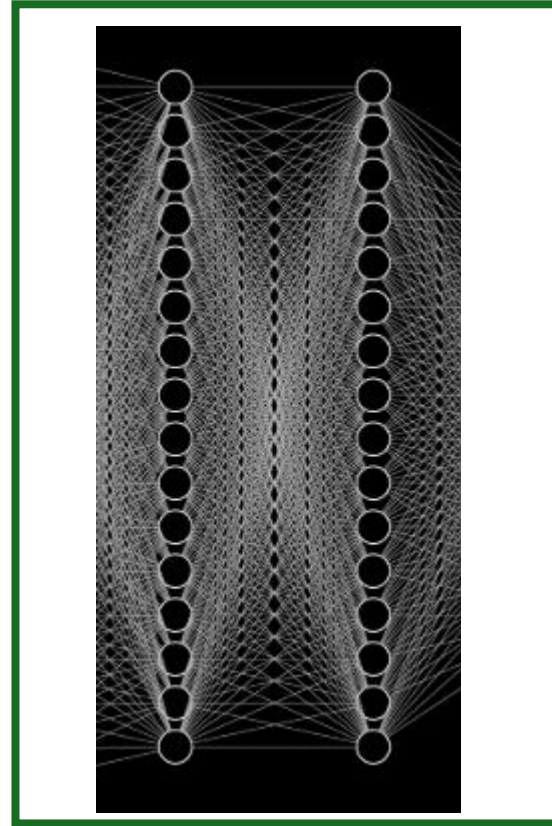


How Artificial Neural Networks work?

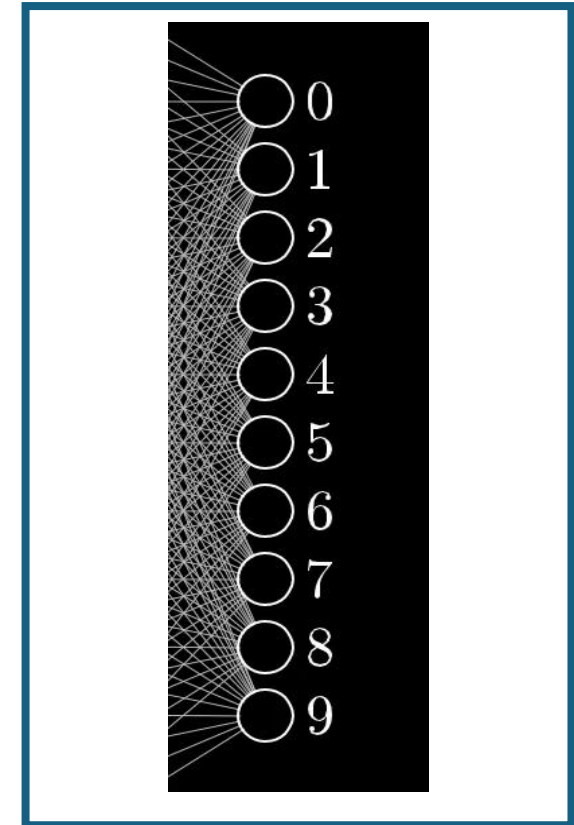
Create a NN to classify image?



INPUTS



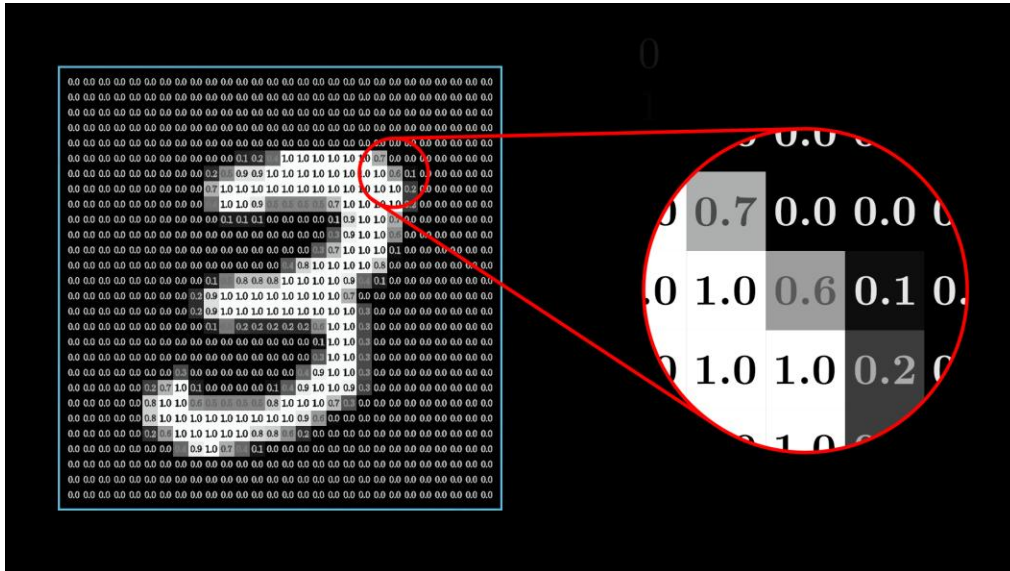
**Hidden
layers**



OUTPUTS

How Artificial Neural Networks work?

Create a NN to classify image?



One image is:
28x28pixels= 784 pixels

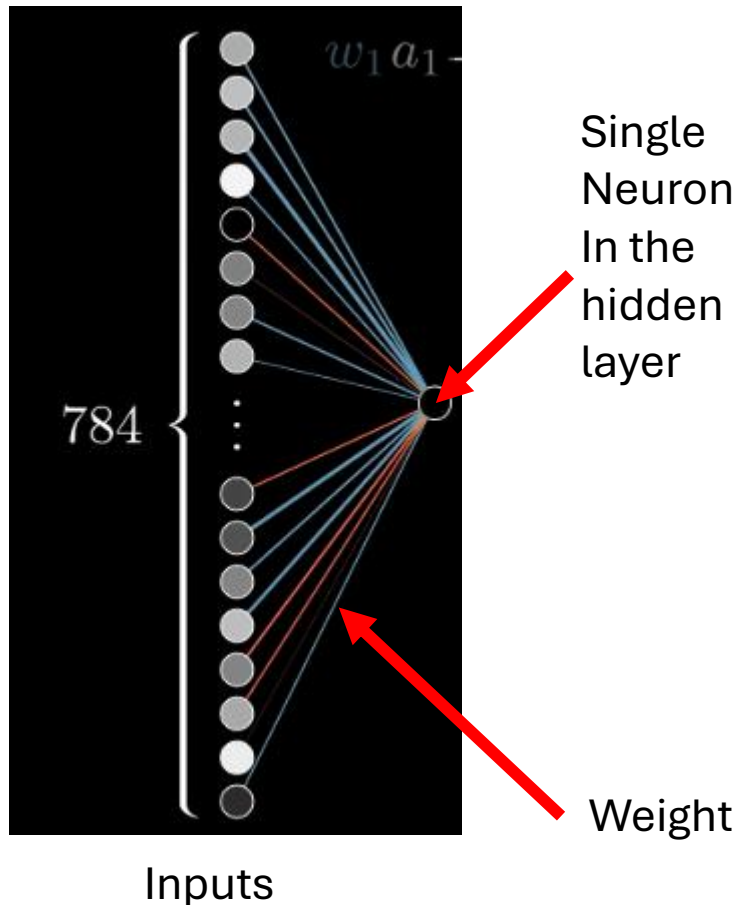


784 neurons of inputs



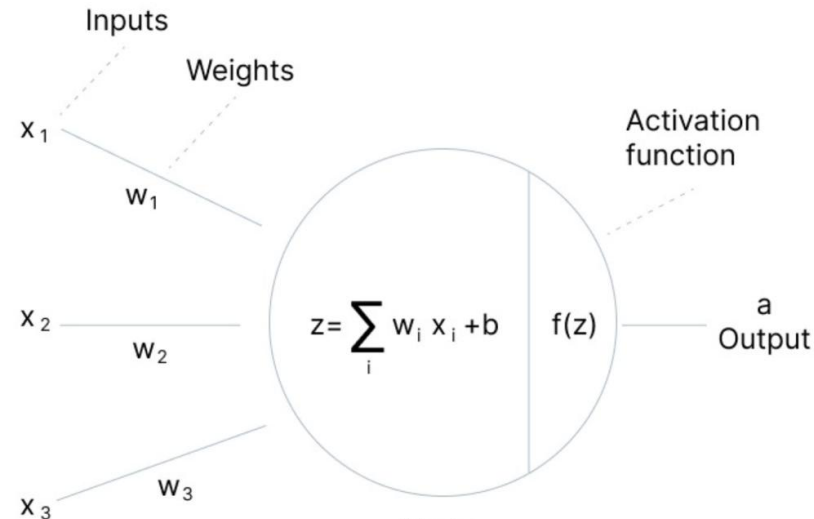
How Does a Neuron Compute?

Inside a Single Neuron



Each input has a different weight
(strength/importance):
 $w_1, \dots, w_n$

The output of each neuron is calculated by applying a weighted sum of its inputs (from the previous layer), adding a bias, and then passing this sum through an activation function (f).



Weight

What Are Weights in a Neural Network?

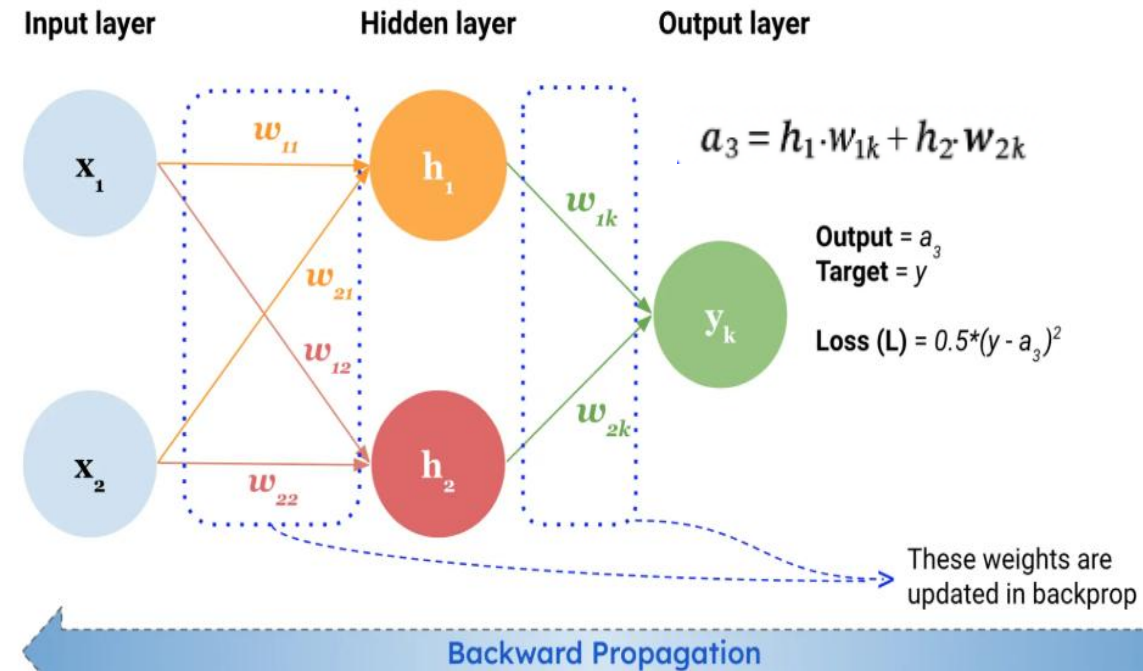
- Weights are the learnable parameters of a neural network.
- They determine how much influence each input feature has on the neuron's output.

During training, the network adjusts weights to minimize prediction error.

- A high weight = strong influence
- A zero weight = no effect

Weight pruning: is a technique used to reduce the complexity of an NN by removing weights that have little to no influence on the output, effectively setting them to zero

Importance: they directly influence the signals transmitted across the network and ultimately determine the network's output.



Bias

What is bias?

- The bias is added to the weighted sum of inputs.
- It shifts the activation function and allows neurons to activate even with zero inputs.

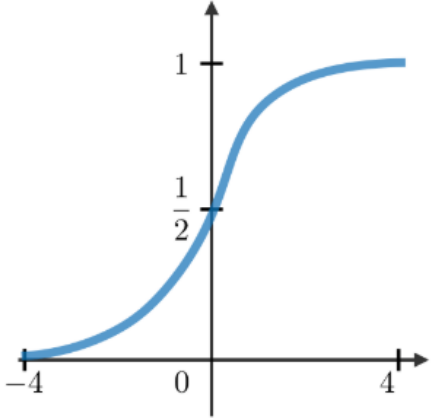
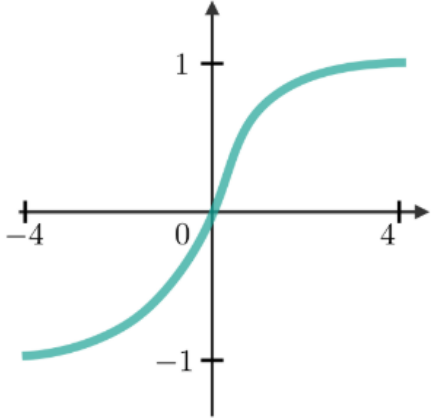
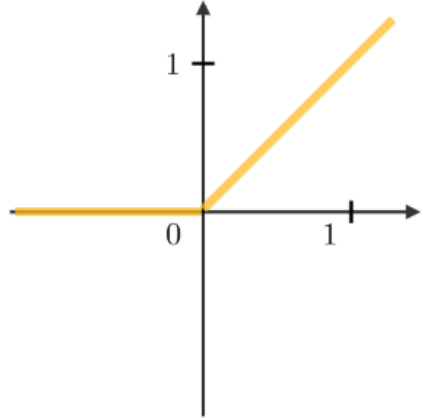
Why is bias important?

- Shifting the activation: Without bias, the output would always be zero when all inputs are zero, which can limit the network's ability to learn complex patterns.
- Improving flexibility: Bias allows the activation function to be shifted to better fit the data, enabling the network to model more complex relationships.

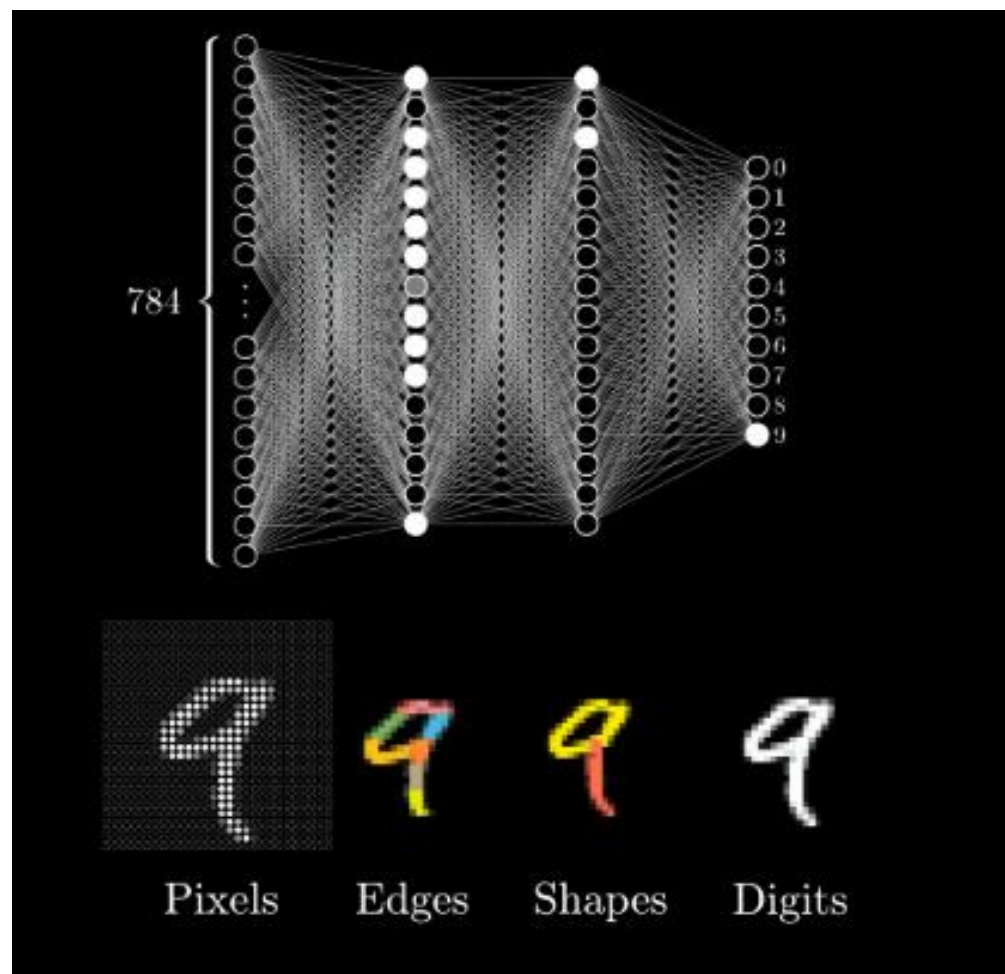
Activation function

The result of the weighted sum like this can be any number, but for this network we want the activations to be values between 0 and 1.

- So it's common to pump this weighted sum into some function that squishes the real number line into the range between 0 and 1
- Crucial as it introduces non-linearity into the model, allowing it to learn more complex patterns

activated if output > 0.5 Sigmoid	Tanh	activated if output > 0 ReLU
$g(z) = \frac{1}{1 + e^{-z}}$	$g(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	$g(z) = \max(0, z)$
		

All together



Limitation of the ANN

- Fully connected (vanilla) networks flatten the image and **ignore spatial structure**.
 - This leads to very large parameter spaces and poor generalization to spatial patterns.
- With ANN, image classification problems become difficult because 2-dimensional images need to be converted to 1-dimensional vectors.
- **Convolutional Neural Networks (CNNs) are designed for spatially structured data like images.**
- CNNs preserve spatial relationships by using small filters (kernels) that scan over the image.
- **Weight sharing and local connectivity drastically reduce parameters and improve generalization.**

CNNs for Classification

Convolutional Neural Networks (CNNs) are powerful architectures designed specifically for **image and spatial data**. Unlike standard neural networks that treat all inputs independently, CNNs process images by scanning small regions using **convolutional filters**. This allows them to **capture local patterns** like edges, shapes, and textures effectively.

A typical CNN is composed of the following layers:

- **Input Layer:**

Receives the image.

- **Convolutional Layer:**

Applies filters to extract features.

- **Activation Function:**

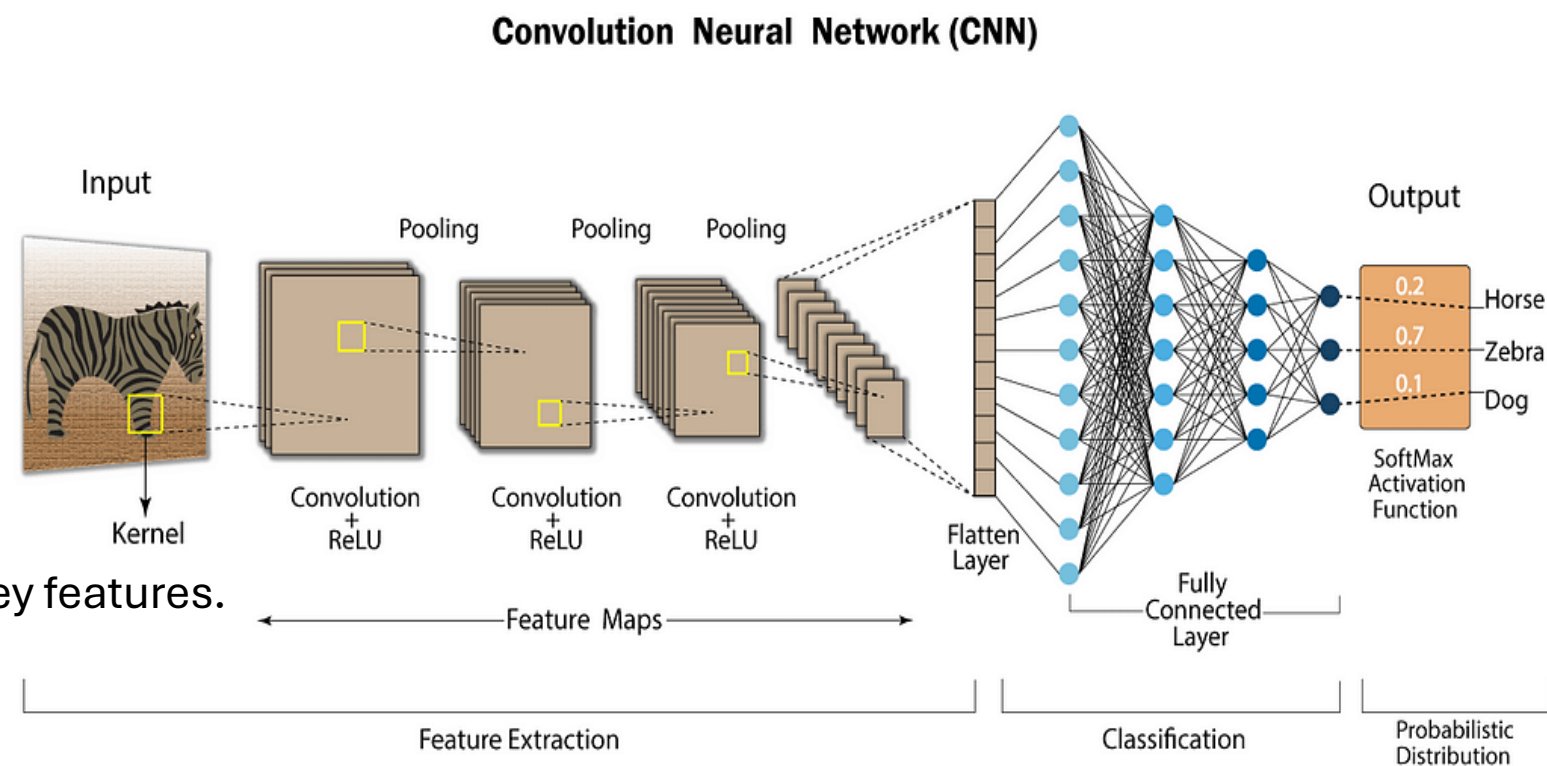
Introduces non-linearity (commonly ReLU).

- **Pooling Layer:**

Reduces dimensionality and emphasizes key features.

- **Fully Connected Layer:**

Performs the final classification.



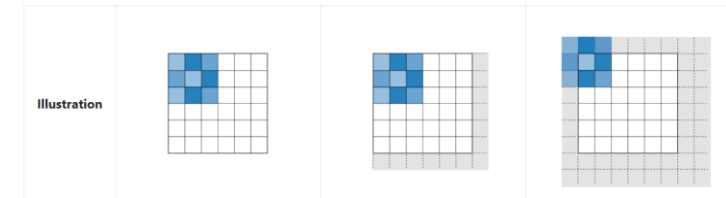
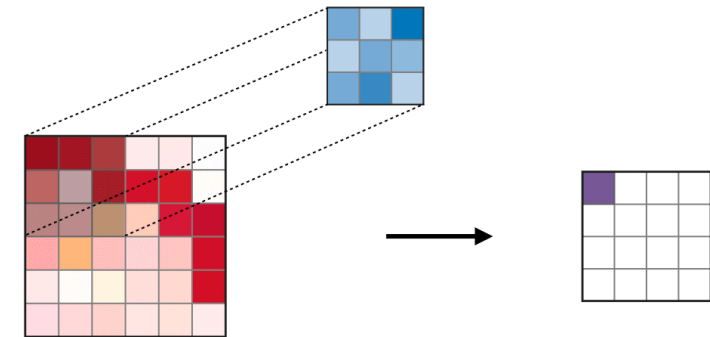
CNN: Convolutional Layer

This layer is responsible for extracting important features from the input image using a set of learnable filters/kernels (kernel are learned parameters!!)

- Applies a small learnable filter across the input image.
- Detects features like edges, corners, and textures.
- Uses the same weights at each location (weight sharing).

Its hyperparameters include:

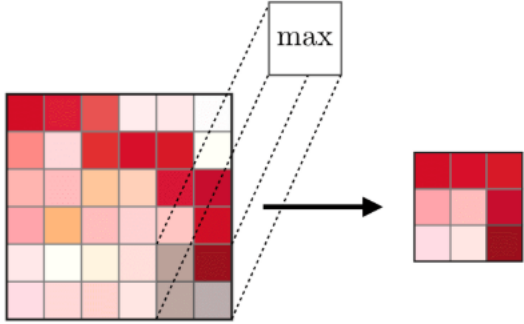
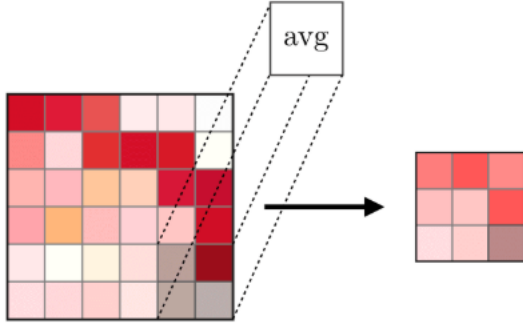
- Padding (P): adds a border of zeros around the input image. This helps control the spatial size of the output and preserves edge information when applying filters.
- Filter size (F): determines the spatial area each filter covers (e.g., 3×3)
- Stride (S): controls how far the filter moves across the image (e.g. 1 pixel) : larger strides reduce output size



Adding an activation function to the activation layers add nonlinearity to the network

CNN: Pooling

Pooling layers follow convolution layers and are used to reduce the spatial dimensions of feature maps while retaining important information.

Type	Max pooling	Average pooling
Purpose	Each pooling operation selects the maximum value of the current view	Each pooling operation averages the values of the current view
Illustration		
Comments	• Preserves detected features • Most commonly used	• Downsamples feature map • Used in LeNet

CNN

Benefits:

- Feature extraction: Automatically learns features from raw image data, reducing manual preprocessing.
- Spatial invariance: Can recognize patterns regardless of their location, orientation, or size.
- Robust to noise: Performs well even when input images are cluttered or noisy.
- Transfer learning: Pre-trained CNNs can be adapted to new tasks with less data and compute.
- High performance: Achieves state-of-the-art results in tasks like classification, detection, and segmentation.

Limitations:

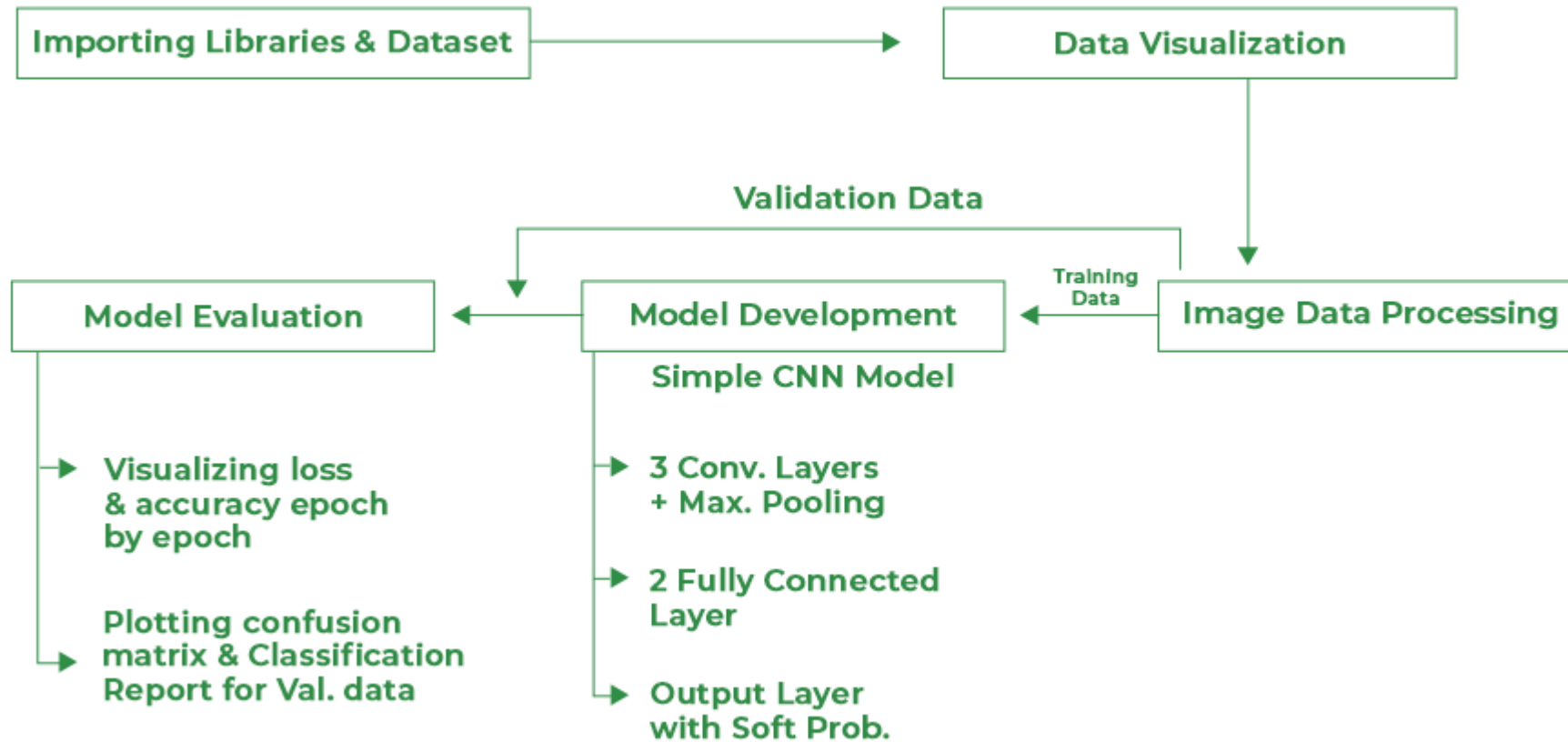
- Computational cost: Training deep CNNs can require a lot of resources and time.
- Overfitting: Prone to memorizing training data when datasets are small.
- Lack of interpretability: It's difficult to explain how or why a CNN made a specific decision.
- Structured input required: Works only with grid-like data (e.g., images), not irregular formats

Training a CNN

We will do the following steps in order:

- Load data (for pre defined dataset use torchvision)
- Clean your data: normalization, augmentation, resize, outlier rejection, etc
- Define a Convolutional Neural Network (number of con2D, pooling)
- Define a loss function to compare predictions to true labels
- Train the network on the training data
 - Compute the gradients of the loss with respect to the model parameters (weights and biases) using backpropagation.
 - Adjust the weights and biases using an optimizer (e.g., SGD, Adam) to minimize the loss.
 - Loop over all batches for multiple epochs until the model performance stabilizes.
- Test the network on the test data

Training a CNN



Overfitting

When a model learns the training data too well : including noise and details that don't generalize.

- Results in good training accuracy but poor performance on unseen (test) data.

How to prevent or reduce overfitting?

- More training data: Exposes the model to more variations.
- Data augmentation: Randomly change training images (rotate, flip, crop, etc.).
- Regularization: Add penalty terms to the loss (e.g., L2 weight decay).
- Dropout: Randomly disable some neurons during training.
- Early stopping: Stop training when validation loss starts to rise.
- Batch normalization: Helps by regularizing and stabilizing training.
- Simpler model: Use fewer layers or neurons if the model is too complex for the data.



Image
Augmentation



Batch normalization

Applied after convolutional or fully connected layers:

Normalizes intermediate activations to zero mean and unit variance

Why?

- **Stabilizes training:** reduces fluctuations in layer outputs, helping the model learn more consistently.
- **Reduces internal covariate shift:** Keeps the input distribution of each layer more stable during training.
- **Allows higher learning rates:** Makes training more robust to large updates, speeding up convergence.
- **Acts as a form of regularization:** Adds slight noise from mini-batch stats, which helps prevent overfitting.

Recommended in most CNNs for audio/image tasks

Introduction to Pytorch

PyTorch is a powerful open-source deep learning framework developed by Meta AI. It is widely used in research and industry for building and training neural networks.

Why?

- Dynamic computation graphs: More intuitive and flexible than static graphs (more intuitive debugging)
- Pythonic and easy to debug: Works seamlessly with NumPy and native Python tools
- Extensive GPU support: Easily switch between CPU and GPU with `.to(device)`
- Strong community and ecosystem: Includes torchvision, torchaudio, and PyTorch Lightning
- Widely adopted: Used in academia, industry, and production systems.

Feature Extraction & Segmentation

Session 4

Feature Extraction & Segmentation

Session 4

Objectives:

- Understand what feature extraction is and why it is important
- Identify different types of image features (edges, texture, color, shape)
- Explain the difference between local and global features
- Apply popular feature extraction algorithms (SIFT, ORB) in practice
- Describe and implement common image segmentation techniques
- Analyze and compare feature extraction and segmentation results

Introduction & Motivation

Why do we need features and segmentation in computer vision?

Introduction & Motivation

Why do we need features and segmentation in computer vision?

- Raw pixel values are not enough for most computer vision tasks.
- **Feature extraction** transforms images into numerical representations that a model can understand (**edges, textures, color, shapes**).
- **Segmentation isolates regions of interest** (e.g., separate an object from the background), making it easier to analyze or classify images.
- **These processes are foundational for tasks like recognition, tracking, medical analysis, and autonomous vehicles.**

Feature extraction

Definition: Identify important characteristics or properties of the segmented regions or the entire image. This process transforms raw data into a set of features that can be used for further analysis or machine learning. **However no need to extract features when using CNN (only for RF, SVM, etc..)**

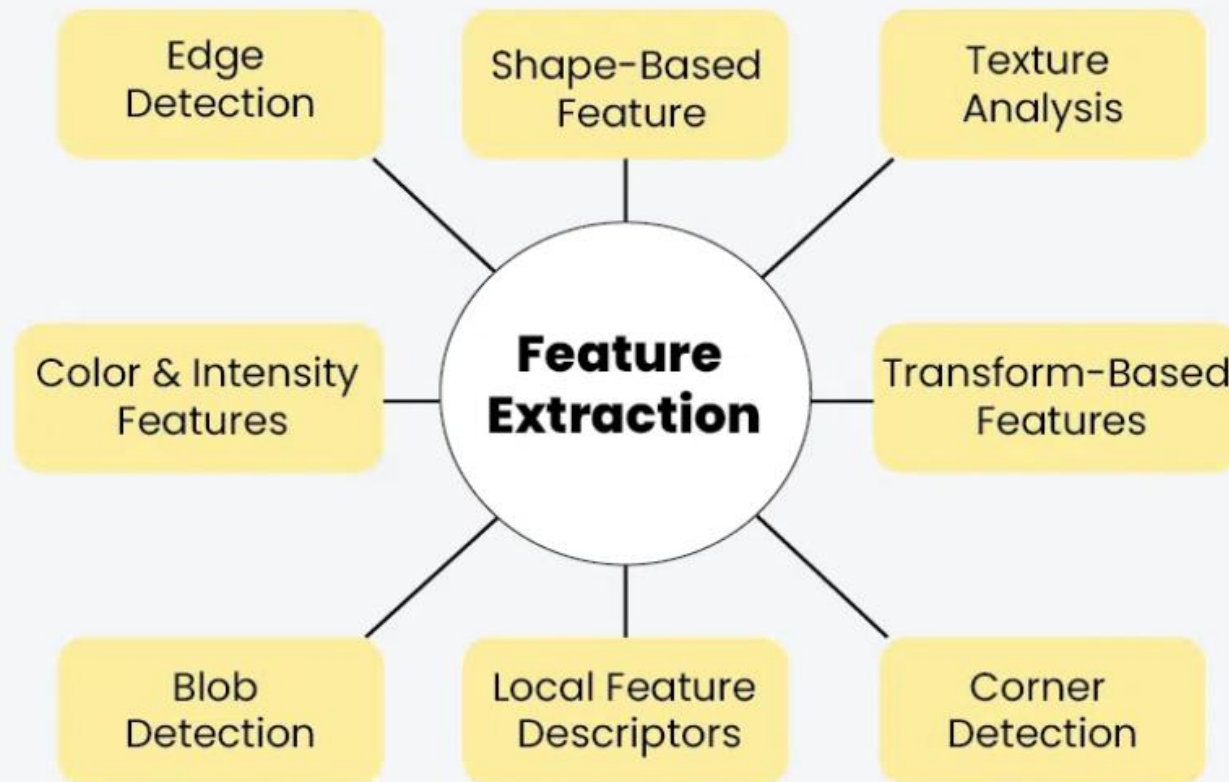
Purpose: Reduces the complexity of data to improves performance of learning algorithms (classification, recognition, and regression)

Type of features:

- **Edge Features:** Sudden changes in intensity, highlight object boundaries
- **Texture Features:** Repeated local intensity variations, capture material or surface type
- **Color Features:** Distribution of colors (RGB, HSV), useful for classification/tracking
- **Shape Features:** Object outlines, contours, geometric moments, helpful for object recognition

Feature extraction

Feature Extraction Techniques in Image Processing



Local vs. Global Features

Local Features: Calculated in small neighborhoods; robust to image changes

- SIFT, corners, blobs
- Matching and recognition

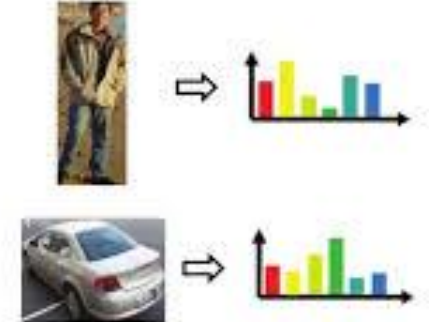
Global Features: Describe entire image; fast, less robust to change

- Color histogram, Gabor texture
- Image retrieval, scene classification

Local features



Global (holistic) features



In Practice:

- Use local features for detailed tasks (e.g., matching parts of two images)
- Use global features for broad similarity (e.g., find "all sunset images")

Edge detection (session 2)

Definition: Edges: Locations with strong intensity change.

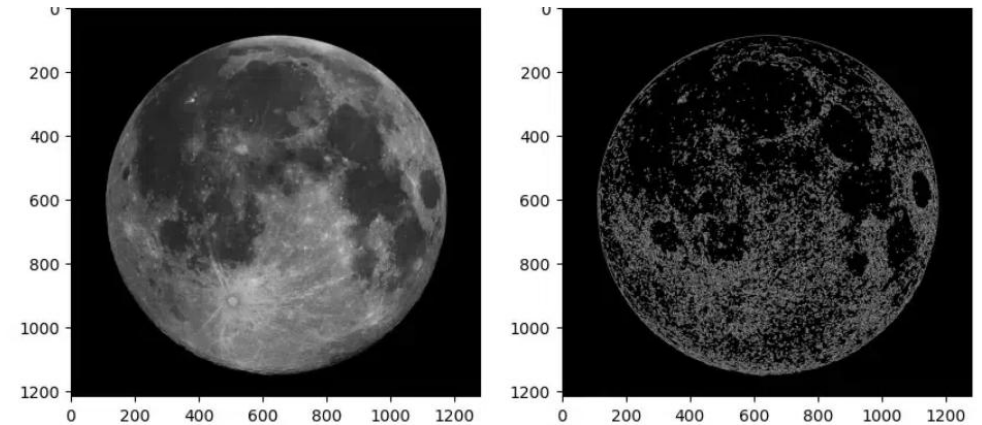
Common detectors: Sobel, Canny

Used for: object boundary detection

```
img = cv2.imread('moon.jpg',cv2.IMREAD_GRAYSCALE)
```

```
# Apply Canny edge detector
```

```
edges = cv2.Canny(img, 100, 200)
```



Edge Detection using OpenCV

Texture detection

What is Texture?

Texture is the “feel” of an image surface: patterns, granularity, and repeated structures that make it possible to distinguish materials like grass, sand, brick, or fur, even if their colors are similar.

Why is Texture Important?

Our eyes use texture all the time: Is that a smooth desk or a rough stone? Are you looking at grass or carpet, sand or concrete?

Texture helps us answer these questions visually, and the same goes for machines!

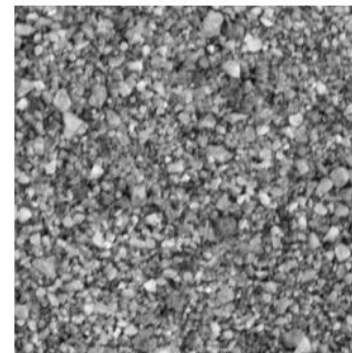
How Do We Capture Texture in Images?

Raw pixels don't say much about texture. Instead, we need to analyze how intensities vary locally:

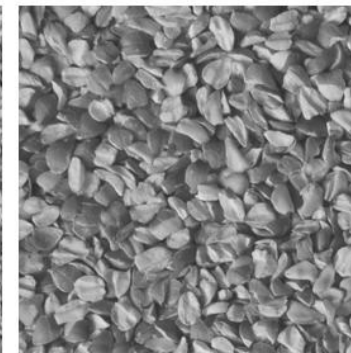
- Is there a repeated pattern?
- Are there rapid or slow changes in brightness?
- Does the pattern stay the same across a region?

Applications of Texture Analysis:

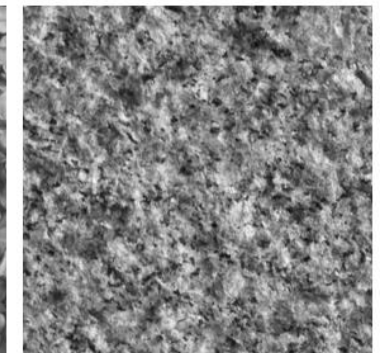
- Medical imaging: distinguish healthy and diseased tissue
- Industrial inspection: identify defects on surfaces
- Remote sensing: classify forests, water, and urban areas
- Face recognition, fingerprint analysis, document analysis



Sand



Seed



Stone

Texture detection

Methods:

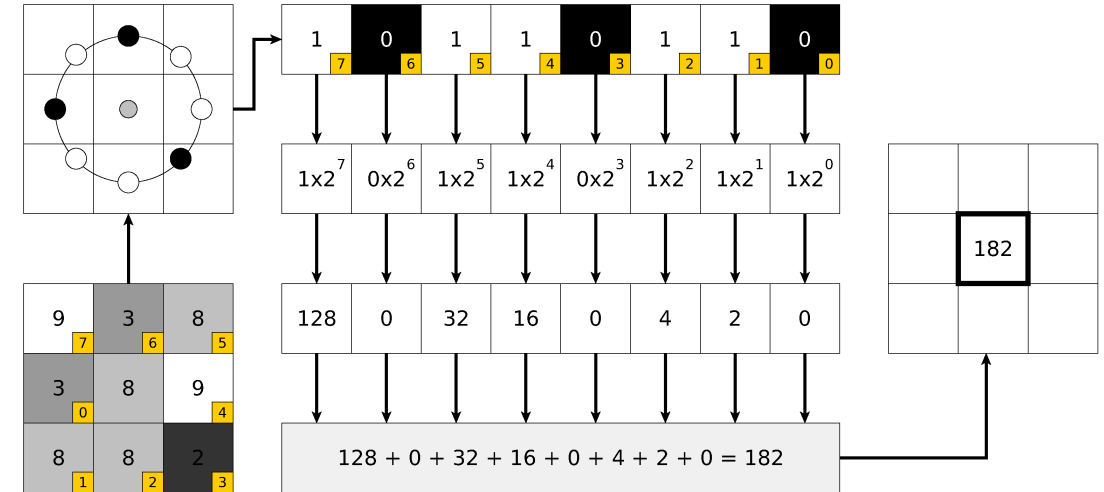
- **Local Binary Patterns (LBP):** Compares each pixel with its neighbors to create a simple code: fast and robust.
- **Gabor Filters:** Analyze textures at different scales and orientations (like tuning to different “frequencies”).
- **Haralick Features:** Capture more complex statistics of texture.

Local Binary Patterns (LBP):

Purpose: Simple, fast way to describe local texture. Robust to illumination changes, excellent for material/surface recognition.

How it works:

- For each pixel, compare its value with its 8 neighbors.
- If a neighbor is greater or equal, mark “1”; else, “0”.
- Read the 8 bits as a binary number → LBP code.
- Every pixel is replaced by its LBP code.
- Extract features from the histograms



Applications:

Face recognition, defect detection, medical imaging, surface inspection.

Variants

Multi-scale LBP

Rotation-Invariant LBP

Address scale and rotation issues.

Local Binary Patterns (LBP):

In Python `local_binary_pattern(img, P, R, METHOD)`

- **P:** Number of circularly symmetric neighbor set points (typically 8)
- **R:** Radius of circle (1 pixel)
- **METHOD:**
 - **default:** Original local binary pattern which is grayscale invariant but not rotation invariant.
 - **ror:** Extension of default pattern which is grayscale invariant and rotation invariant.
 - **uniform:** Uniform pattern which is grayscale invariant and rotation invariant, offering finer quantization of the angular space. For details
 - **nri_uniform:** Variant of uniform pattern which is grayscale invariant but not rotation invariant.
 - **var:** Variance of local image texture (related to contrast) which is rotation invariant but not grayscale invariant.

Haralick Features:

Haralick features are a family of texture descriptors based on the **Gray-Level Co-occurrence Matrix (GLCM)**, introduced by **Robert Haralick** in **1973**.

GLCM: Measures how often pairs of pixels with specific values (gray levels) occur at a certain spatial relationship (distance and angle) in an image. This matrix records *how many times two gray-level pixels adjacent to each other appear* in an image. Then based on this matrix, Haralick proposes 13 values that can be extracted from the GLCM to *quantify texture*.

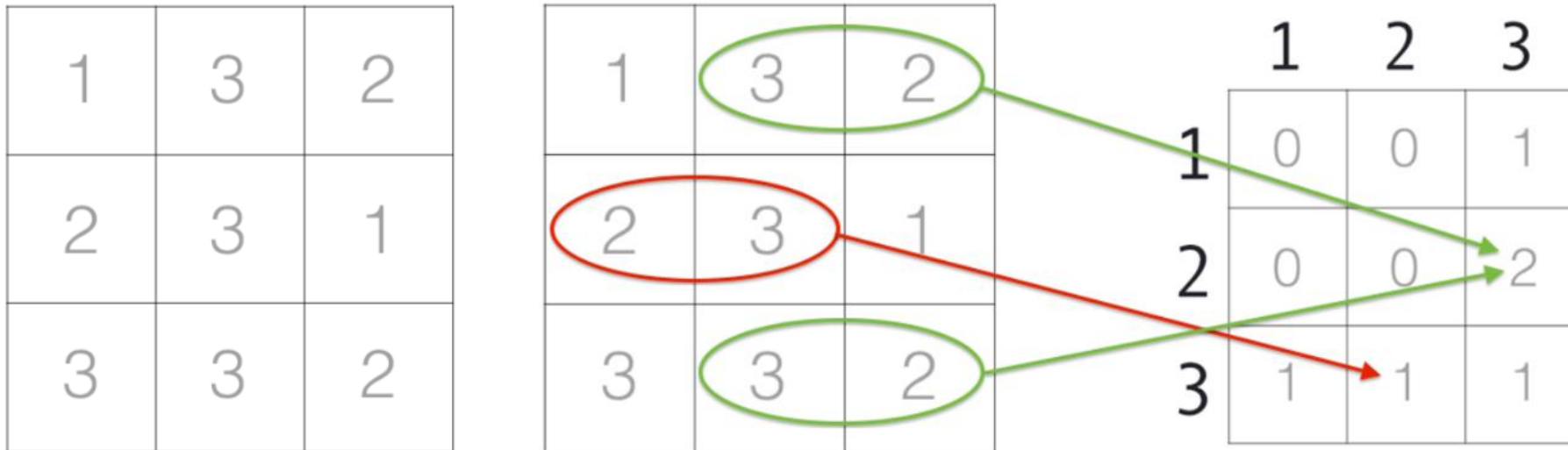


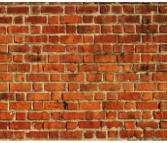
FIGURE 1: OUR INPUT IMAGE, WHERE EACH CELL IN THE GRID REPRESENTS A GRAYSCALE PIXEL INTENSITY.

the pixel intensities *2* and *3* appear next to each other *once*, so they have an entry of *1* in the GLCM (row 2 col 3). The pixel intensities *3* and *2*, however, appear next to each other *twice*, so they have a corresponding entry of *2* in the GLCM (row 3 col 2)

Haralick Features:

Feature	What it measures	Ordering (High → Low + justification)
Energy (ASM)	Uniformity	Bricks > Grass > Gravel : bricks are highly repetitive, gravel is irregular
Contrast	Intensity variation	Gravel > Grass > Bricks : gravel has sharp local changes, bricks are smooth
Correlation	Pattern regularity	Bricks > Grass > Gravel : bricks follow consistent repeating structure
Variance	Gray-level spread	Gravel > Grass > Bricks : gravel has wide intensity range, bricks are uniform
Homogeneity	Local similarity	Bricks > Grass > Gravel : bricks have similar neighboring pixels
Sum Variance	Sum variability	Gravel > Grass > Bricks : irregular textures create larger sum fluctuations
Sum Entropy	Disorder in sums	Gravel > Grass > Bricks : gravel is most chaotic in intensity combinations
Entropy	Randomness	Gravel > Grass > Bricks : gravel is most disordered, bricks most structured
Difference Variance	Local differences	Gravel > Grass > Bricks : gravel shows strongest pixel-to-pixel changes
Difference Entropy	Difference randomness	Gravel > Grass > Bricks : gravel differences are least predictable
Information Measure Corr.	Pixel dependency	Bricks > Grass > Gravel : bricks have strongest spatial dependency
Max Correlation Coefficient	Repeating structure strength	Bricks > Grass > Gravel : bricks show strongest periodic patterns

Bricks



Grass



Rocks



Gabor filter:

What is a Gabor Kernel?

A Gabor kernel is a type of bandpass filter, i.e. it passes frequencies in a certain band and attenuates the other frequencies outside such band.

Mathematically:

A Gabor kernel is a Gaussian envelope (which controls the size and shape) multiplied by a sinusoidal wave (which controls frequency and orientation).

$$g(x, y; \lambda, \theta, \psi, \sigma, \gamma) = \exp\left(-\frac{x'^2 + \gamma^2 y'^2}{2\sigma^2}\right) \exp\left(i\left(2\pi\frac{x'}{\lambda} + \psi\right)\right)$$

How It Works:

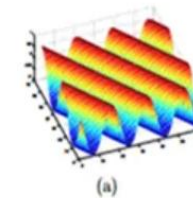
Gaussian envelope: - Defines the spatial support (“window”) of the filter
- Controls size, shape, and spatial locality

Sinusoidal wave: - Sets the frequency (how fine/coarse the patterns the filter detects)

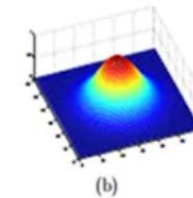
Orientation: Rotating the sinusoid makes the filter sensitive to patterns at different angles

What Does This Mean for Images?

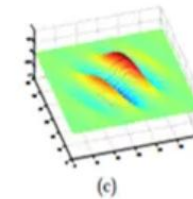
The Gabor filter is sensitive to texture at a specific scale (frequency) and direction (orientation). It acts as a “texture microscope,” highlighting image features that repeat at particular angles or spacings.



A Sinusoid oriented 30° with X-axis



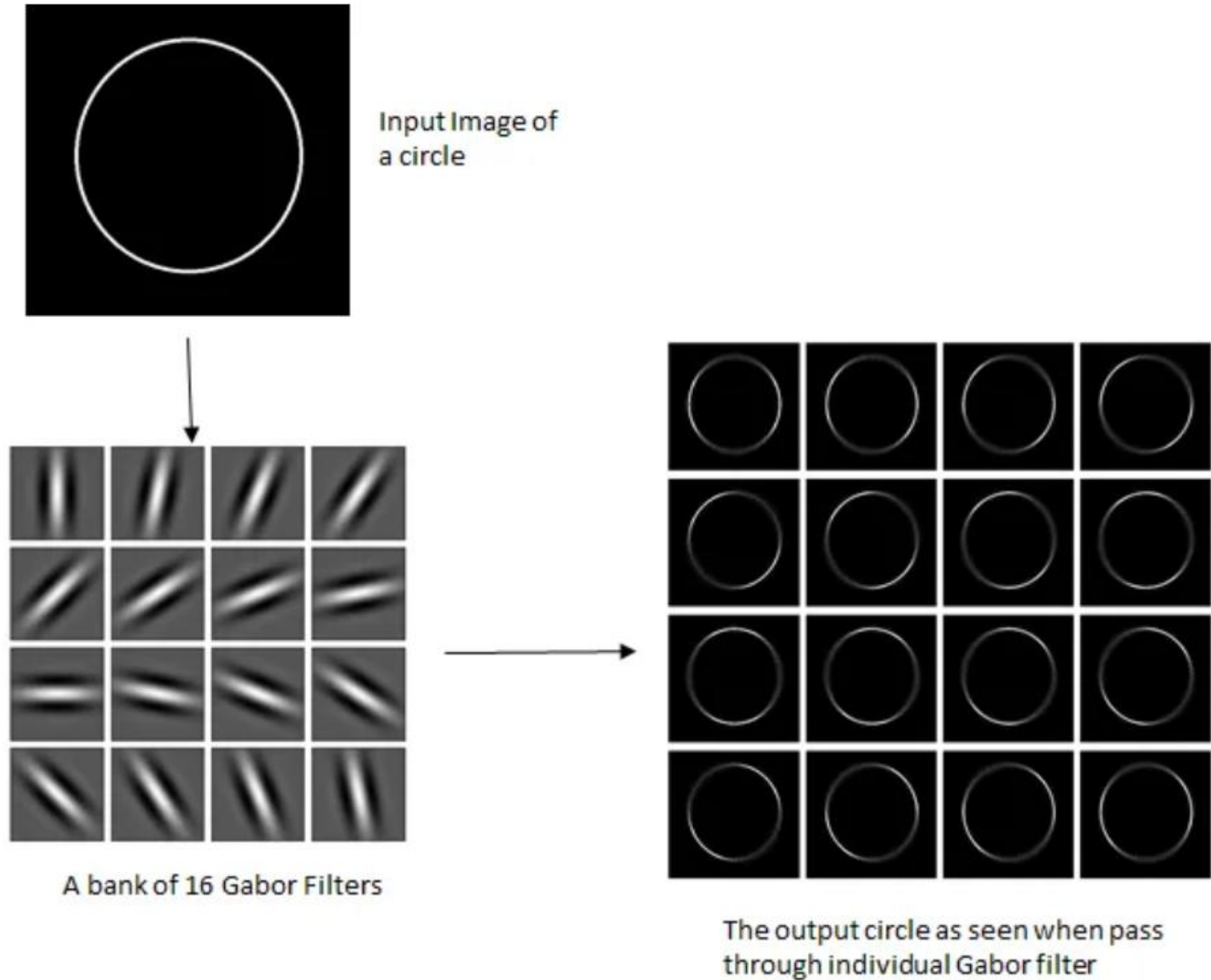
A 2-D Gaussian



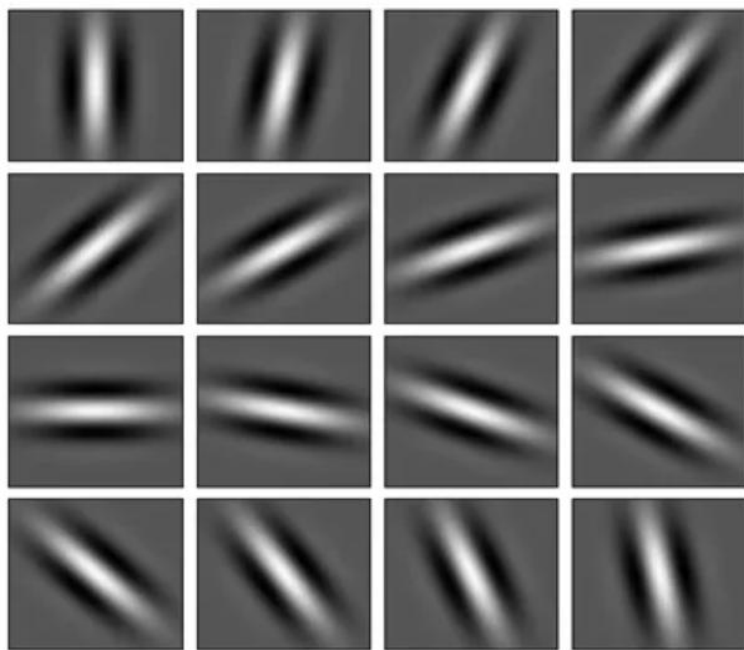
The corresponding 2-D Gabor filter

Gabor filter:

When this image is passed through each filter in the filter bank, the edge of the circle which gets detected is the edge oriented at an angle at which the Gabor filter is oriented



Gabor filter:

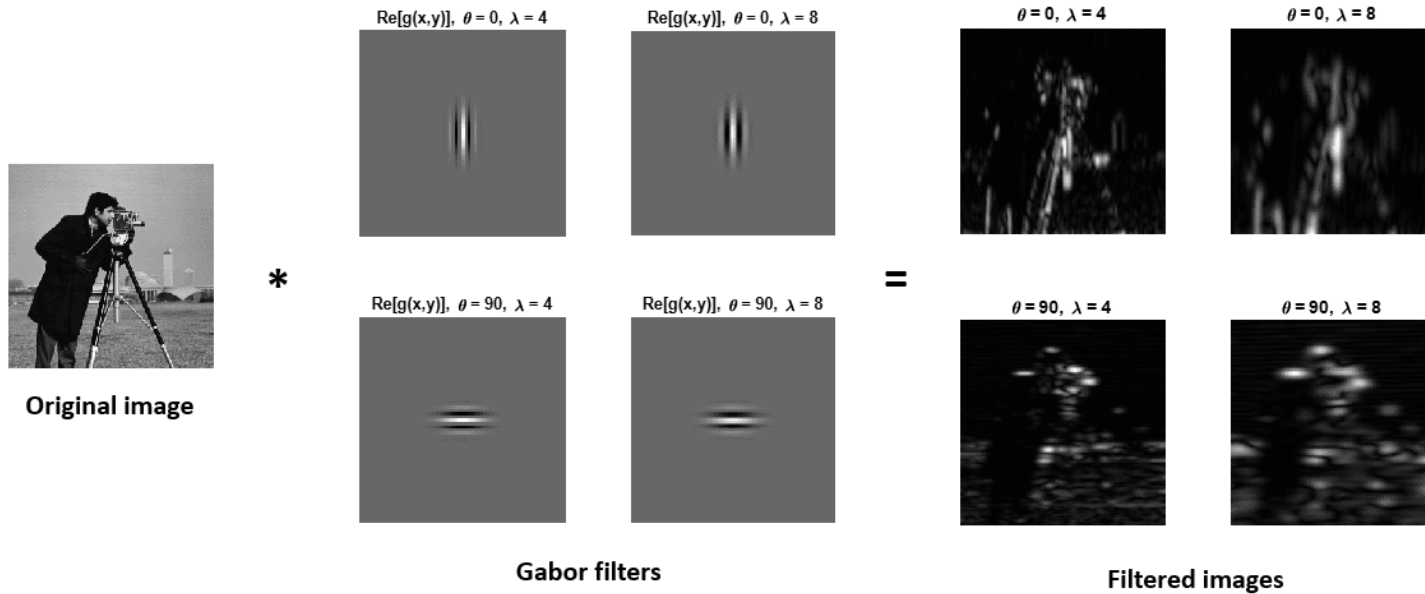


a

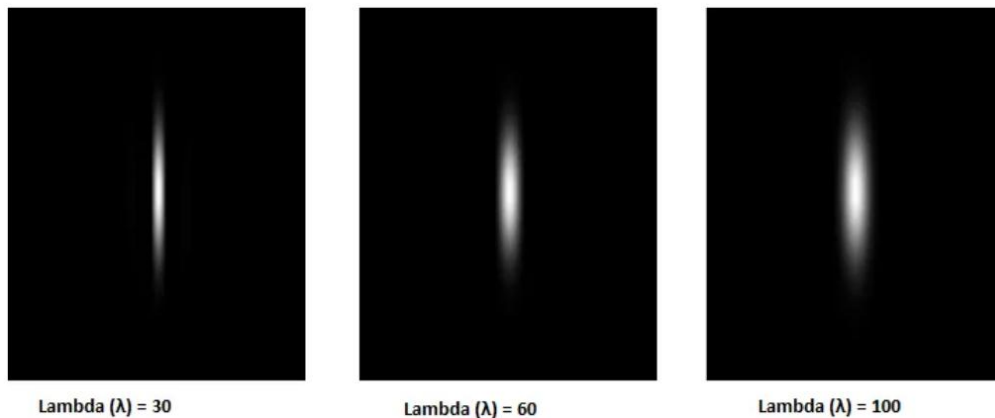


b

Gabor filter:



Horizontal and vertical features are extracted with $\theta=0$ and $\theta=90$. Finer details with lower wavelength values.



The wavelength governs the width of the strips of the Gabor function. Increasing the wavelength produces thicker stripes and decreasing the wavelength produces thinner stripes.

Gabor filter:

Import required libraies

import cv2

import numpy as np

import matplotlib.pyplot as plt

kernel size

ksize = 31

sigma for Gaussian envelope

sigma = 5

range of orientation values

theta_range = np.arange(0, np.pi, np.pi / 4)

frequency of sinusoidal wave

frequency = 0.3

phase of sinusoidal wave

phase = 0

for loop for generating different rotation

for theta **in** theta_range:

 kernel = cv2.getGaborKernel(ksize, sigma, theta, lambda, gamma, psi, ktype)

 plt.imshow(kernel,

 cmap='gray')

 plt.title("Kernel with orientation = " + str(theta))

 plt.show()

- **ksize** is the size of the Gabor kernel. If ksize = (a, b), we then have a Gabor kernel of size a x b pixels. As with many other convolution kernels, ksize is preferably odd and the kernel is a square (just for the sake of uniformity).
- **sigma** is the standard deviation of the Gaussian function used in the Gabor filter.
- **theta** is the orientation of the normal to the parallel stripes of the Gabor function.
- **lambda** is the wavelength of the sinusoidal factor in the above equation.
- **gamma** is the spatial aspect ratio.
- **psi** is the phase offset.
- **ktype** indicates the type and range of values that each pixel in the Gabor kernel can hold

Color features

What is color feature?

Color features describe the distribution of color information in an image, helping machines recognize, compare, and classify scenes or objects based on their color content.

Why is Texture Important?

Many objects/scenes are easily distinguished by color: think of fruit, skies, traffic lights, etc.

Method?

- Color histogram
- Color moments
- Color space

Color histogram

What is a Color Histogram?

A color histogram represents the distribution of pixel colors in an image.

It shows how many pixels fall into each range (bin) of color intensity for one or more color channels.

Can be computed per **channel (R, G, B)**, or in other **color spaces (HSV, Lab)**.

How Is It Computed?

- Divide the possible pixel values (0–255 for 8-bit images) into bins.
- Count the number of pixels whose value falls into each bin

Why Use Color Histograms?

- Scene Recognition:
 - Quickly tells you if an image is “mostly blue” (ocean/sky), “mostly green” (forest/field), or “mostly red” (sunset/flowers).
- Image Retrieval:
 - Find images with similar color “profiles” even if the content is different.
- Tracking/Detection:
 - Track colored objects (e.g., a red ball in sports footage).

Color moment

Color moments are statistical summaries:

- **Mean:** The average color value per channel (B, G, R).
- **Standard deviation:** How much the color varies (spread).
- **Skewness:** Whether the color is biased toward low or high values (asymmetry).
 - Skewness ≈ 0 : The histogram is fairly symmetric.
 - Skewness > 0 : The distribution has a longer right tail (few high-intensity pixels).
 - Skewness < 0 : The distribution has a longer left tail (few low-intensity pixels)

Motivation:

- **Compact feature:** Only a few numbers per image.
- **Used for:** Fast image retrieval, scene comparison, input to ML models.

Dominant Colors with K-means Clustering

What is K-means?

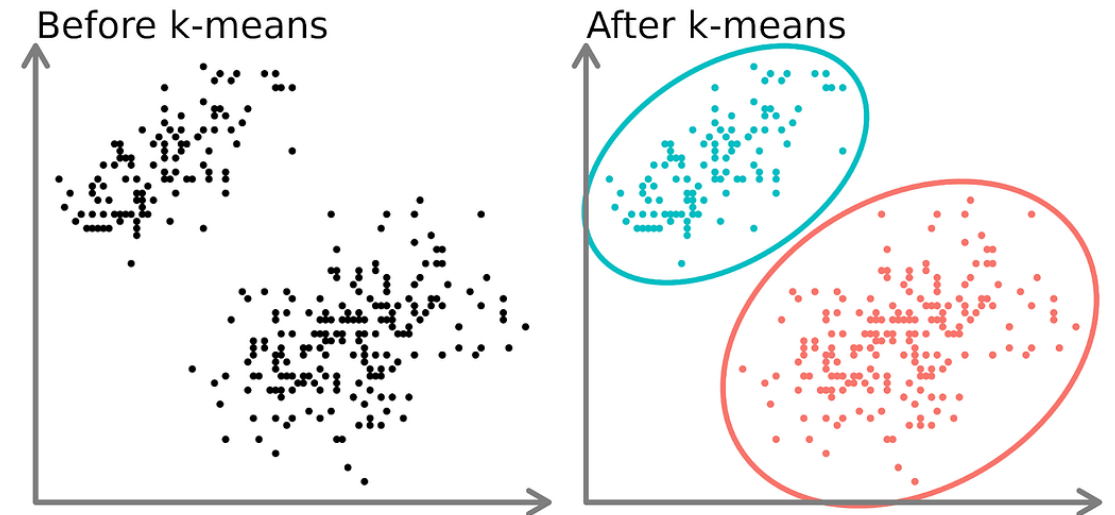
An unsupervised machine learning algorithm that groups data points (e.g., pixels) into K clusters based on their similarity.

How does it work?

- Choose K (number of clusters).
- Randomly initialize K cluster centers (centroids).
- Assign each pixel to the nearest
- Recompute centroids as the mean of assigned pixels.
- Repeat steps 3–4 until assignments stabilize.

Limitations:

- Must choose K in advance (try the “elbow method”).
- Sensitive to initial centroids.
- May struggle with complex or non-spherical clusters.



Shape features

Shape features describe the geometric properties of objects or regions in an image and help recognize, classify, and analyze objects regardless of their color or texture.

The two most common tools:

Contours: Boundaries of shapes/objects (see session 2).

Moments: Mathematical properties:

- **Area:** Size of the shape.
- **Centroid:** Center point.
- **Orientation/Eccentricity:** Main direction or “tilt”.
- **Hu Moments:** Set of 7 values that are invariant to translation, rotation, and scale (good for matching shapes)

Feature extractions

There are various techniques and algorithms used for feature detection and extraction. One popular approach is the use of **local feature detectors**, such as the **Scale-Invariant Feature Transform (SIFT)**, **Speeded-Up Robust Features (SURF)**, and **Oriented FAST and Rotated BRIEF (ORB)**. These methods aim to identify and extract features that are **invariant to changes in scale, rotation, and illumination**.

Descriptor	Invariant to	What for?	Note
SIFT	Scale, rotation	Keypoint matching, stitching	Patented (not for commercial use in old OpenCV, now patent expired)
SURF	Scale, rotation	Fast detection	Patent/licensing issues
ORB	Rotation	Real-time, patent-free	Fast, good for robotics
BRIEF	None (very fast)	Fast matching	Used with FAST keypoints
HOG	Illumination	Pedestrian/vehicle detection	Used in object detectors

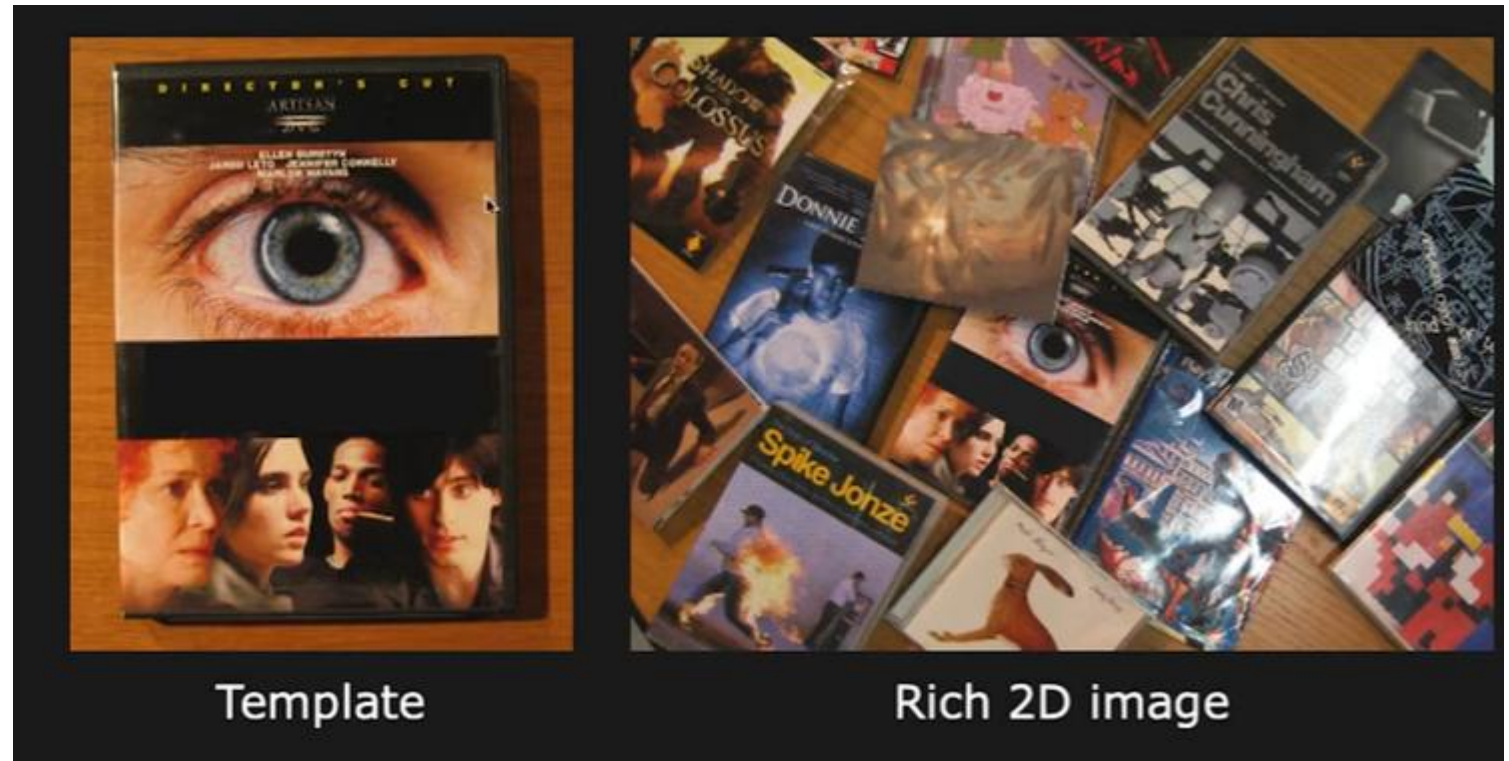
Why you need to be invariant to scale, rotation?

You want to find an object into an image.

You can take a template and try to match **BUT** object does not show up exactly as the template (rotation, illumination, scaling, etc...)

You have to create a lot of template to have all the possibilities....

You have also occlusion



**BEST SOLUTION FIND DESCRIPTIVE
UNIQUE FEATURES
-> SIFT DETECTOR**

What is an interesting point?

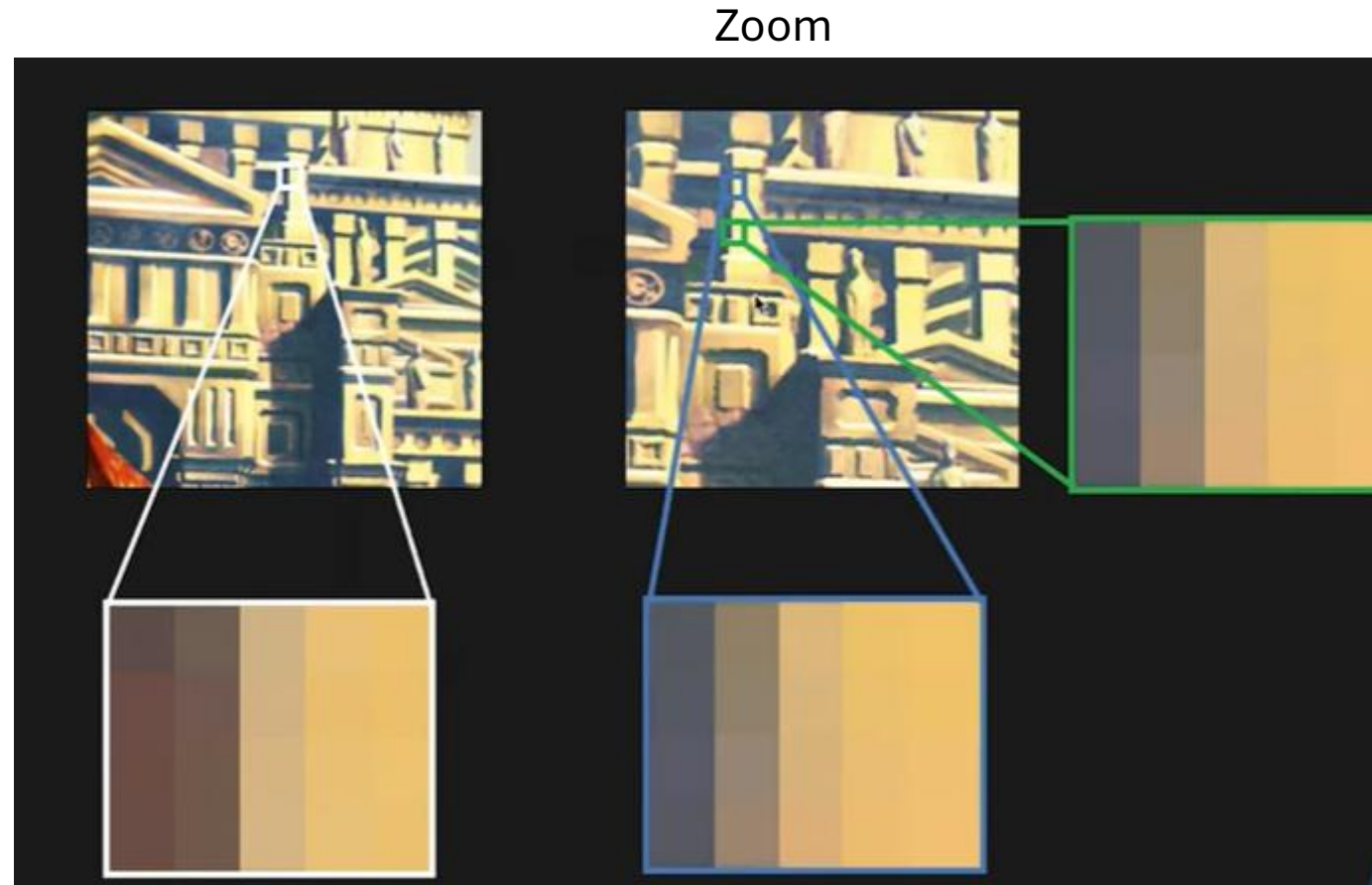
- Has **rich image content** (brightness variation, color variation, etc.) within the local window
- Has well-defined **representation (signature)** for matching/comparing with other points
- Has a well-defined **position** in the image
- Should be **invariant to image rotation and scaling**
- Should be **insensitive to lighting** changes

Are lines and edges interesting?

Are lines and edges interesting?

Edge are not interesting because you can find many edges that they have the same kind of brightness variation (along the boundary)

NOT UNIQUE

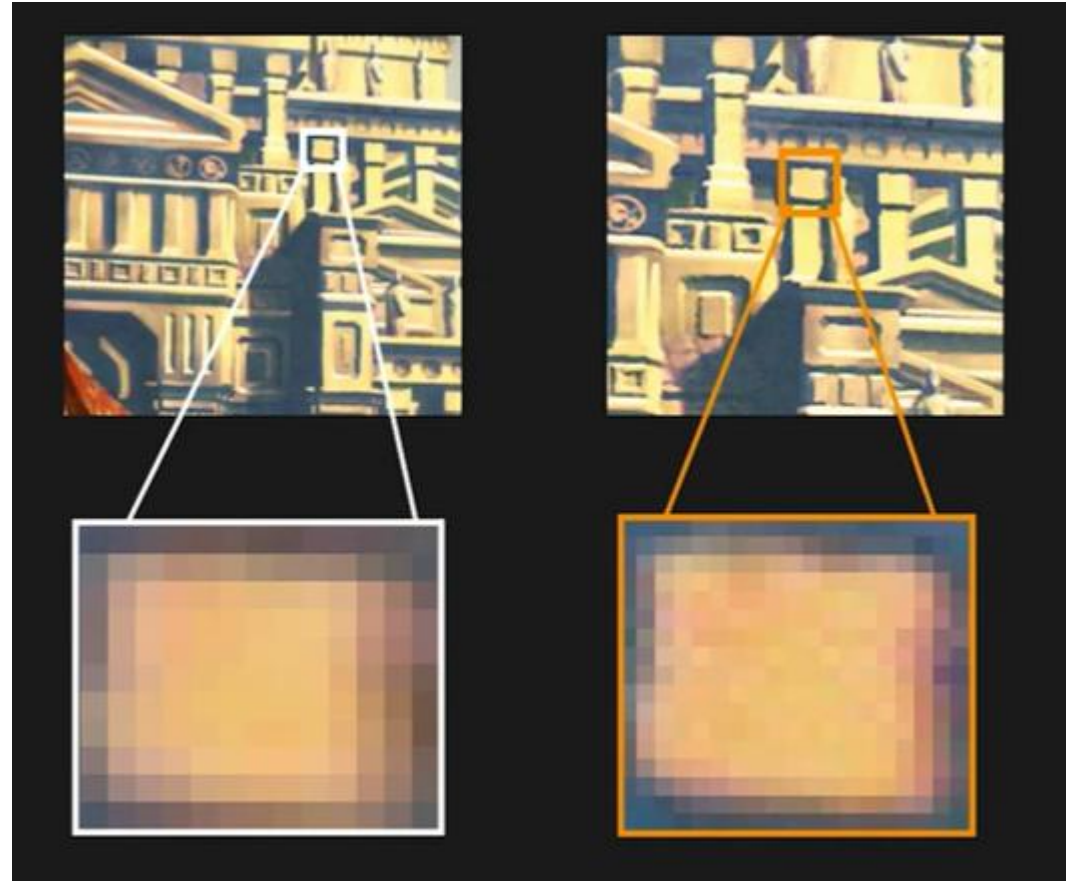


Rapid change in
brightness

Are blobs interesting?

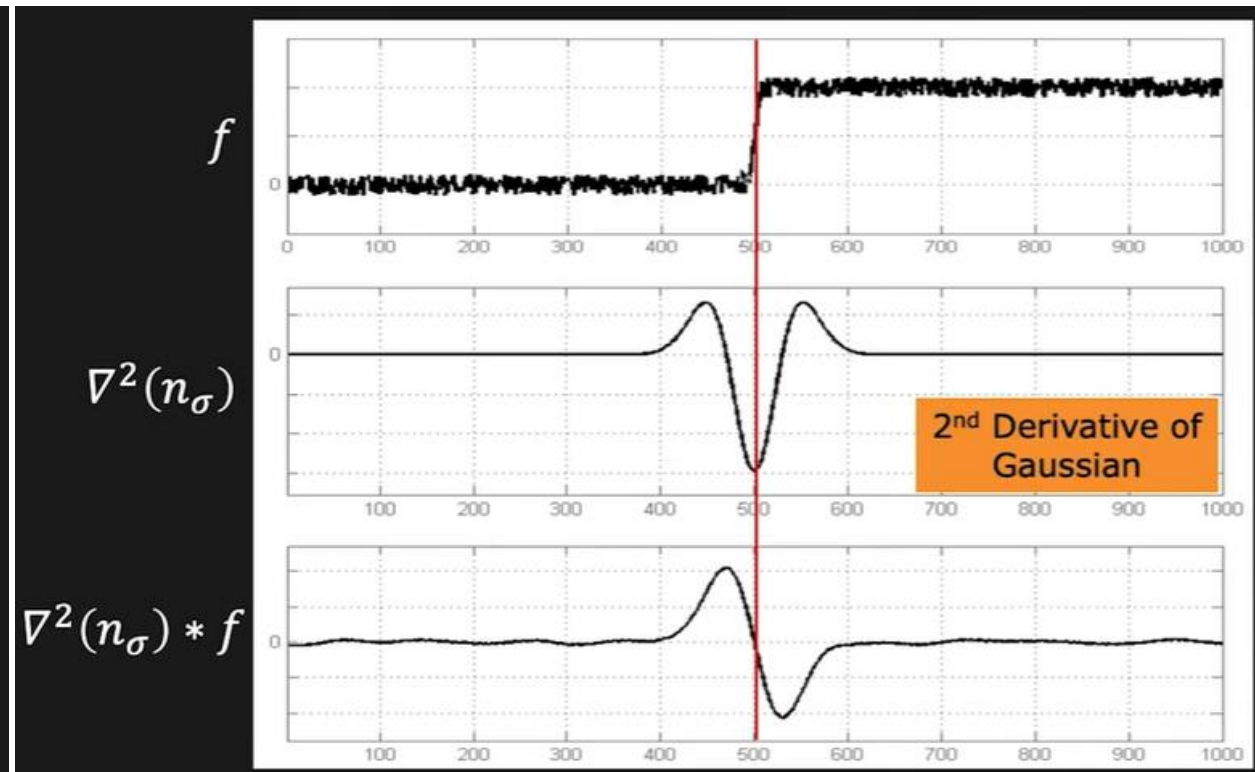
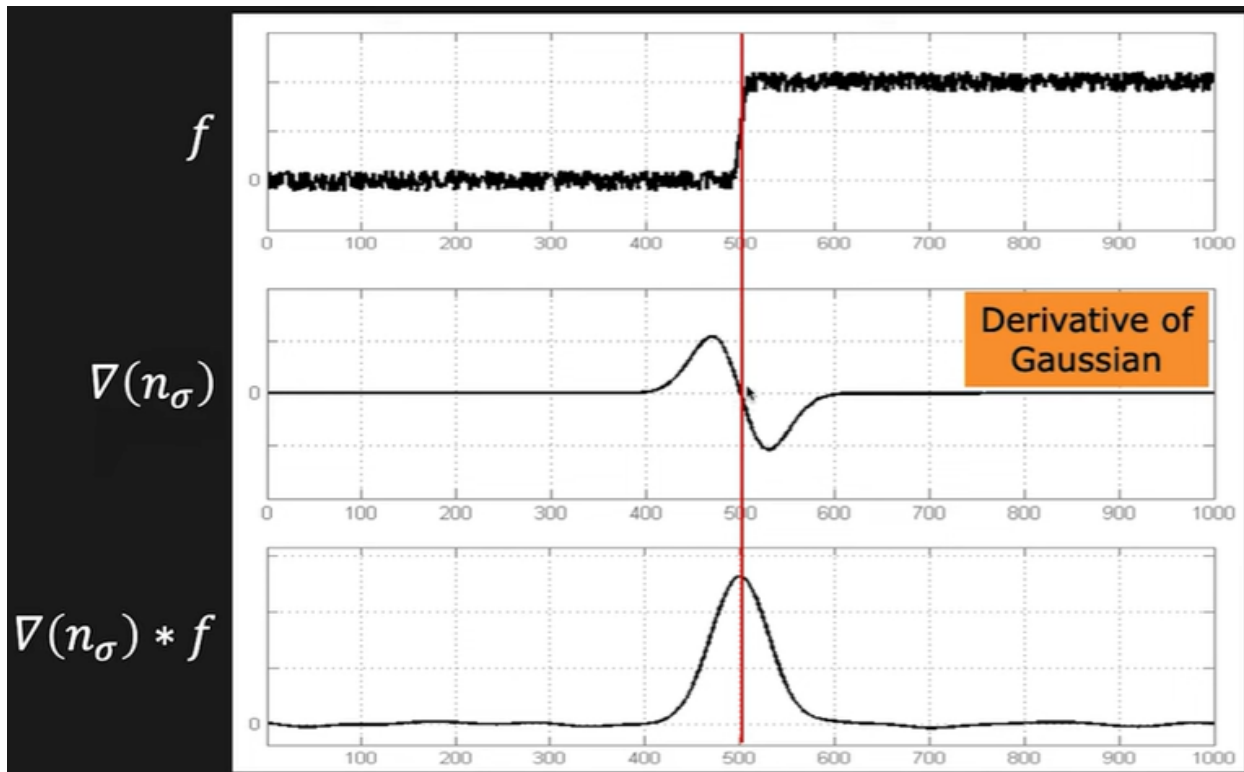
blobs are regions in an image that are **visually distinct from their surroundings**, meaning they are areas where pixel intensity (brightness or color) is relatively uniform inside the region but different from the outside.

They have a fix position and size



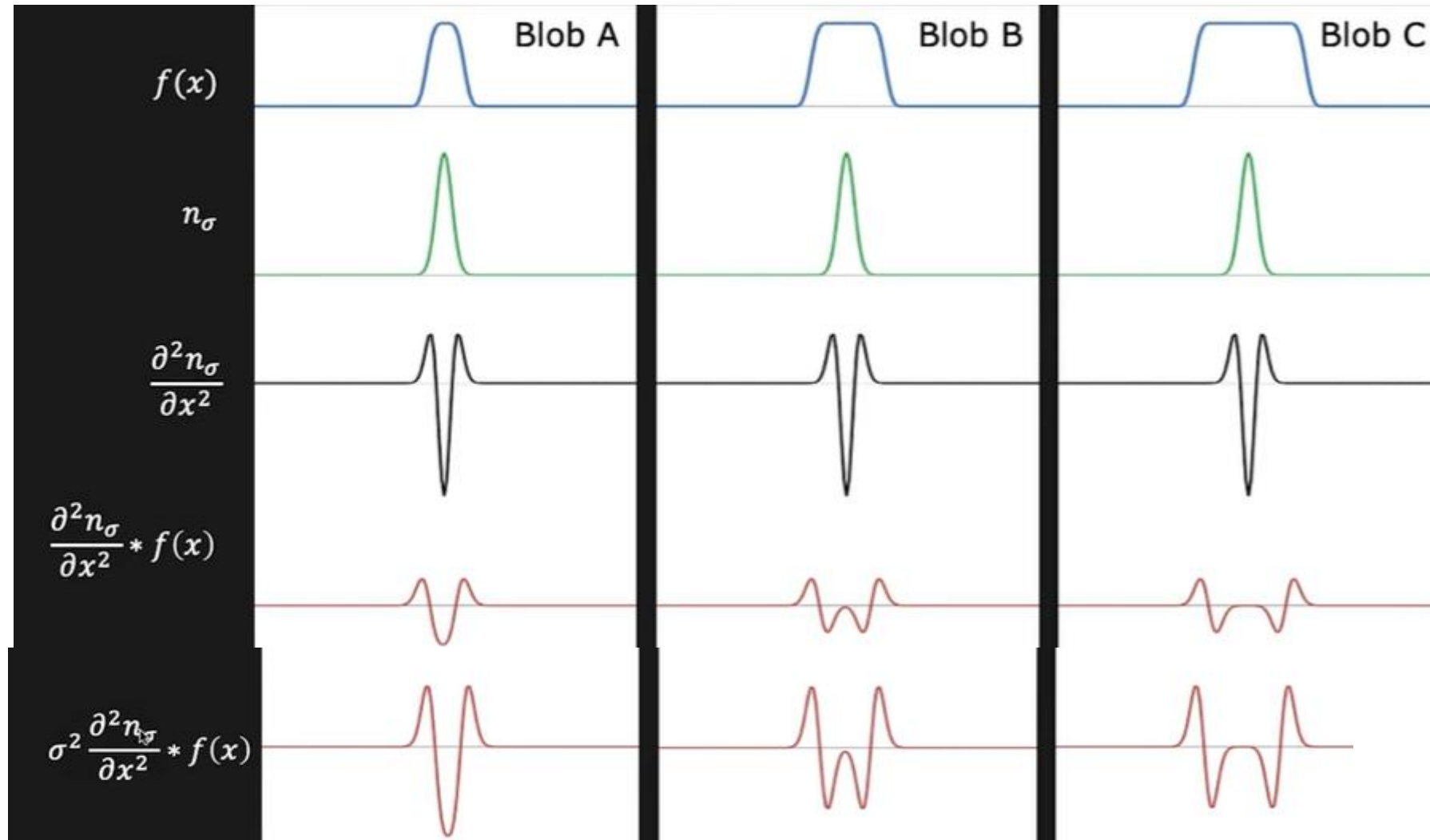
How to detect blobs

Reminder, how to find edge: first derivative or second derivative



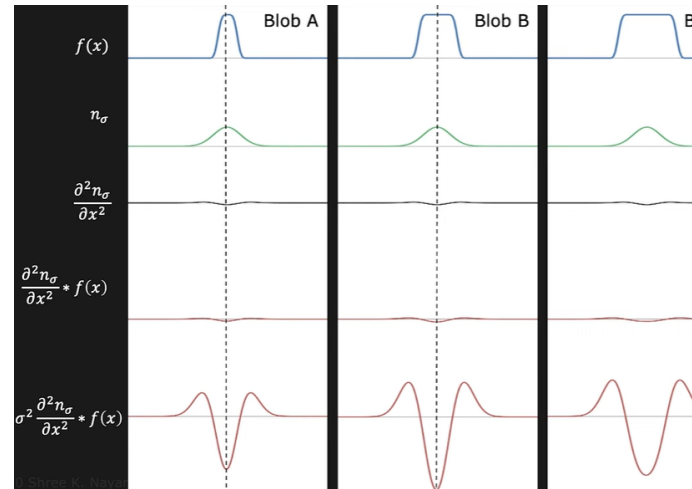
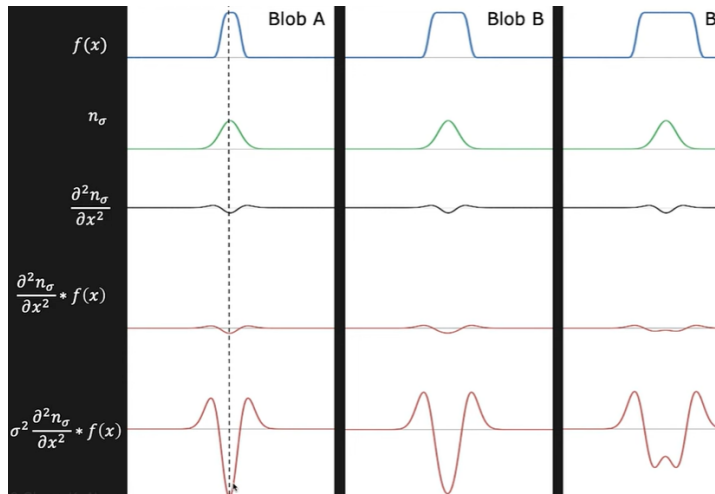
How to detect blobs

3 blobs, B=2A, C=3A

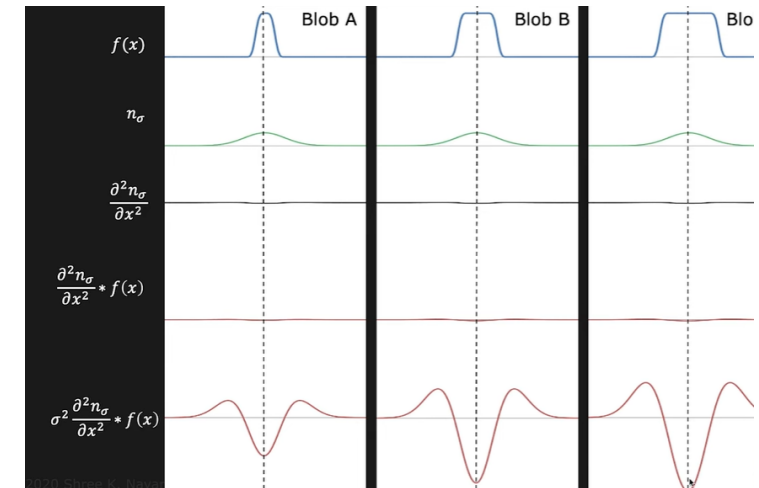


Response depends on sigma

INCREASE SIGMA ->



Detect blob B



Blob C

We have a sharp response for blob A, for B and C not, we did not find them

1D blobs

Given: 1D signal $f(x)$

Compute: $\sigma^2 \frac{\partial^2 n_\sigma}{\partial x^2} * f(x)$ at many scales $(\sigma_0, \sigma_1, \sigma_2, \dots, \sigma_k)$.

Find: $(x^*, \sigma^*) = \arg \max_{(x, \sigma)} \left| \sigma^2 \frac{\partial^2 n_\sigma}{\partial x^2} * f(x) \right|$

x^* : Blob Position

σ^* : Characteristic Scale (Blob Size)

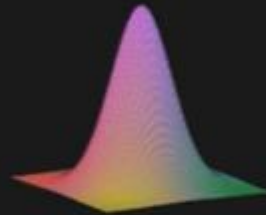
2D blobs

Normalized Laplacian of Gaussian (NLoG) is used as the 2D equivalent for Blob Detection.

Laplacian

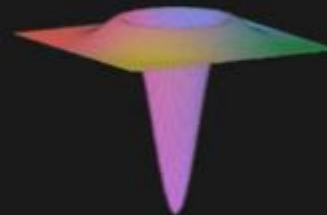
$$\nabla^2 = \frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2}$$

Gaussian



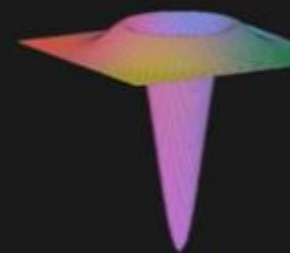
n_σ

LoG



$\nabla^2 n_\sigma$

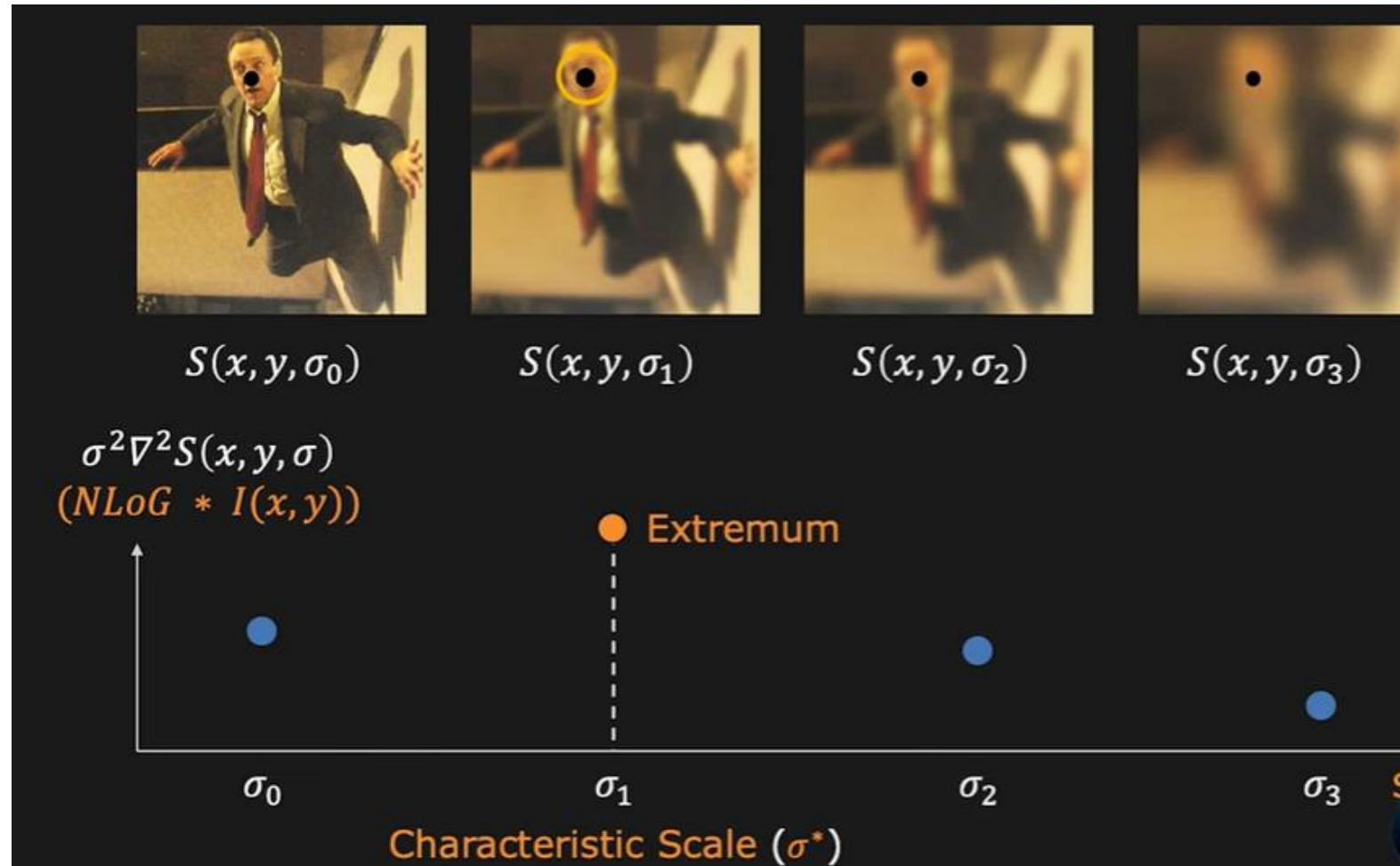
NLoG



$\sigma^2 \nabla^2 n_\sigma$

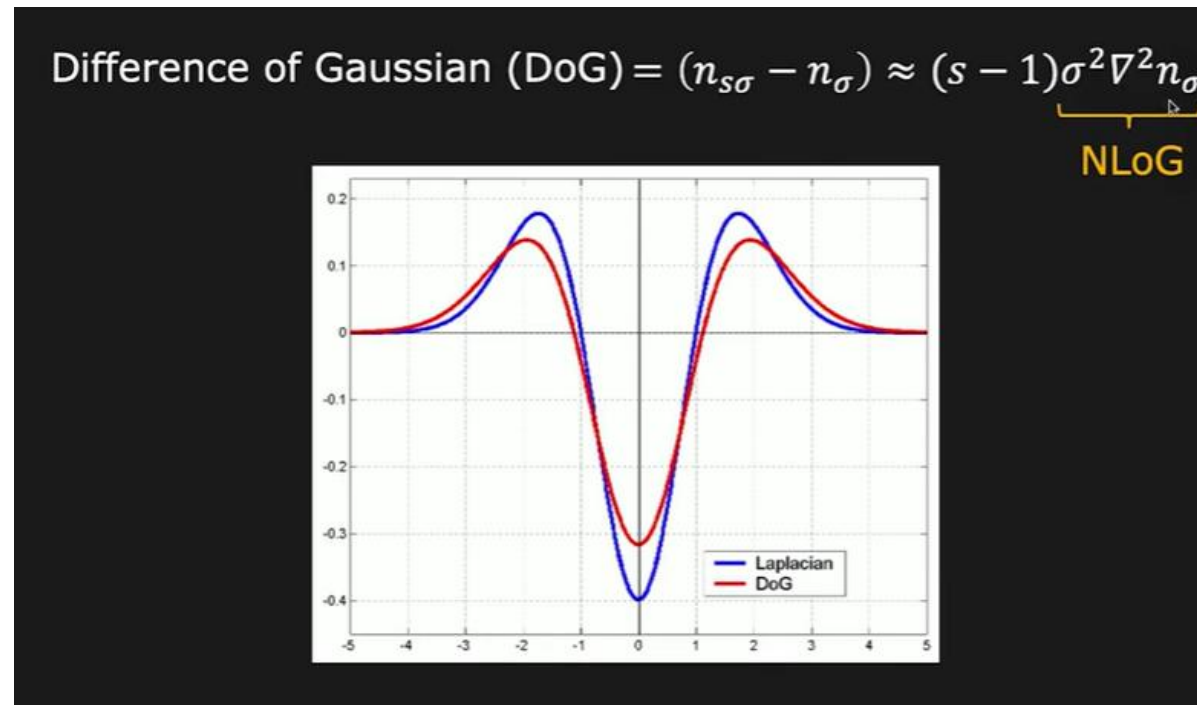
Location of Blobs given by **Local Extrema** after applying Normalized Laplacian of Gaussian at many scales.

2D blobs



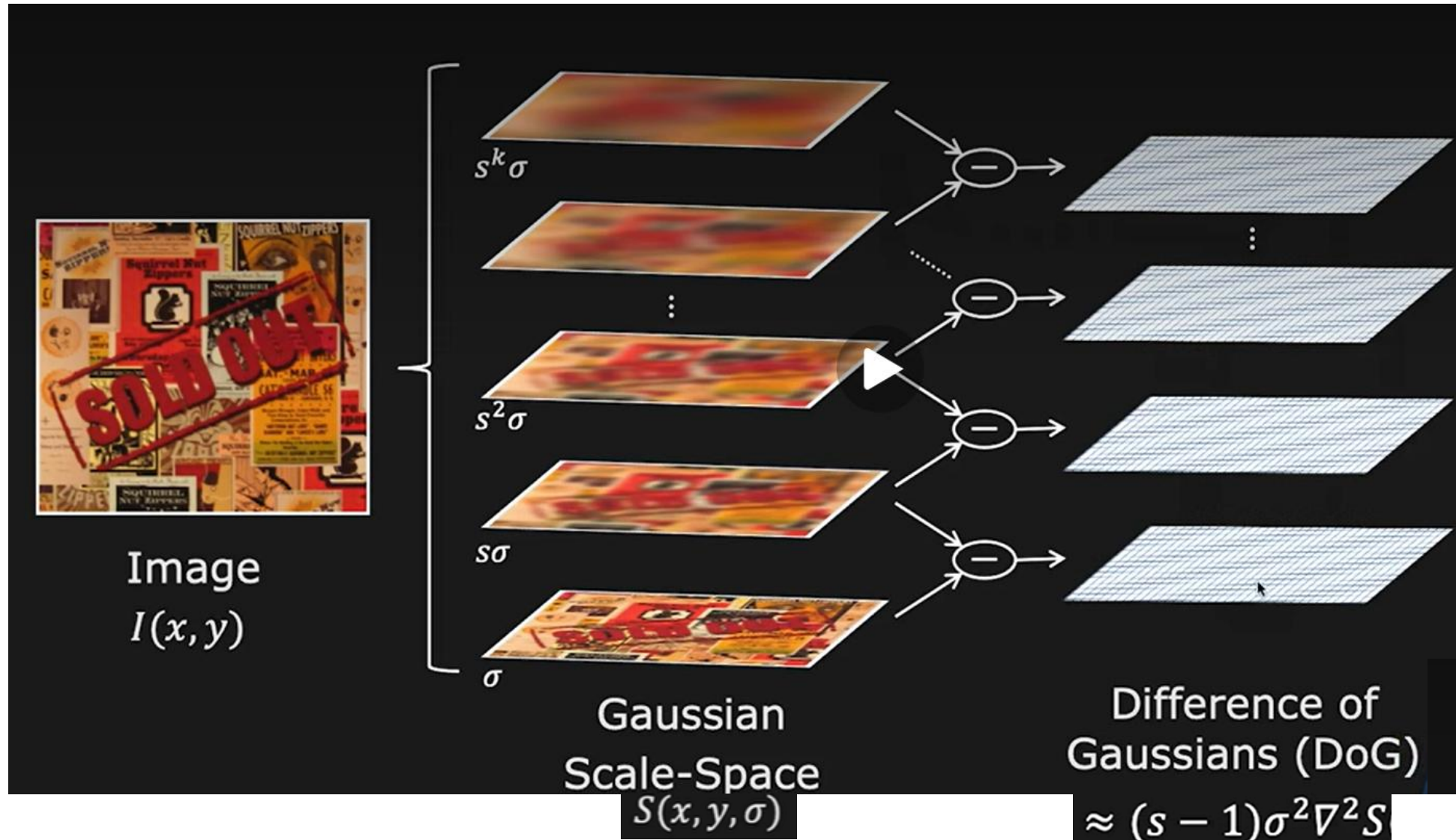
Difference of Gaussian (DOG)

The difference of two Gaussians is approximately the same as Normalized Laplacian of Gaussian



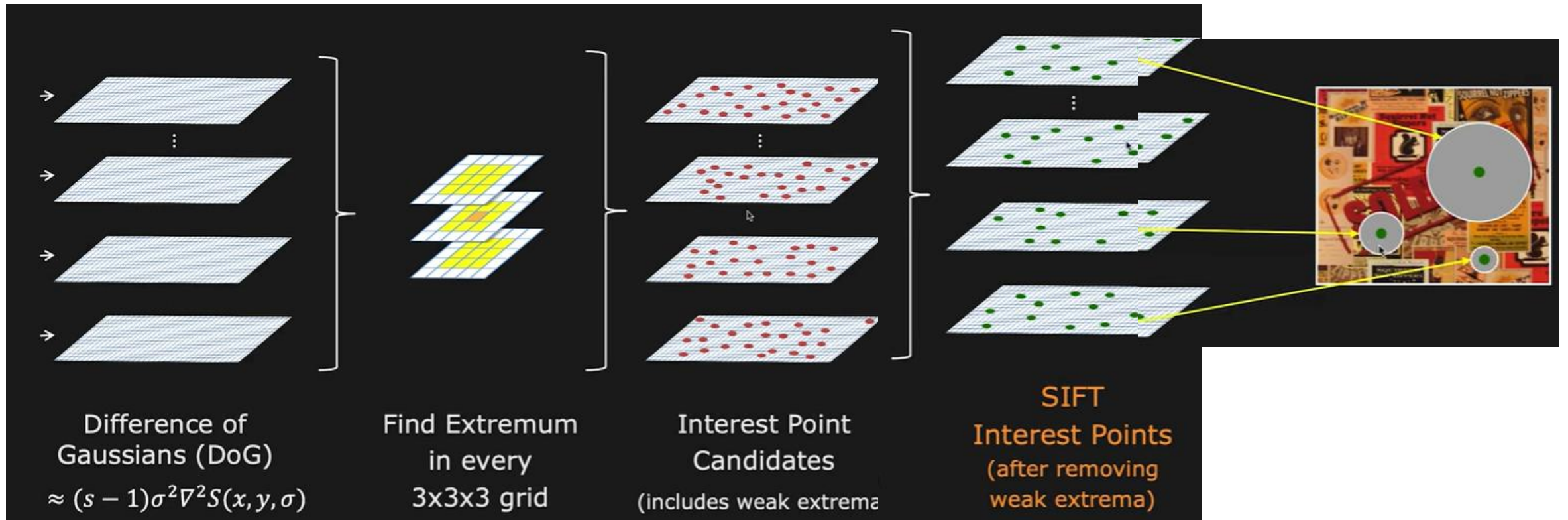
Instead of calculating the NLoG, you can take you stack of images with different sigma and subtract them

SIFT: Scale-Invariant Feature Transform



SIFT: Scale-Invariant Feature Transform

To find keypoints (extrema) or detect keypoints, each pixel in a blurred image is compared to its 8 neighbors in the current image and 9 neighbors in the scale above and below



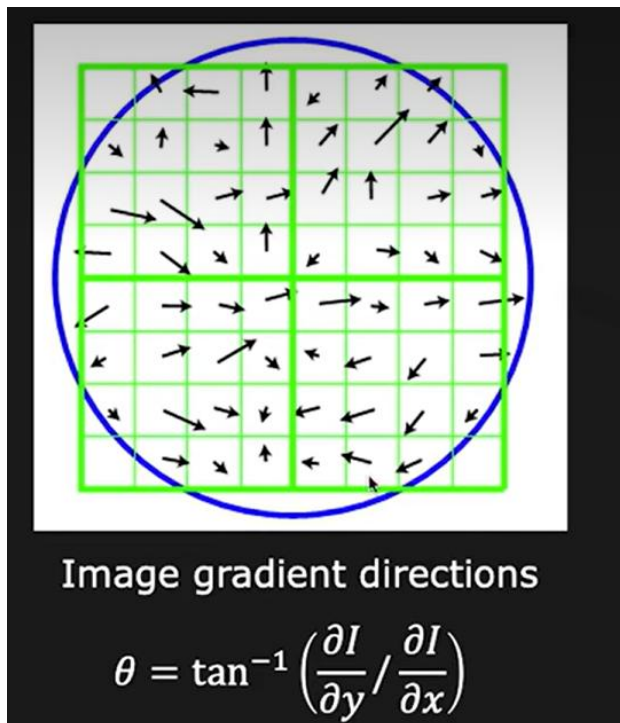
SIFT: Scale-Invariant Feature Transform



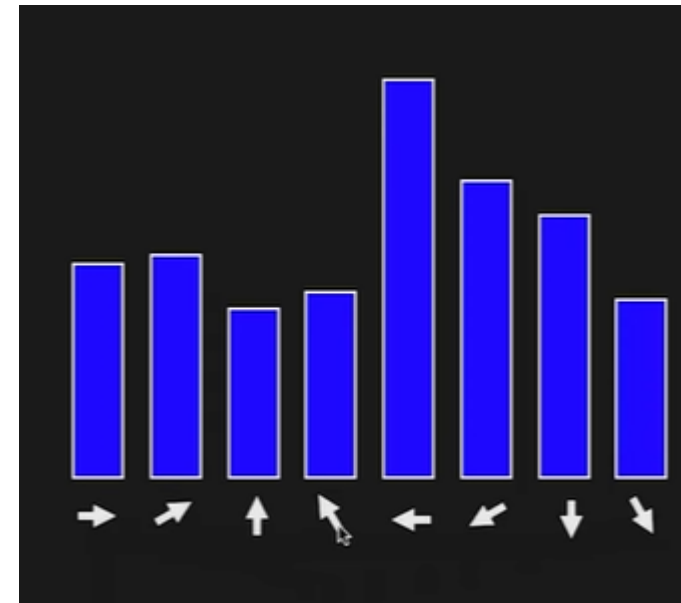
We need orientation!!

SIFT: Scale-Invariant Feature Transform

For each blob, create a grid of NxN and compute the gradient for every pixel (you have magnitude and orientations)



Create an histogram, and the peak is the principale orientation



SIFT: Scale-Invariant Feature Transform



SIFT: Keypoint descriptor

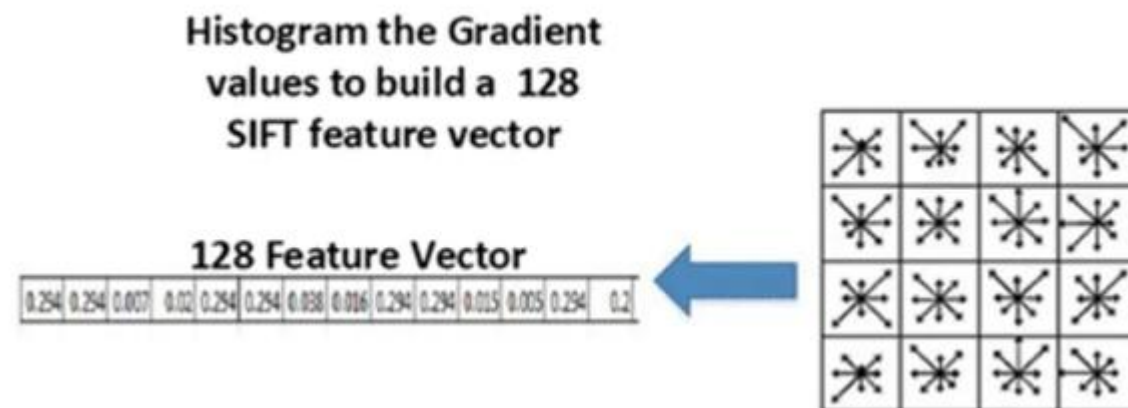
A descriptor is generally a vector of numerical values used to represent and characterize a keypoint. This step focuses on generating a robust descriptor for the region surrounding each detected keypoint, with the goal of making it resistant to variations such as illumination changes or different viewing angles.

To build the descriptor, the gradient magnitude and orientation are computed around the keypoint. A 16×16 neighborhood centered at the keypoint is selected and aligned according to the keypoint's dominant orientation to achieve rotation invariance.

This region is then divided into 16 smaller cells. For each cell, a Histogram of Oriented Gradients (HoG) with 8 orientation bins is calculated. The final descriptor is formed by concatenating all histogram values into a single feature vector, resulting in a descriptor of $16 \times 8 = 128$ dimensions.

keypoints from two images are matched by comparing their descriptors. Since each SIFT descriptor contains 128 values, every descriptor can be represented as a point in a 128-dimensional feature space.

For a descriptor D_i from image 1 and a descriptor D_j from image 2, the similarity between them is measured using the Euclidean distance



$$d(D_i, D_j) = \sqrt{\sum_{k=1}^{128} (D_{ik} - D_{jk})^2}$$

SIFT advantages

- Locality: features are local, so robust to occlusion and clutter (no prior segmentation)
- Distinctiveness: individual features can be matched to a large database of objects
- Quantity: many features can be generated for even small objects
- Efficiency: close to real-time performance
- Extensibility: can easily be extended to wide range of differing feature types, with each adding robustness

```
import numpy as np
import cv2 as cv
img = cv.imread('home.jpg')
gray= cv.cvtColor(img,cv.COLOR_BGR2GRAY)
sift = cv.SIFT_create()
kp = sift.detect(gray,None)
img=cv.drawKeypoints(gray,kp,img)
cv.imwrite('sift_keypoints.jpg',img)
```

ORB : Oriented FAST and Rotated BRIEF

What is ORB?

Ethan Rublee, Vincent Rabaud, Kurt Konolige and Gary R. Bradski: **ORB: An efficient alternative to SIFT or SURF** in 2011

ORB is rotation invariant and resistant to noise, making it suitable for images with varying orientations and reliable in real-world scenarios with noisy images. It combines two algorithms: FAST (for keypoint detection) and BRIEF (for descriptor extraction)

Process:

1. FAST: Keypoint Detection: For each pixel, check if a circle of neighboring pixels are all brighter/darker than the center.

- If yes, it's a corner (keypoint).

2. Harris Score for Ranking: ORB ranks FAST corners using a Harris corner measure and keeps the top N strongest points.

3. Orientation Assignment: For each keypoint, compute the intensity centroid in its neighborhood.

- The direction from the keypoint to this centroid gives its orientation.

4. BRIEF: Binary Robust Independent Elementary Features: generates a binary string for each patch:

Randomly selects pairs of pixels within the patch.

For each pair: if the first is brighter than the second, output a 1, else a 0.

5. Descriptor Matching: ORB descriptors are matched using Hamming distance (counting bit differences).

```
import numpy as np
import cv2 as cv
from matplotlib import pyplot as plt
img = cv.imread('simple.jpg',
cv.IMREAD_GRAYSCALE)
# Initiate ORB detector
orb = cv.ORB_create()
# find the keypoints with ORB
kp = orb.detect(img, None)
# compute the descriptors with ORB
kp, des = orb.compute(img, kp)
# draw only keypoints location, not size and orientation
img2 = cv.drawKeypoints(img, kp, None,
color=(0,255,0), flags=0)
plt.imshow(img2), plt.show()
```

Histogram of Oriented Gradients (HOG)

What is HOG?

HOG is a feature descriptor used in computer vision and image processing to detect objects by capturing edge and gradient structures.

- Converts image regions into histograms of gradient orientations.
- Used in: pedestrian detection, face detection, car detection, etc.

Process:

1. **Convert the image to grayscale** and apply normalization
2. **Gradient Computation:** Calculate the gradient and magnitude of the image using techniques like Sobel operators.

$$G_x = I(x+1, y) - I(x-1, y), \quad G_y = I(x, y+1) - I(x, y-1) \quad \text{Magnitude: } M = \sqrt{G_x^2 + G_y^2}, \quad \text{Orientation: } \theta = \arctan \frac{G_y}{G_x}$$

3. **Cell division:** Divide the image into small cells (e.g., 8x8 pixels) to capture local gradient information.

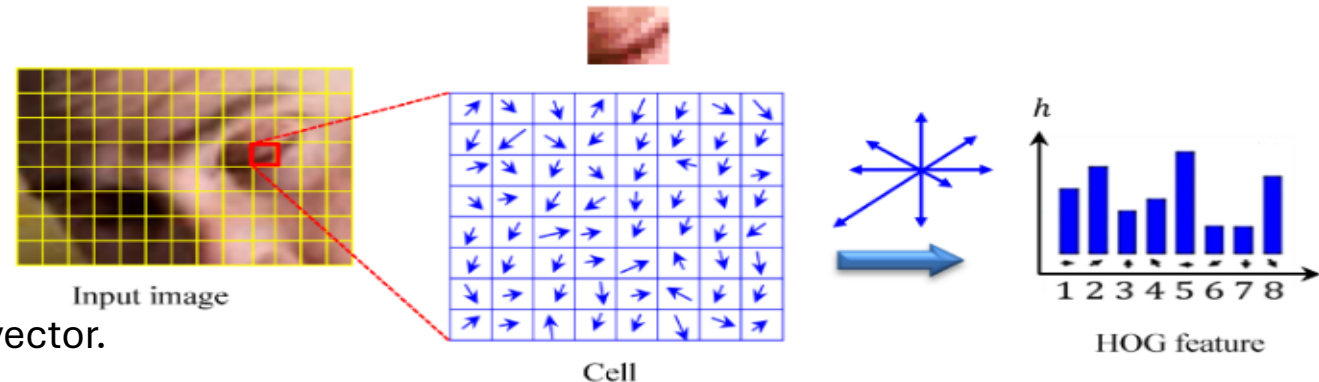
4. **Orientation Histograms:** For each cell:
 - Create a histogram of gradient orientations.
 - Each pixel votes for a bin using its magnitude.

5. Block Normalization:

- Group cells into blocks (e.g., 2x2 cells).
- Normalize the histograms in each block to reduce lighting/shadow effects

5. Normalized vector Feature Vector Construction:

Concatenate all normalized histograms into one long feature vector.



Segmentation

Definition: Process of partitioning an image into multiple segments or regions without prior labeling.

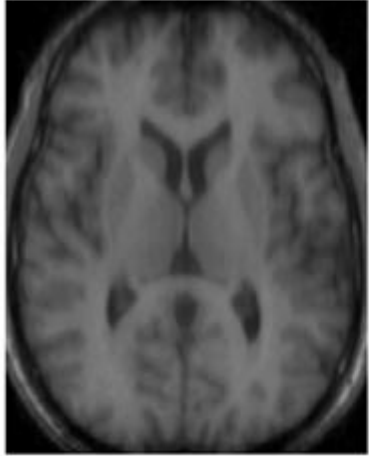
Purpose: The main goal is to identify and isolate objects or regions within an image (object detection, image analysis, and scene understanding or to separate foreground/background)

Methods: Common techniques include:

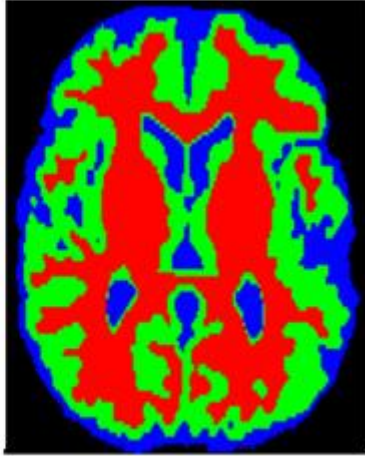
- Thresholding: Simple, based on pixel intensity
- Clustering (K-means): based on pixels color/intensity
- Watershed: Splits touching/overlapping objects.

Output: The result of segmentation is typically a set of labeled regions or segments within the image, where each segment corresponds to a distinct object or area of interest.

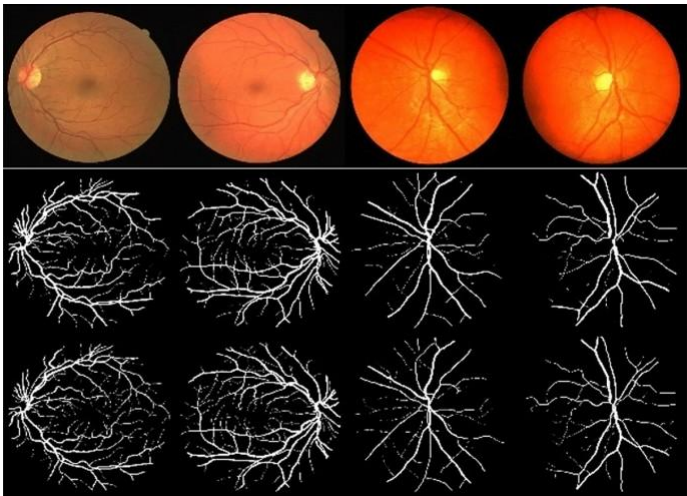
Segmentation



(a) coupe axial



(b) segmentation des tissus



Segmentation: Binary images

It is the simple type of images you can use in computer vision:
binary images

Can have only have two values: **0** or **1**.

Simple to process and analyze



Segmentation: Binary images

How do we make binary images $b(x,y)$?

- Take a gray scale image: $g(x,y)$
- Choose a threshold value T .
- Any pixel value $> T$ becomes white (foreground/object).
- Any pixel value $\leq T$ becomes black (background).

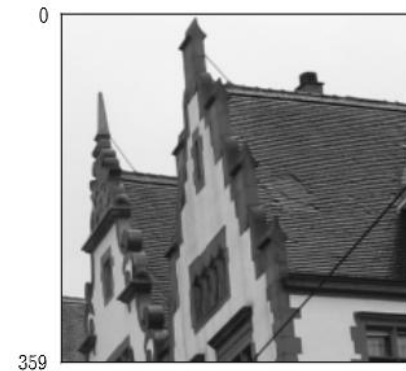
$$b(x,y) = \begin{cases} 0, & g(x,y) < T \\ 1, & g(x,y) \geq T \end{cases}$$



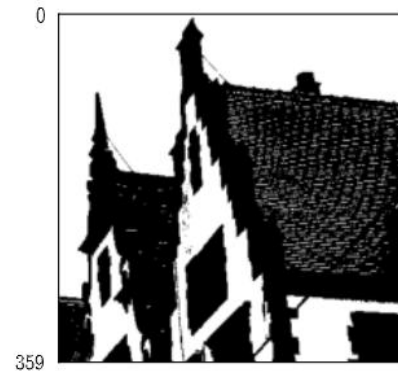
Original Image



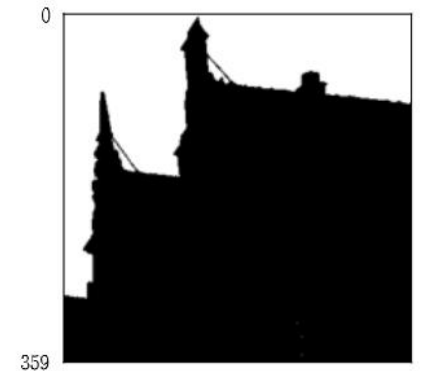
Thresholded and segmented Image



Original image



$T = 150$

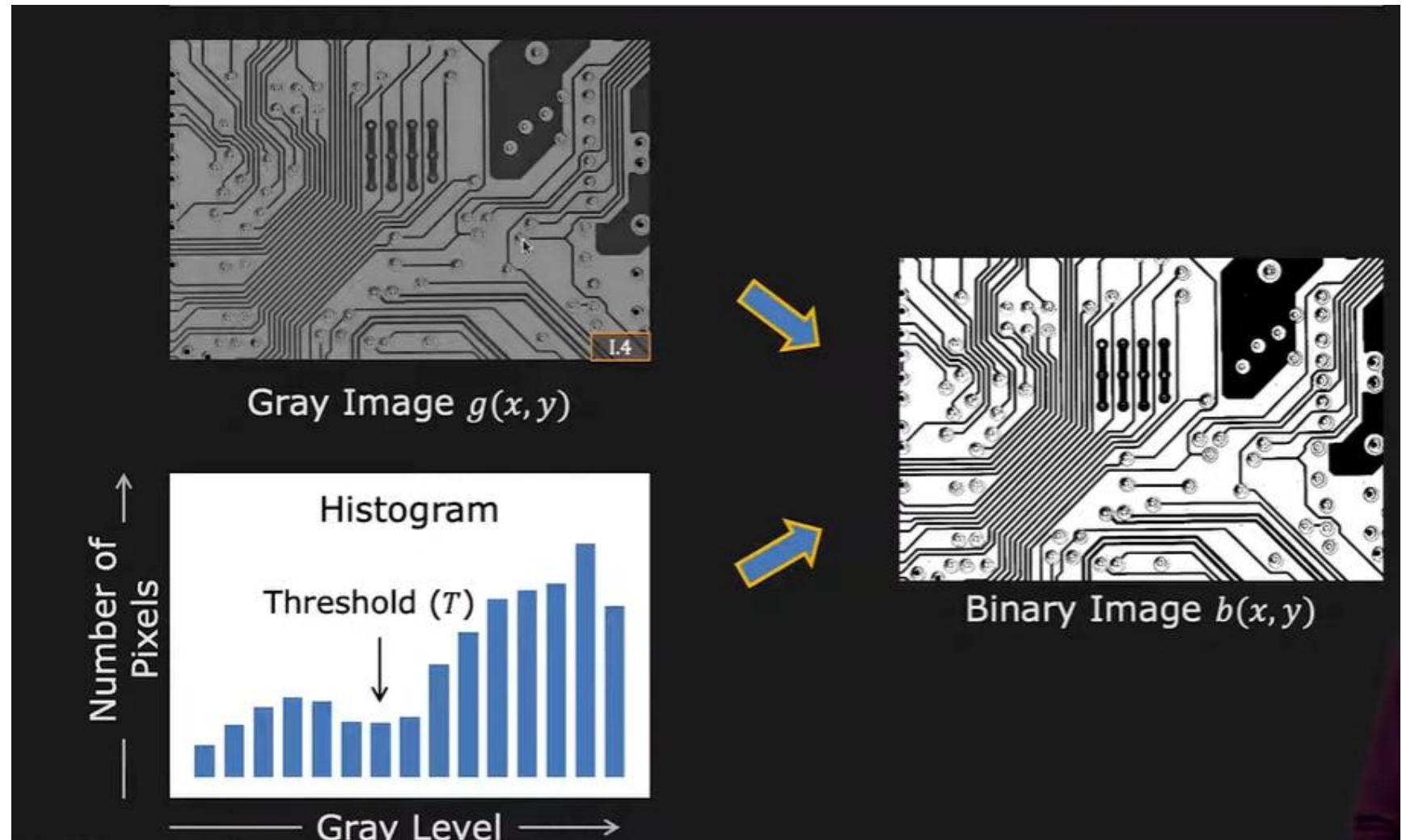


$T = 200$

Segmentation: How to select the threshold

How do you set the threshold?

- Compute the histogram
Gray level from 0-255
- From the histo, you see 2 parts, maybe one for background and other the objects
- The gray level corresponding the valley between the 2 parts will be a good choice!!



Segmentation: Otsu's method

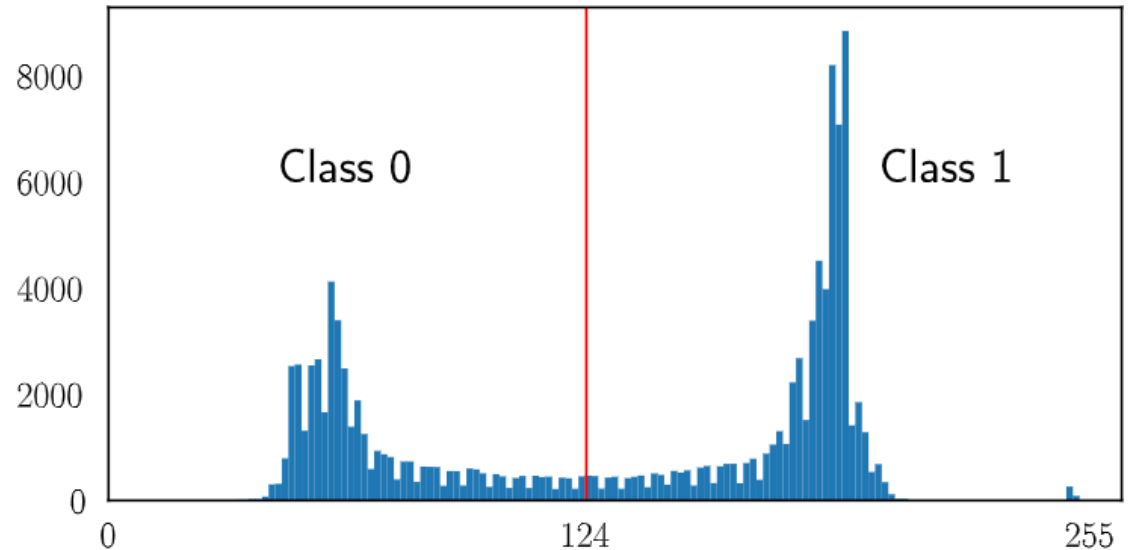
A problem with simple thresholding is that you have to manually specify the threshold value

We have a way to automatically determine the threshold: the Otsu's technique

- Divides the histogram of the images in two groups, namely class 0 and class 1
- Each group has a number of pixel with $w_i(T)$ with mean $\mu_i(T)$ and variance $\sigma_i^2(T)$.

, where i is the group index (0 or 1). Otsu's computes the threshold which minimizes the intra-class variance

$$\sigma_w^2(T) = w_0(T)\sigma_0^2(T) + w_1(T)\sigma_1^2(T).$$



Segmentation: binary

Use Cases:

- Scanning/Document binarization
- Separating objects from simple backgrounds
- Preprocessing for contour detection

Variants:

- Global Threshold: Same T for the whole image (simple, fast).
- Adaptive Threshold: T can vary locally (good for uneven lighting).
- Otsu's Method: Algorithmically finds “best” T to separate two peaks in the image histogram.



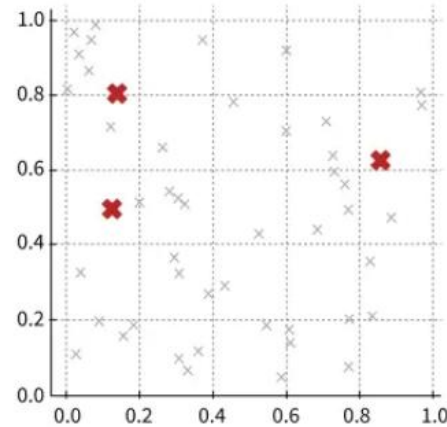
Segmentation: K-means

K-Means Clustering groups similar data points into clusters without needing labeled data.

Choose Initial Centroids

Centroids are randomly chosen from the data points. These represent the initial cluster centers.

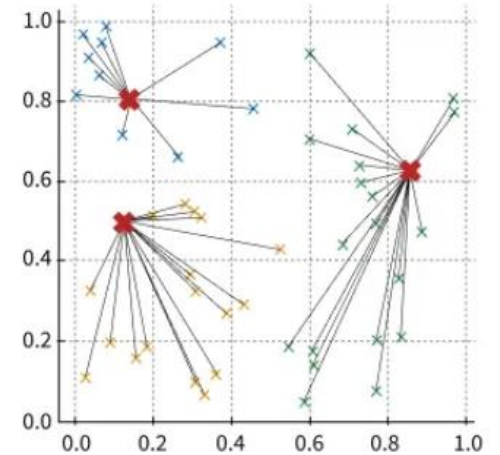
✖ Centroids ✕ Data Points



Assign Points to Nearest Centroid

Each point is assigned to the nearest centroid, forming clusters

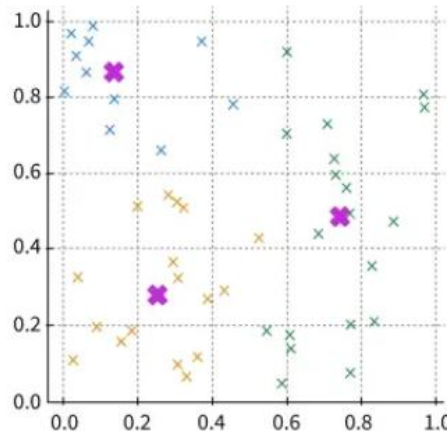
✖ Centroids ✕ Cluster 1
✕ Cluster 2 ✕ Cluster 3



Update Centroids

Centroids are recalculated as the mean of the points in each cluster

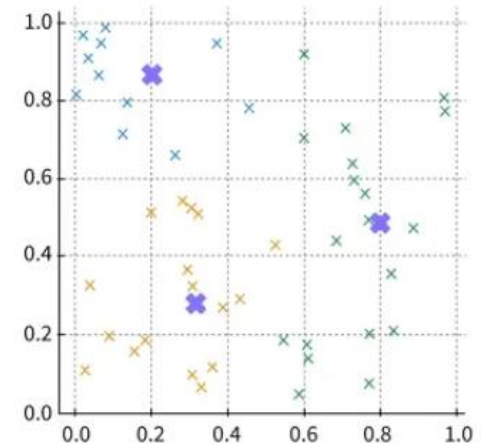
✖ New Centroids
✕ Cluster 1
✕ Cluster 2
✕ Cluster 3



Repeat Until Convergence

This process repeats until the centroids stabilize and do not move further.

✖ Final Centroids
✕ Cluster 1
✕ Cluster 2
✕ Cluster 3



Segmentation: K-means

Goal: The output is a “simplified” image, where each pixel takes on the color of its cluster center.

Workflow:

- Reshape the image into a list of pixels (each as a feature vector).
- Apply K-means to assign a cluster label to each pixel.
- Replace each pixel with its cluster center color.

Advantages:

- Works well for images with distinct color/texture regions.
- Useful for image “posterization” and pre-segmentation.



Limitations:

- Sensitive to K value (number of clusters).
- May not separate touching objects or handle uneven lighting well.

Typical Applications:

Color quantization, object/background separation, preprocessing

Segmentation: Watershed

What is the Watershed Algorithm?

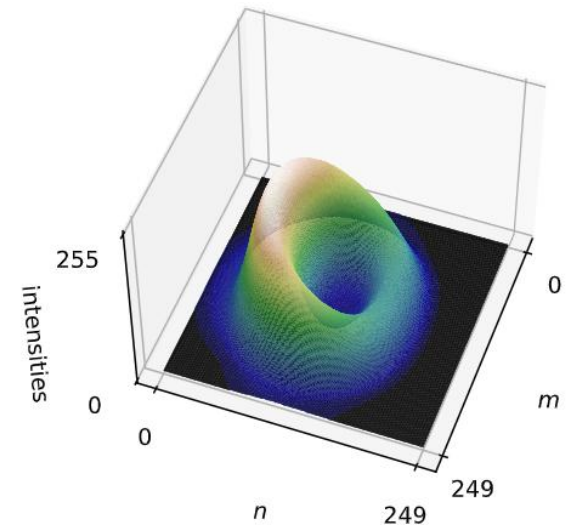
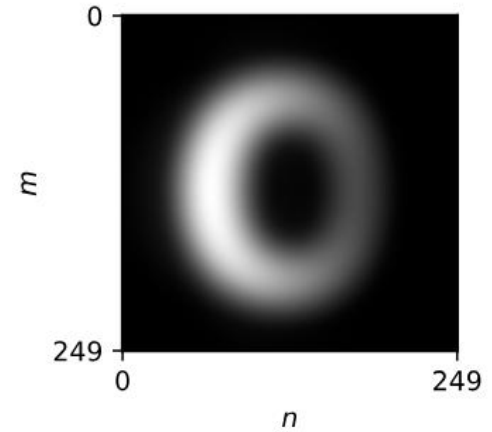
A region-based segmentation technique, especially powerful for separating overlapping or touching objects.



Segmentation: Watershed

How does watershed Algorithm work?

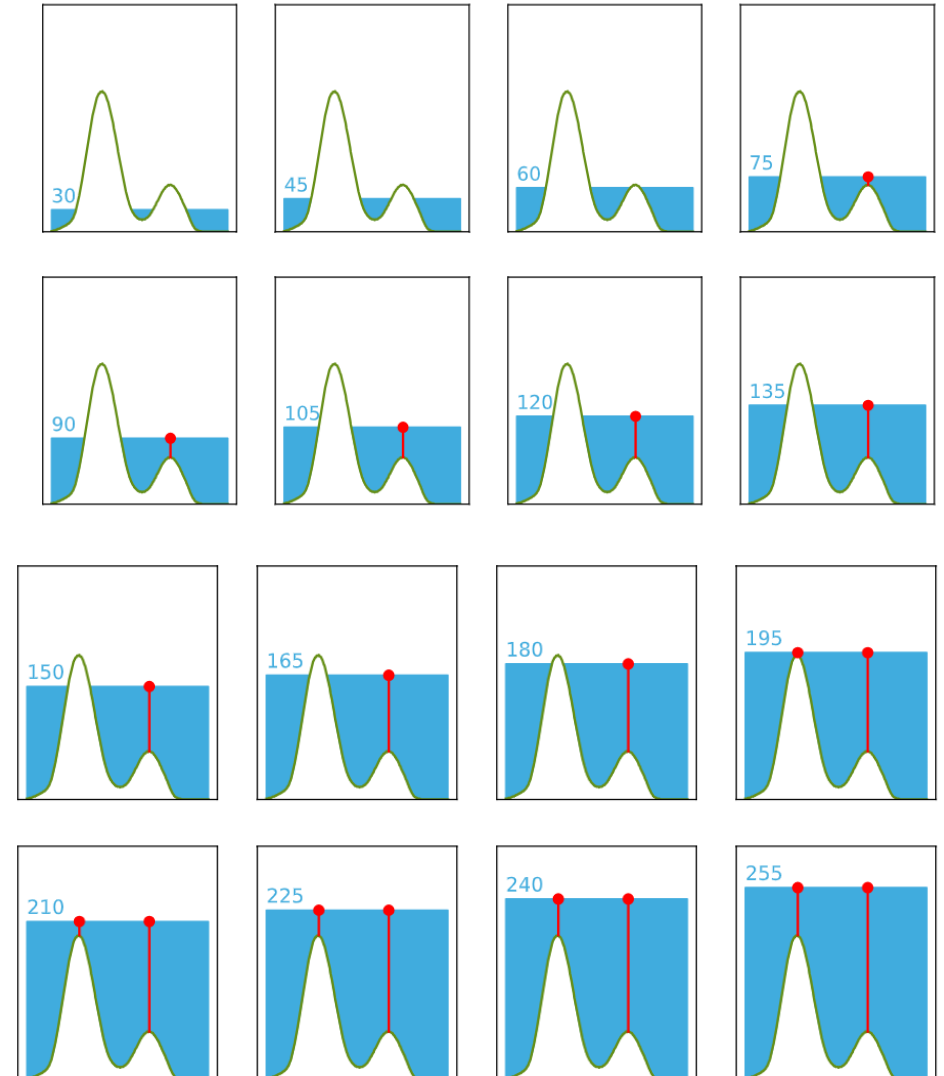
- Treat the grayscale image as a topographic surface (mountains/valleys).



Segmentation: Watershed

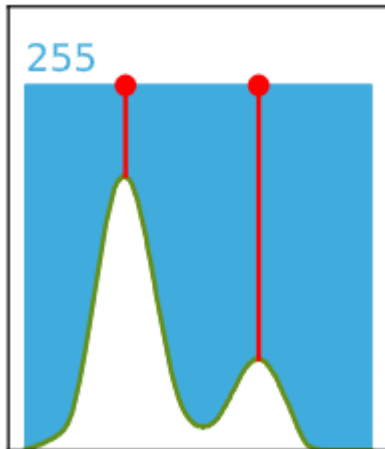
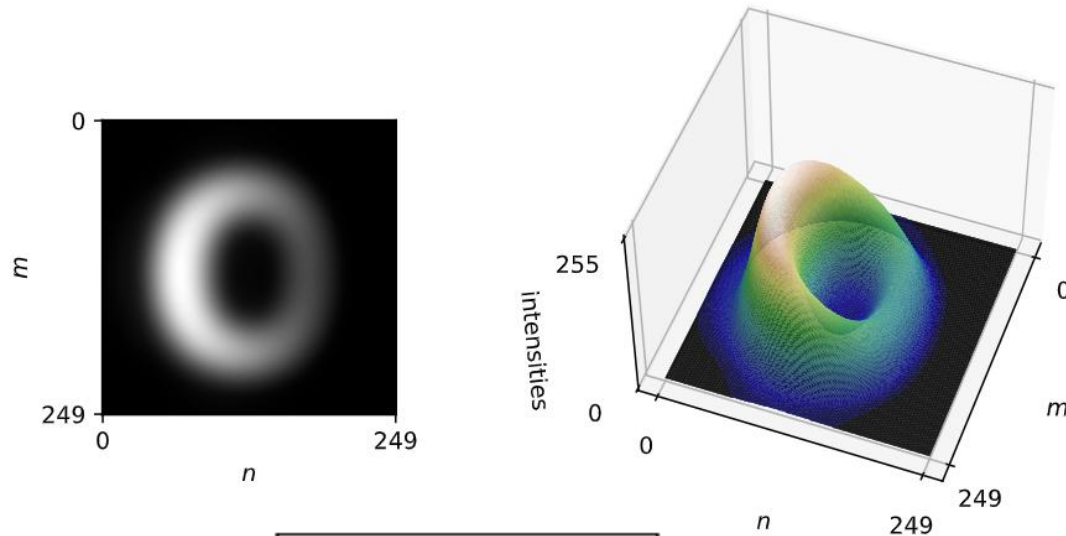
How does watershed Algorithm work?

- Treat the grayscale image as a topographic surface (mountains/valleys).
- The basic idea of watershed segmentation is to flood the topographic map and letting water rise through the valleys at a uniform rate.
- When the rising water of two distinct valleys is about to merge, a dam is built to prevent the merging.

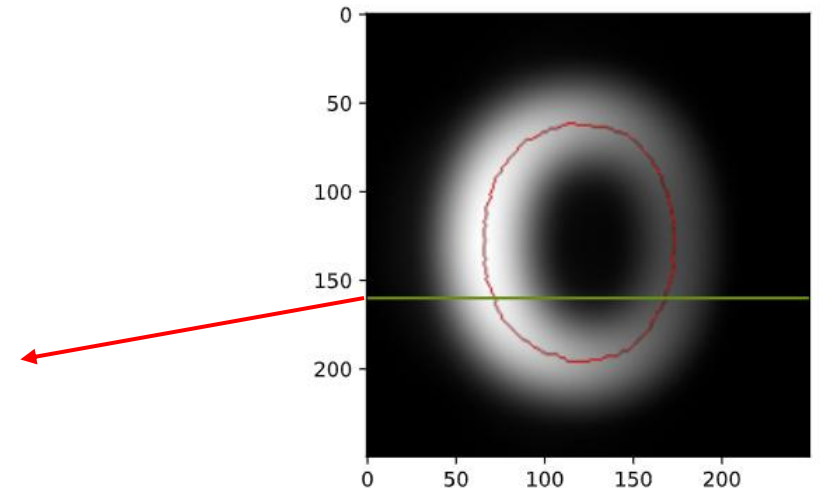


Segmentation: Watershed

How does watershed Algorithm work?



The profile considered in draw is plotted in green.



Segment Anything Model (SAM)

What is SAM?

- Deep learning model by Meta AI (2023)
- Designed to “segment anything” in any image, with little or no manual guidance
- Trained on 11 million images, 1.1 billion masks

How does it work?

- You can click, draw a box, or prompt text to tell it “what” to segment.
- SAM instantly returns a high-quality mask for the object or region.

